Non-Linear Filtering for Vision-and-Telemetry-Based Target Tracking

Mathematical Foundations of the IMM-CKF Architecture

Huiyuan Zang

Advanced Concepts, EDGE Group

First Edition
June 2026

First Edition: June 2026

Contents

List of Tables

List of Figures

Preface

The Kalman Filter (KF) is arguably the most important algorithm in modern estimation theory. At its core, it is an elegant mathematical framework designed to extract the true state of a system from a series of incomplete and noisy measurements. In a perfectly linear world with Gaussian noise, the standard KF is the optimal estimator.

However, modern tactical tracking systems do not operate in a linear world.

In Real-World Vision-and-Telemetry-Based application, tracking has traditionally relied on 2D pixel-space bounding boxes generated by visual sensors. While sufficient for rudimentary camera-following, a 2D bounding box is physically meaningless for a real-world multi-sensor fusion system because pixels lack physical scale. A drone at 100 meters and an airliner at 10,000 meters can occupy the exact same number of pixels. To estimate actual target kinematics—true velocity, physical size, and metric trajectory—the tracking system must project 2D detections into a 3D Earth-fixed coordinate system, such as a North-East-Down (NED) frame.

This projection is where textbook mathematics collide with engineering reality. Mapping a 2D pixel backwards through nested Euler rotation matrices (representing the Gimbal, Mount, and UAV chassis) and a pinhole camera's perspective divide introduces severe mathematical nonlinearities.

The Evolution of Non-Linear Filtering

To handle systems where the measurement function is non-linear, engineers traditionally turn to variations of the Kalman Filter, though each comes with distinct failure modes in high-stress environments:

- **The Extended Kalman Filter (EKF)**: The EKF attempts to linearize the system by calculating the Jacobian (the first-order derivative) of the measurement function. However, deriving the analytical Jacobian of nested trigonometric rotation matrices coupled with perspective division is computationally fragile. During high-agility target maneuvers, the EKF's Taylor series approximation frequently fails, leading to rapid linearization error and track divergence.
- **The Unscented Kalman Filter (UKF)**: The UKF avoids Jacobians by propagating a deterministic set of sigma points through the non-linear function. While an improvement, the UKF requires the manual tuning of scaling parameters. In higher-dimensional state spaces subjected to extreme measurement noise, the central sigma point can develop a negative weight. This frequently causes the state covariance matrix to lose positive definiteness, resulting in a catastrophic numerical crash (NaN output) on edge hardware.
- **The Cubature Kalman Filter (CKF)**: The CKF resolves these issues by utilizing a third-degree spherical-radial cubature rule. It requires zero manual tuning parameters and mathematically guarantees strictly positive weights for all propagated points. This ensures maximum numerical stability, making it the premier choice for visual coordinate projection.

The Need for the IMM Architecture

Even with a stable non-linear filter, a single kinematic model cannot track all target types. For instance, a tracker using a purely 3D model on a ground target will misinterpret inherent bounding-box pixel jitter as vertical velocity, causing the mathematical track to "drift" into the sky or sink underground. Conversely, a constrained 2D ground model will completely fail to track an airborne target.

Furthermore, modern visual AI sensors act as noise multipliers. Bounding boxes inherently vibrate frame-to-frame. If an engineer implements a standard 11-state Constant Acceleration (CA) model, this high-frequency pixel jitter is mathematically interpreted as violent, instantaneous acceleration. The filter amplifies this noise, causing the predicted target state to vibrate uncontrollably until the track is completely lost.

The Interacting Multiple Model (IMM) architecture solves this by running multiple discrete mathematical models in parallel (e.g., a Ground model enforcing a strict altitude constraint, and an Airborne Bearings-Only model). The IMM dynamically shifts probability weights between these models based on how accurately their internal predictions match the incoming visual measurements.

Engineering Example 1: The Domain Transition (Liftoff)

When a drone is stationary on the tarmac, the IMM assigns maximum probability weight to the constrained Ground model. The moment the drone takes off, the 2D bounding box moves vertically in the video frame. The Ground model, mathematically insisting the altitude is zero, registers a massive prediction error (Innovation), and its calculated likelihood collapses. Simultaneously, the Airborne model correctly predicts the vertical movement. Governed by a Markov transition matrix, the IMM instantly transfers the tracking weight to the Airborne model, seamlessly unlatching the target from the Earth without losing the tracking ID.

Engineering Example 2: Multispectral Sensor Toggling

Consider an interceptor toggling its payload from a high-resolution 1080p visible sensor to a 480p Infrared (IR) sensor. The physical target kinematics do not change, but the measurement uncertainty space is drastically altered. IR microbolometers suffer from "thermal blooming," where thermal inertia causes the heat signature of a fast-moving target to smear across pixels, resulting in severe bounding box fluctuation. To prevent the tracking filter from chasing this noise—which would cause mechanical whiplash in the gimbal—the IMM-CKF dynamically injects an inflated measurement covariance scalar during the hardware toggle, forcing the filter to momentarily distrust the visual edges and rely on its internal kinematic predictions.

This book bridges the gap between pure mathematics and these operational realities. We will build the IMM-CKF from the ground up, proving every equation, and translating those proofs into robust C++ architectures designed for strict Size, Weight, and Power (SWaP) constrained hardware.

Acknowledgments

Writing a book that bridges the gap between pure mathematical theory and strict engineering reality is not a solitary endeavor. It is built upon the shoulders of giants and refined through the patience of peers and family.

First, my deepest gratitude goes to the mathematicians and engineers who laid the foundation for modern state estimation. From Rudolf E. Kálmán's original masterpiece, to the pioneers of non-linear filtering, and the architects of the cubature rule—thank you. Your theoretical brilliance has allowed generations of engineers to pull order from chaos. I also want to thank the wider aerospace and software engineering community, whose relentless pursuit of robust tracking systems under extreme constraints inspired the pragmatic approach of this book.

I am immensely grateful to the reviewers, colleagues, and early readers who dedicated their time to comb through the manuscript. Translating high-level mathematics into production C++ code is a complex task, and your rigorous feedback, mathematical debugging, and code reviews were instrumental in catching errors and refining the proofs. Your dedication has made this a far more accurate and valuable guide. Any errors that remain are entirely my own.

Finally, and most importantly, I want to thank my family. To my wife, Jinjing: thank you for your unwavering support throughout this demanding process. You constantly encourage me, ground me, and push me to be a better person every single day. And to my daughter, Halley: you are the angel of my life. Your bright spirit and joy enlighten my world, reminding me of the true purpose behind the work we do. This book is as much yours as it is mine.

Part I: The Mathematical Engine Room

Author's Note: Building the Toolbox

If you are an engineer looking at the integrals, partial derivatives, and series expansions in the upcoming pages and feeling a sense of mathematical dread, take a breath. This first section of the book exists simply to establish your foundational "toolbox."

You do not need to perfectly memorize how to analytically solve a multivariate Jacobian or calculate a spherical-radial integral right now. In the later chapters on the Extended (EKF), Unscented (UKF), and Cubature Kalman Filters (CKF), we will pull these tools out one by one. When we do, we will provide complete, step-by-step mathematical proofs explaining exactly how and why they are used to process real sensor data, alongside the C++ architectures to implement them on edge hardware.

For now, as you read through these mathematical prerequisites, simply focus on understanding the fundamental purpose of each tool and why a tracking system might need it.

Chapter 1

Advanced Calculus & Optimization

Before a tracking system can estimate a target's state, it must understand how that state changes over time and how it relates to the sensors observing it. In linear systems, these relationships are simple algebraic multipliers. In the real world of visual sensors and maneuvering targets, these relationships are highly non-linear, requiring advanced calculus to break them down into computable components.

1.1 Partial Derivatives and the Jacobian Matrix

The Mathematical Concept

For a single-variable function $y = f(x)$, the derivative $f'(x)$ gives the rate of change. However, tracking filters operate on multi-dimensional state vectors (e.g., 3D position, velocity, and acceleration). For a multi-variable vector function $\mathrm{y} = \mathrm{h}(\mathrm{x})$, where $\mathrm{x} \in \mathbb{R}^n$ and $\mathrm{y} \in \mathbb{R}^m$, we must calculate the partial derivative of every output with respect to every input.

This forms the **Jacobian Matrix** (J):

$$J = \begin{bmatrix} \frac{\partial h_1}{\partial x_1} & \cdots & \frac{\partial h_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial h_m}{\partial x_1} & \cdots & \frac{\partial h_m}{\partial x_n} \end{bmatrix}$$

Application in Target Tracking (The EKF)

The Extended Kalman Filter (EKF) relies entirely on the Jacobian matrix. When a camera observes a drone, the measurement function $\mathrm{z} = \mathrm{h}(\mathrm{x})$ maps the target's 3D real-world coordinates into 2D pixel space. Because this involves perspective division and trigonometric rotations, it is highly non-linear. The EKF uses the Jacobian $H_k = \frac{\partial \mathrm{h}}{\partial \mathrm{x}}$ evaluated at the current state estimate to linearize the function, allowing the filter to project the state covariance matrix into the measurement space[2]:

$$S_k = H_k P_k H_k^T + R_k$$

The Engineering Dilemma

Deriving the analytical Jacobian for a nested camera projection matrix (UAV chassis → gimbal → camera pinhole) results in massive, computationally fragile trigonometric chains. On SWaP-constrained edge devices, evaluating these long sine/cosine chains for every target, every frame, burns critical CPU cycles. Furthermore, if the target maneuvers rapidly, the first-order Taylor series approximation of the Jacobian fails, causing catastrophic linearization error and track divergence.

1.2 Multivariate Taylor Series Expansion

The Mathematical Concept

In engineering mathematics, it is often easier to approximate a highly complex function rather than solve it perfectly. A Taylor series allows us to approximate any smooth non-linear function as an infinite sum of polynomial terms, based on its derivatives at a specific operating point.For a multi-dimensional state vector x evaluated near a known predicted state $\hat{x}$, the multivariate Taylor series expansion of a non-linear measurement function $h(x)$ is expressed as:

$$h(x) = h(\hat{x}) + J(x - \hat{x}) + \frac{1}{2!}(x - \hat{x})^T \mathcal{H}(x - \hat{x}) + \dots$$

Where: - J is the Jacobian matrix (the first-order derivative, representing the immediate linear slope). - $\mathcal{H}$ is the Hessian matrix (the second-order derivative, representing the curvature). - The $+ \dots$ represents an infinite series of increasingly higher-order derivatives.

Application in Target Tracking (The EKF Linearization)

In classic tracking, we absolutely require matrices to calculate and propagate covariance (uncertainty). The Taylor series is the mathematical bridge that attempts to force a non-linear world to behave linearly.

The Extended Kalman Filter (EKF) takes a harsh, pragmatic approach to the equation above: it truncates the Taylor series right after the first-order Jacobian term, completely discarding the Hessian and all higher-order derivatives. It assumes that $h(x) \approx h(\hat{x}) + J(x - \hat{x})$.

The Engineering Dilemma

This first-order Taylor series approximation only holds mathematically true if the actual target state x remains extremely close to our filter's prediction $\hat{x}$. If a fixed-wing UAV is flying straight and level, this approximation works well enough.

However, if an evasive drone suddenly executes a violent, high-G split-S maneuver, the true spatial state diverges rapidly from the filter's linear prediction. As the distance between x and $\hat{x}$ grows, the ignored higher-order terms (the truncation errors) become massive. The EKF's first-order Taylor series approximation mathematically shatters, causing rapid linearization error and total track divergence. This fundamental geometric flaw is precisely why modern aerospace interceptors must abandon the EKF and migrate toward the UKF and CKF architectures, which do not rely on Taylor series truncation.

C++ Implementation: The Cost of Linearization

In an object-oriented tracking architecture, understanding the difference between the true non-linear evaluation and the Taylor series approximation dictates how you architect your measurement updates.

```
#include <Eigen/Dense>
#include <cmath>
```

```cpp
using namespace Eigen;

class KinematicApproximation {
public:
    // 1. The True Non-Linear Measurement (e.g., Camera Projection)
    // In the CKF and UKF, we pass deterministic sigma/cubature points directly through this,
    // preserving the true curvature of the physics.
    static VectorXd trueNonLinearMeasurement(const VectorXd& x) {
        VectorXd z(2);
        // Example: highly non-linear perspective division and trigonometric rotation
        z(0) = std::sin(x(0)) / x(2);
        z(1) = std::cos(x(1)) / x(2);
        return z;
    }

    // 2. The First-Order Taylor Approximation (The EKF Approach)
    // h(x) ≈ h(xHat) + J * (x - xHat)
    static VectorXd linearApproximation(const VectorXd& xTrue, const VectorXd& xHat, const MatrixXd& Jac
        VectorXd zHat = trueNonLinearMeasurement(xHat);
        VectorXd deltaX = xTrue - xHat;

        // The EKF applies the Jacobian but completely ignores second-order
        // curvature (the Hessian), leading to catastrophic errors during rapid maneuvers.
        return zHat + (Jacobian * deltaX);
    }
};
```

1.3 Numerical Integration and Cubature Rules

The Mathematical Concept

Because analytical Jacobians are dangerous in high-agility tracking, modern filtering seeks to avoid them entirely. Instead of linearizing the function, we want to calculate the true expected value of the non-linear function mathematically. This requires evaluating a Gaussian-weighted integral over the entire state space:

$$I(f) = \int_{\mathbb{R}^n} f(\mathrm{x}) \exp(-\mathrm{x}^T \mathrm{x}) d\mathrm{x}$$

Solving this integral analytically in real-time is impossible. The **Spherical-Radial Cubature** Rule provides a numerical approximation. It decomposes the multi-dimensional integral into a spherical integral (direction) and a radial integral (distance). By carefully selecting exactly $2n$ deterministic points (where n is the state dimension), we can approximate the true integral with third-degree accuracy.

Application in Target Tracking (The CKF)

This integral is the beating heart of the Cubature Kalman Filter (CKF). Instead of differentiating the camera projection model, the CKF generates a set of 3D points representing the current uncertainty, pushes those specific points through the true non-linear camera projection model, and calculates the variance of the output.

1.4 The Origins of Cubature: Spherical-Radial Integration

In standard linear tracking, solving the Bayesian equations is clean and effortless. However, as we will explore in later chapters, when your target tracking pipeline incorporates non-linear camera geometry (such as perspective projections or trigonometric bearing angles), the recursive Bayesian integrals become mathematically unsolvable.

When an equation cannot be solved analytically via calculus, the CPU must approximate it numerically.

For decades, engineers relied on the **Extended Kalman Filter (EKF)**, which uses Jacobian matrices to forcibly flatten non-linear physics into linear tangents. For highly evasive targets, this forced linearization introduces massive truncation errors, causing the track to violently diverge.

A more accurate alternative is the Particle Filter, which uses Monte Carlo integration. It floods the uncertainty space with thousands of randomly generated "particles" (points) to blindly map the curve. While highly accurate, pushing 5,000 particles through complex camera geometry at 30 frames per second will immediately overwhelm and melt the CPU of an edge-deployed UAV interceptor.

We need a method that is as accurate as the Particle Filter, but as lightweight as the EKF. This is the birthplace of the **Cubature Kalman Filter (CKF)**.

The Geometric Solution

The CKF is built entirely upon a mathematical breakthrough known as the **Spherical-Radial Integration Rule**.

Instead of generating thousands of random points, the Spherical-Radial rule proves that we can perfectly approximate the continuous integral of a Gaussian distribution by strategically selecting a tiny, deterministic set of points.

Any standard multi-dimensional Gaussian integral can be converted from a Cartesian coordinate system (X, Y, Z) into a spherical coordinate system. Once converted, the impossible integral beautifully fractures into two separate, solvable integrals:

1. **The Spherical Integral**: An integral over the surface area of a unit sphere (accounting for direction).
2. **The Radial Integral**: A 1D line integral from the center of the sphere outward to infinity (accounting for distance).

The Magic of 2n Points

By solving these two integrals using polynomial moment-matching, the math reveals a profound shortcut. To completely capture the behavior of a multi-dimensional Gaussian distribution up to its third statistical moment, we only need to evaluate points that rest exactly on the intersections of the state-space axes and the uncertainty sphere

For an n-dimensional state space, there is one positive and one negative intersection per axis. Therefore, the CKF requires exactly $2n$ cubature points.

If you are tracking an 8-state Constant Velocity model (Position, Velocity, and Bounding Box dimensions), you only need to push $2(8) = 16$ points through your non-linear camera equations. This transforms an impossible calculus problem into 16 simple C++ function calls, providing near-optimal accuracy while remaining incredibly cache-friendly for edge hardware.

Deep Dive: The Spherical-Radial Derivation

> We just stated that converting to spherical coordinates miraculously collapses an infinite Gaussian integral down to exactly $2n$ points. But how does the calculus actually prove this? And how do we prove that the radius of the points must be exactly $\sqrt{n}$ to perfectly capture the system's variance? For the rigorous mathematical proof involving Gaussian-Laguerre quadrature and moment-matching, refer to **Appendix: Derivation of the Spherical-Radial Cubature Rule**.

Engineering Example: Programming the Cubature Points

To understand why the Cubature Kalman Filter (CKF) is so powerful for edge computing, we must look at how simple it is to program.

Let us assume we are tracking a drone using a 4-dimensional state vector $\mathrm{x} = [X, Y, V_x, V_y]^T$. The target is currently flying with some kinematic uncertainty represented by our 4×4 covariance matrix P.

The camera is about to take a picture. To predict what pixel bearing the camera will see, we must push our state through the highly non-linear measurement function $h(\mathrm{x}) = \arctan(Y/X)$.

Because the state space has $n = 4$ dimensions, the Spherical-Radial Cubature rule dictates that we only need exactly $2(4) = 8$ cubature points to perfectly approximate this non-linear integral.

The Three-Step Algorithm:

1. **Decompose:** We use the Cholesky decomposition to find the square root of the covariance matrix ($P = LL^T$). This gives us the mathematical "axes" of our uncertainty ellipsoid.
2. **Generate:** We take the standard Cartesian unit vectors (1 and -1 for each dimension), scale them by $\sqrt{n}$, multiply them by L to rotate them into our uncertainty space, and add them to our current state estimate x.
3. **Evaluate:** We pass those 8 specific points through our arctan function and average the result.

C++ Implementation: Cubature Point Generation

Here is the exact C++ implementation using the Eigen linear algebra library. Notice how the terrifying non-linear calculus problem we discussed earlier is resolved with a simple for loop.

```cpp
#include <Eigen/Dense>
#include <cmath>
#include <iostream>

using namespace Eigen;

class CubatureMath {
public:
    // Generates the 2n Cubature points for a given state and covariance
    // x: The current state vector (mean)
    // P: The current state covariance matrix (uncertainty)
    static MatrixXd generateCubaturePoints(const VectorXd& x, const MatrixXd& P) {
        int n = x.size();
        int num_points = 2 * n;

        // The spherical-radial scaling factor
        double scale = std::sqrt(n);
```

```
        // 1. Cholesky Decomposition (P = L * L^T)
        // This calculates the lower-triangular matrix L
        LLT<MatrixXd> lltOfP(P);
        MatrixXd L = lltOfP.matrixL();

        // Initialize a matrix to hold our points.
        // Each column will be one n-dimensional cubature point.
        MatrixXd points(n, num_points);

        // 2. Generate the 2n points
        for (int i = 0; i < n; ++i) {
            // Create a unit vector for the current axis
            VectorXd u = VectorXd::Zero(n);
            u(i) = 1.0;

            // Positive intersection on the uncertainty sphere
            points.col(i) = x + scale * L * u;

            // Negative intersection on the uncertainty sphere
            points.col(i + n) = x - scale * L * u;
        }

        return points;
    }

    // Pushes the generated points through a non-linear camera bearing function
    static double predictNonLinearMeasurement(const MatrixXd& points) {
        int num_points = points.cols();
        double z_predicted = 0.0;

        // 3. Evaluate the non-linear function for every point
        for (int i = 0; i < num_points; ++i) {
            // Extract the X and Y spatial coordinates from the point
            double X = points(0, i);
            double Y = points(1, i);

            // The uncorrupted non-linear geometry: h(x) = arctan(Y/X)
            double bearing = std::atan2(Y, X);

            // Sum the results
            z_predicted += bearing;
        }

        // The final predicted measurement is simply the weighted average
        // (divided by 2n) of all evaluated points.
        return z_predicted / num_points;
    }
};
```

Look closely at the predictNonLinearMeasurement function. There are no Jacobian derivative matrices. There is no forced flattening of the camera geometry. If you want to change your sensor from a 2D camera to a 3D radar, you do not need to recalculate complex calculus derivatives by hand—you simply change the std::atan2 line to your new radar equation, and the Cubature Kalman Filter instantly adapts.

Chapter 2

Linear Algebra for Systems

Tracking algorithms are fundamentally exercises in linear algebra. Every prediction, measurement, and update is a matrix operation. To deploy an IMM-CKF on a modern embedded architecture (such as an NVIDIA Jetson Orin NX), an engineer must not only understand the mathematical identities but also how matrix shapes and memory bandwidth impact the overall processing pipeline.

2.1 Vectors, matrices, and tensor basics.

The Mathematical Concept

A tracking system defines the physical world using column vectors. A state vector $\mathrm{x} \in \mathbb{R}^n$ holds the target's kinematics (e.g., X, Y, Z positions and velocities). Matrices act as operators that transform these vectors. The transition matrix $F \in \mathbb{R}^{n \times n}$ projects the state forward in time, while the measurement matrix $H \in \mathbb{R}^{m \times n}$ maps the n-dimensional state space into an m-dimensional observation space.

Application in Target Tracking

The dimensionality of these vectors and matrices dictates the computational load of the filter. An 11-state Constant Acceleration (CA) model requires maintaining an 11×11 covariance matrix containing 121 elements. An 8-state Constant Velocity (CV) model requires an 8×8 matrix with 64 elements.

While theoretical texts freely expand state dimensions, embedded systems processing multispectral perception pipelines face strict limits on Tensor Core availability. Structuring matrices cleanly ensures cache-friendly memory access during high-frequency telemetry fusion.

2.2 Eigenvalues, eigenvectors, and matrix decompositions (Cholesky decomposition, Singular Value Decomposition).

The Mathematical Concept

In Bayesian filtering, uncertainty is modeled using the State Covariance Matrix, P. A fundamental rule of probability is that variance cannot be negative. Therefore, a valid covariance matrix must be Positive Definite. Mathematically, a symmetric matrix A is positive definite if its eigenvalues are all strictly greater than zero, ensuring:

$$x^T Ax > 0$$

Before we decompose the covariance matrix, we must understand what it physically represents. If the state vector x is the filter's "best guess" of where the target is, the State Covariance Matrix P is the filter's "uncertainty bubble" around that guess.

For an n-dimensional state vector, P is a symmetric $n \times n$ matrix:

1 **The Diagonals (Variances)**: The elements along the main diagonal (e.g., $P_{0,0}, P_{1,1}$) represent the variance (σ^2) of each individual state. If $P_{0,0}$ is the variance of the X-position, a large number means the filter is highly uncertain about the target's exact X coordinate.

2 **The Off-Diagonals (Covariances)**: The elements off the main diagonal (e.g., $P_{0,1}$, $P_{1,0}$) represent the correlation between two states. If a target is turning, its X-velocity and Y-velocity are mathematically linked. A high covariance means that if the filter updates its belief about the X-velocity, it must proportionally update its belief about the Y-velocity.

Geometrically, in 3D space, the P matrix defines an ellipsoid (a 3D oval). The diagonals dictate how large the oval is, and the off-diagonals dictate how the oval is tilted or rotated in space.

To generate deterministic cubature points, the CKF requires calculating the "square root" of the P matrix. Because P is symmetric and positive definite, we use Cholesky Decomposition. This algorithm factors the matrix into the product of a lower triangular matrix L and its transpose L^T:

$$P = LL^T$$

> **Deep Dive: The Mathematical Proof**
>
> This chapter focuses on the application of Cholesky decomposition for edge hardware. However, understanding exactly why every symmetric, positive-definite matrix can be uniquely factored this way is a beautiful piece of linear algebra. If you are interested in the rigorous step-by-step mathematical derivation and the proof by induction, please refer to **Appendix B: The square root of a matrix and Cholesky decomposition**.

The Engineering Dilemma

In pure mathematical simulations, covariance matrices remain perfectly positive definite. However, when deployed on embedded hardware, 32-bit floating-point truncation, asynchronous telemetry latency, and sudden injections of measurement noise can cause the P matrix to become slightly asymmetric or develop small negative eigenvalues. If this happens, a standard Cholesky solver will throw a NaN error and crash the tracking thread.

C++ Implementation: Robust Cholesky Decomposition

A production-grade tracking orchestrator must wrap the decomposition in a protective layer, enforcing symmetry and regularizing the diagonal to guarantee execution.

```
#include <Eigen/Dense>
#include <iostream>
#include <stdexcept>

using namespace Eigen;
```

```
// Computes the lower triangular matrix L of a covariance matrix P.
MatrixXd computeRobustCholesky(MatrixXd P) {
    // 1. Enforce strict mathematical symmetry to mitigate floating-point drift
    P = 0.5 * (P + P.transpose());

    // 2. Attempt standard Cholesky Decomposition
    LLT<MatrixXd> lltOfP(P);

    if (lltOfP.info() == Eigen::NumericalIssue) {
        // Engineering Fallback: Matrix lost positive definiteness.
        // Inject a tiny process noise (epsilon) along the diagonal
        // to force eigenvalues back into positive territory.
        double epsilon = 1e-6;
        MatrixXd PReg = P + MatrixXd::Identity(P.rows(), P.cols()) * epsilon;

        LLT<MatrixXd> lltOfPReg(PReg);
        if (lltOfPReg.info() == Eigen::Success) {
            return lltOfPReg.matrixL();
        } else {
            throw std::runtime_error("Cholesky decomposition failed post-regularization.");
        }
    }

    return lltOfP.matrixL();
}
```

2.3 Matrix inversion lemma (Woodbury identity) and proofs.

The Mathematical Concept

Inverting a matrix is one of the most computationally expensive operations in linear algebra ($O(n^3)$ complexity). The Woodbury matrix identity dictates that the inverse of a rank-k correction of a matrix can be computed by doing a rank-k correction to the inverse of the original matrix:

$$(A + UCV)^{-1} = A^{-1} - A^{-1}U(C^{-1} + VA^{-1}U)^{-1}VA^{-1}$$

Application in Target Tracking

The Kalman Gain (K) dictates how much the filter trusts the incoming visual measurement versus its own internal prediction. It is calculated as:

$$K = P_{k|k-1}H^T(HP_{k|k-1}H^T + R)^{-1}$$

The term $(HP_{k|k-1}H^T + R)$ is the Innovation Covariance matrix (S). If we are fusing dense data arrays (where the measurement vector is massive), directly inverting S will cause pipeline stalls. The Woodbury identity allows us to mathematically rearrange the equation so we only invert matrices equal to the size of the state vector, rather than the measurement vector.

Engineering Example

Imagine an architecture utilizing an 8-state kinematic model (X, Y, Z, Vx, Vy, Vz, Width, Height) but processing an array of 50 simultaneous pixel threshold measurements from a segmented infrared target. Inverting a 50×50 matrix using standard LU decomposition on an embedded GPU/CPU complex is expensive. Using the Woodbury identity, the tracking engine can mathematically bypass the 50×50 inversion and instead invert an 8×8 matrix, maintaining a strict 60fps processing throughput.

Chapter 3

Differential Equations & Kinematics

While Chapter 2 provided the linear algebra tools to handle uncertainty, tracking algorithms ultimately need something physical to calculate. We must mathematically describe how a target moves through space.

The core challenge of tracking is a mismatch of domains: target physics (gravity, thrust, drag) operate continuously without interruption, but our tracking processors and visual sensors operate discretely, capturing snapshots of the world at 60 frames per second or processing telemetry at 10Hz. This chapter details how we model the continuous world and safely discretize it for embedded hardware.

3.1 Ordinary Differential Equations (ODEs) and continuous-time system modeling.

The Mathematical Concept

In kinematics, we describe a system's state using a vector $\mathrm{x}(t)$. To understand where the target will be in the future, we must model its rate of change over time. This is expressed as an Ordinary Differential Equation (ODE).

For a non-linear continuous-time dynamic system, the state derivative is defined as:

$$\dot{\mathrm{x}}(t) = f(\mathrm{x}(t), \mathrm{u}(t), \mathrm{w}(t))$$

Where: - $\dot{\mathrm{x}}(t)$ is the rate of change of the state vector (e.g., velocity is the derivative of position, acceleration is the derivative of velocity). - $f(\dots)$ is the non-linear function describing the physical laws governing the target. - $\mathrm{u}(t)$ is the known control input (e.g., autopilot commands, if tracking our own vehicle). - $\mathrm{w}(t)$ is the continuous-time process noise, capturing unknown disturbances like wind gusts or unmodeled target intent.

Application in Target Tracking

If we are tracking an uncooperative target—like an evasive drone—we do not have access to its control inputs $\mathrm{u}(t)$. We must rely entirely on our kinematic assumptions. The ODE simplifies to

$$\dot{\mathrm{x}}(t) = f(\mathrm{x}(t)) + \mathrm{w}(t)$$

For a simple 1D position (p) and velocity (v) model where velocity is assumed constant, the ODE system looks like this:

$$\dot{p} = v$$

$$\dot{v} = 0 + w$$

The Engineering Dilemma

Computers do not understand continuous time. An edge processor cannot evaluate an integral from $t = 0$ to $t = \infty$. It only understands arrays of memory evaluated at specific, discrete clock cycles. To predict a target's position at the time of the next camera shutter, we must numerically integrate the continuous ODE over a specific time step, Δt.

3.2 Discretization of continuous-time models (Runge-Kutta methods).

The Mathematical Concept

To convert continuous physics into discrete software updates ($\mathrm{x}_k \rightarrow \mathrm{x}_{k+1}$), we must approximate the integral of the ODE over the time interval Δt.

The simplest approach is the Euler method: $\mathrm{x}_{k+1} = \mathrm{x}_k + \Delta t \cdot f(\mathrm{x}_k)$. However, Euler integration assumes the rate of change remains completely flat over the entire Δt window. For high-speed targets executing turns, this assumption accumulates massive errors.Because the local error (error per step) is proportional to h^2, and the global error is proportional to h, therefore it relies entirely on a linear approximation, which means it drifts significantly if the true curve is highly non-linear.For stiff equations or highly dynamic systems, if the step size h is not infinitesimally small, the Euler method can become numerically unstable, causing the state estimates to diverge wildly.

The gold standard for aerospace tracking is the 4th-Order Runge-Kutta (RK4) method. RK4 evaluates the continuous derivative at four distinct points across the Δt window (the beginning, two at the midpoint, and one at the end) and computes a weighted average.

The RK4 update equations are:

$$\mathrm{k}_1 = f(\mathrm{x}_k)$$

$$\mathrm{k}_2 = f(\mathrm{x}_k + \frac{\Delta t}{2}\mathrm{k}_1)$$

$$\mathrm{k}_3 = f(\mathrm{x}_k + \frac{\Delta t}{2}\mathrm{k}_2)$$

$$\mathrm{k}_4 = f(\mathrm{x}_k + \Delta t\mathrm{k}_3)$$

$$\mathrm{x}_{k+1} = \mathrm{x}_k + \frac{\Delta t}{6}(\mathrm{k}_1 + 2\mathrm{k}_2 + 2\mathrm{k}_3 + \mathrm{k}_4)$$

Engineering Example When dealing with asynchronous sensor telemetry, Δt is rarely a static value. If a camera frame drops, the time since the last measurement might jump from 16ms to 32ms. Integrating over this variable Δt using Euler would cause the predicted bounding box to jitter wildly. Using RK4, the tracking engine safely interpolates the curved kinematic trajectory regardless of the fluctuating time step, ensuring the predicted bounding box lands exactly where the visual AI expects it.

C++ Implementation: The RK4 Integrator

In our object-oriented architecture, we isolate the integrator so it can be used by any kinematic model. Because we are operating on edge hardware where cycle counts matter, we pass the derivative function as a lambda or std::function to keep the memory footprint tight.

```
#include <Eigen/Dense>
#include <functional>

using namespace Eigen;

class KinematicIntegrator {
public:
    // f: The continuous-time ODE function describing the physics
    // xVector: The current state vector
    // dt: The exact time elapsed since the last measurement
    static VectorXd rungeKutta4(const std::function<VectorXd(const VectorXd&)>& f,
                                const VectorXd& xVector,
                                double dt) {

        VectorXd k1 = f(xVector);
        VectorXd k2 = f(xVector + 0.5 * dt * k1);
        VectorXd k3 = f(xVector + 0.5 * dt * k2);
        VectorXd k4 = f(xVector);

        return xVector + (dt / 6.0) * (k1 + 2.0*k2 + 2.0*k3 + k4);
    }
};
```

3.3 Modeling constant velocity (CV), constant acceleration (CA), and coordinated turn (CT) dynamics.

Once we have our integrator, we must define the physical functions ($f(\mathrm{x})$) we intend to track. Within an Interacting Multiple Model (IMM) architecture, we run several of these concurrently.

3.3.1 The Constant Velocity (CV) Model

The CV model assumes the target travels in a straight line at a constant speed. Any changes in velocity (accelerations) are modeled purely as random process noise.

For visual tracking applications, we utilize an 8-state vector: $[X, Y, Z, V_x, V_y, V_z, W, H]^T$, where W and H are the physical width and height of the target.

In continuous time, the derivative function $f(\mathrm{x})$ is simply the velocities:

- $\dot{X} = V_x$
- $\dot{Y} = V_y$
- $\dot{Z} = V_z$
- $\dot{V}_x = 0$
- $\dot{V}_y = 0$

- $\dot{V}_z = 0$
- $\dot{W} = 0$ (Target size is physically constant).
- $\dot{H} = 0$ (Target size is physically constant)

Because the CV model is perfectly linear, it does not strictly require RK4 integration. The discrete-time kinematic equations become:

$$X_{k+1} = X_k + V_{x,k}\Delta t$$
$$V_{x,k+1} = V_{x,k}$$
$$W_{k+1} = W_k$$
$$H_{k+1} = H_k$$

Mapping these equations across all variables yields our highly efficient, 8×8 discrete-time State Transition Matrix (F_{CV}):

$$F_{CV} = \begin{bmatrix} 1 & 0 & 0 & \Delta t & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & \Delta t & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & \Delta t & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

The Engineering Dilemma: The Lag vs. Noise Trade-off For a vision-and-telemetry-based tracker, the CV model is incredibly powerful because it naturally filters out the high-frequency "pixel jitter" generated by neural network bounding boxes. By refusing to explicitly calculate acceleration, the filter prevents the target's 3D state from vibrating wildly.

However, this mathematical stubbornness introduces a severe engineering dilemma: **Dynamic Lag**.

If an evasive target suddenly executes a sharp, high-G maneuver, the CV model mathematically assumes the target is still flying straight. It will confidently predict the target's next position along that straight line. By the time the visual measurement proves the target has turned, the prediction error (the Innovation) becomes massive. If the filter trusts its internal physics too much, the bounding box will lag behind the physical target, eventually separating entirely and causing a dropped track.

To solve this without reverting to the noisy Constant Acceleration model, engineers use a mathematical workaround: Process Noise Inflation. Within the IMM architecture, we deploy a secondary "High-Agility" CV model. This model uses the exact same F_{CV} matrix, but couples it with a massively inflated Process Noise Covariance matrix (Q). When a maneuver occurs, the IMM shifts probability to this high-agility model. The inflated Q matrix forces the Kalman Gain to mathematically distrust its own straight-line prediction and snap directly to the raw visual measurements until the turn is complete, completely eliminating dynamic lag without adding matrix dimensionality.

C++ Implementation

Because the CV model is the baseline state for the majority of the tracking lifecycle, its matrix generation must be incredibly lean. In C++, constructing this 8×8 identity matrix and injecting the Δt parameters is highly cache-friendly.

```cpp
#include <Eigen/Dense>

using namespace Eigen;

class KinematicModels {
public:
    // Generates the 8x8 State Transition Matrix for the Constant Velocity (CV) Model
    // dt: The time elapsed since the last update
    static MatrixXd getConstantVelocityMatrix(double dt) {
        // Initialize an 8x8 Identity matrix
        MatrixXd F = MatrixXd::Identity(8, 8);

        // Position updates linearly based on velocity (e.g., X = X + Vx * dt)
        F(0, 3) = dt;
        F(1, 4) = dt;
        F(2, 5) = dt;

        // Note: Velocities (indices 3, 4, 5) and Bounding Box dimensions
        // (indices 6, 7) inherently multiply by 1.0 from the Identity matrix,
        // representing their constant state.

        return F;
    }
};
```

3.3.2 The Constant Acceleration (CA) Model

The CA model introduces spatial acceleration states, expanding to an 11-state vector: $[X, Y, Z, V_x, V_y, V_z, A_x, A_y, A_z, W, H]^T$.

The ODE becomes:

- $\dot{X} = V_x$
- $\dot{Y} = V_y$
- $\dot{Z} = V_z$
- $\dot{V}_x = A_x$
- $\dot{V}_y = A_y$
- $\dot{V}_z = A_z$
- $\dot{A}_x = 0$
- $\dot{A}_y = 0$
- $\dot{A}_z = 0$
- $\dot{W} = 0$ (Target size is physically constant)
- $\dot{H} = 0$ (Target size is physically constant)

Therefore, the kinematic equations are:

- Position: $X_{k+1} = X_k + V_{x,k}\Delta t + \frac{1}{2}A_{x,k}\Delta t^2$
- Velocity: $V_{x,k+1} = V_{x,k} + A_{x,k}\Delta t$
- Acceleration: $A_{x,k+1} = A_{x,k}$

Mapping these equations across all spatial axes yields our 11×11 discrete-time State Transition Matrix

(F_{CA}):

$$F_{CA} = \begin{bmatrix} 1 & 0 & 0 & \Delta t & 0 & 0 & \frac{1}{2}\Delta t^2 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & \Delta t & 0 & 0 & \frac{1}{2}\Delta t^2 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & \Delta t & 0 & 0 & \frac{1}{2}\Delta t^2 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & \Delta t & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & \Delta t & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & \Delta t & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

The Engineering Reality of CA: While theoretically rigorous for radar tracking ballistic missiles, the 11-state CA model is explicitly dangerous for vision-based tracking. Because visual AI bounding boxes suffer from high-frequency pixel jitter, an explicit acceleration state will interpret this jitter as violent, frame-to-frame G-forces. The filter will amplify this noise, causing the track to vibrate out of control. This is why our IMM architecture relies on an 8-state CV model with dynamically inflated process noise to simulate maneuvers, rather than explicitly tracking acceleration.

** C++ Implementation for the CA Model:**

```
#include <Eigen/Dense>

using namespace Eigen;

class ConstantAccelerationModel {
public:
    // Generates the 11x11 State Transition Matrix for the CA Model
    static MatrixXd getTransitionMatrix(double dt) {
        MatrixXd F = MatrixXd::Identity(11, 11);

        double dt2Half = 0.5 * dt * dt;

        // Position updates based on velocity
        F(0, 3) = dt; F(1, 4) = dt; F(2, 5) = dt;

        // Position updates based on acceleration
        F(0, 6) = dt2Half; F(1, 7) = dt2Half; F(2, 8) = dt2Half;

        // Velocity updates based on acceleration
        F(3, 6) = dt; F(4, 7) = dt; F(5, 8) = dt;

        return F;
    }
};
```

3.3.3 The Coordinated Turn (CT) Model

The CT model is essential for tracking fixed-wing aircraft, which cannot simply slide sideways in the air; they must bank to turn. This introduces a Turn Rate parameter (ω). Assume that the tracking architecture relies on visual AI bounding boxes, you must also maintain the physical width and height of the target to utilize the scale prior. Adapting the CT model for your specific IMM-CKF pipeline results in a 9-state vector:

$$\mathrm{x} = [X, Y, Z, V_x, V_y, V_z, \omega, W, H]^T$$

- Z, V_z: Altitude and vertical velocity.
- W, H: Physical width and height of the target.

The standard CT model assumes the aircraft is banking to execute a turn in the horizontal plane ($X - Y$). Therefore, the turn rate ω strictly cross-couples and rotates the horizontal velocities (V_x and V_y). The vertical trajectory (Z-axis) is considered mathematically decoupled from the horizontal turn.

This model is highly non-linear. The ODE for a 3D horizontal turn is:

- $\dot{X} = V_x$
- $\dot{Y} = V_y$
- $\dot{Z} = V_z$
- $\dot{V}_x = -\omega V_y$
- $\dot{V}_y = \omega V_x$
- $\dot{V}_z = 0$
- $\dot{\omega} = 0$
- $\dot{W} = 0$
- $\dot{H} = 0$

Because the turn rate (ω) multiplies with the velocities (V_x and V_y), the Coordinated Turn model is non-linear. Therefore, a static F_{CT} matrix of pure constants does not exist. (In an EKF, you would derive the Jacobian matrix here, but because we are building a CKF, we do not need to linearize it).

Instead, we directly evaluate the closed-form discrete transition function $\mathrm{x}_{k+1} = f(\mathrm{x}_k, \Delta t)$. The exact geometric equations for a target banking in the horizontal plane over time Δt are:

$$X_{k+1} = X_k + \frac{V_{x,k}}{\omega_k}\sin(\omega_k \Delta t) - \frac{V_{y,k}}{\omega_k}(1 - \cos(\omega_k \Delta t))$$

$$Y_{k+1} = Y_k + \frac{V_{x,k}}{\omega_k}(1 - \cos(\omega_k \Delta t)) + \frac{V_{y,k}}{\omega_k}\sin(\omega_k \Delta t)$$

$$V_{x,k+1} = V_{x,k}\cos(\omega_k \Delta t) - V_{y,k}\sin(\omega_k \Delta t)$$

$$V_{y,k+1} = V_{x,k}\sin(\omega_k \Delta t) + V_{y,k}\cos(\omega_k \Delta t)$$

(Note: Z, V_z, ω, W, and H remain linearly constant or are integrated straight forwardly).

The Engineering Dilemma (Division by Zero): Notice that the equations for X and Y require dividing by ω. If the aircraft stops turning and flies straight, ω approaches 0, which will cause a divide-by-zero

hardware exception (NaN) and crash your filter. To solve this, your C++ implementation must check if ω is near zero, and if so, safely fall back to the standard Constant Velocity calculation for that frame.

C++ Implementation for the CT Model: In the CKF, you will pass your 18 cubature points (2×9 states) through this exact function during the Prediction Step.

```cpp
#include <Eigen/Dense>
#include <cmath>

using namespace Eigen;

class CoordinatedTurnModel {
public:
    // Evaluates the non-linear discrete state transition for the CT model.
    // xState: The 9-state vector [X, Y, Z, Vx, Vy, Vz, omega, W, H]^T
    static VectorXd evaluateTransition(const VectorXd& xState, double dt) {
        VectorXd xNext = xState; // Initialize with current state to carry over constants (Z, Vz, w, W, H

        double vx = xState(3);
        double vy = xState(4);
        double w = xState(6);

        // Z position updates linearly based on Vz
        xNext(2) = xState(2) + xState(5) * dt;

        // Engineering Check: Prevent divide-by-zero if the target is flying straight
        if (std::abs(w) < 1e-5) {
            // Degenerate to Constant Velocity model for X and Y
            xNext(0) = xState(0) + vx * dt;
            xNext(1) = xState(1) + vy * dt;

            // Velocities remain constant
            xNext(3) = vx;
            xNext(4) = vy;
        } else {
            // Apply the non-linear Coordinated Turn equations
            double sin_w_dt = std::sin(w * dt);
            double cos_w_dt = std::cos(w * dt);

            xNext(0) = xState(0) + (vx / w) * sin_w_dt - (vy / w) * (1.0 - cos_w_dt);
            xNext(1) = xState(1) + (vx / w) * (1.0 - cos_w_dt) + (vy / w) * sin_w_dt;

            xNext(3) = vx * cos_w_dt - vy * sin_w_dt;
            xNext(4) = vx * sin_w_dt + vy * cos_w_dt;
        }

        return xNext;
    }
};
```

Chapter 4

Probability Theory & Stochastic Processes

If the kinematics in Chapter 3 described how a target should move, probability theory describes how it actually moves in a chaotic universe. Sensors suffer from thermal noise, wind shears alter flight paths, and uncooperative targets execute unpredictable maneuvers. To build an optimal estimator, we cannot rely on deterministic certainties; we must compute with probabilities.

4.1 Random variables, probability density functions (PDFs), and Bayes' Theorem.

The Mathematical Concept

In tracking, the true kinematic state of a target (x) and the measurements from our sensors (z) are Continuous Random Variables. Because they can take on an infinite number of values (e.g., a drone can be at 100.0 meters, 100.001 meters, etc.), we cannot assign a probability to a specific, exact number.

Instead, we use a Probability Density Function (PDF), denoted as $p(\mathrm{x})$. The PDF defines the relative likelihood of the random variable falling within a particular spatial region.

In the context of aerospace tracking, the target's state x (where it actually is) and the measurement z (where the sensor thinks it is) are not single, absolute numbers. They are probability distributions—specifically, bell curves (Gaussians) representing uncertainty. Bayes' theorem is the mathematical engine that merges these conflicting curves into a single truth.

$$p(\mathrm{x} \mid \mathrm{z}) = \frac{p(\mathrm{z} \mid \mathrm{x})p(\mathrm{x})}{p(\mathrm{z})}$$

Where: - Prior $p(\mathrm{x})$:

The Engineering Meaning : This is the filter's Prediction.

What it is: Based on the kinematic ODEs we built in Chapter 3, this is where the filter believes the target is before the camera shutter opens.

The Shape: Because of process noise (wind, unknown target maneuvers), this prediction is not a single point; it is a Gaussian curve spread out over space. A wider curve means the filter is highly uncertain about its prediction.

- Likelihood $p(z \mid x)$:

  The Engineering Meaning: This is the Sensor Measurement.

  What it is: This answers the question: "If the target is actually at state x, how likely is it that my camera would output the exact bounding box z that I just received?"

  The Shape: Visual AI sensors have inherent pixel jitter and calibration errors. Therefore, the measurement itself is a Gaussian curve centered on the pixel detection. A cheap, noisy sensor produces a very wide, flat curve; a military-grade laser rangefinder produces a very narrow, sharp curve.

- The Posterior: $p(x \mid z)$

  The Engineering Meaning: This is the Updated Estimate (The final output of the filter for this frame).

  What it is: This is our new, refined belief of the target's true state, given both our internal physics prediction and the new sensor data.

  The Math: The numerator of Bayes' theorem simply multiplies the Prior curve and the Likelihood curve together.

- Evidence $p(z)$

  The Engineering Meaning: The Normalizing Constant.

  What it is:When you multiply two Gaussian curves together, the resulting curve shrinks. In probability theory, the total area under a PDF curve must always equal exactly 1.0 (representing 100% total probability). The Evidence term $p(z)$ calculates the total, absolute probability of receiving this specific measurement across the entire universe of possible states. By dividing by this term, we scale the new multiplied curve back up so its area equals 1.0.

Engineering Example: The 1D Drone Intercept

Imagine a 1D tracking scenario. We are tracking a drone's horizontal distance (X) along a runway.

1. The Prediction (Prior)

Our Constant Velocity (CV) model pushes the state forward in time. Based on the drone's previous speed, the filter predicts the drone is currently at 100 meters. Because a heavy crosswind was detected, our process noise is high. The filter's belief is a wide Gaussian curve centered at 100m.

2. The Measurement (Likelihood)

The camera processes the latest video frame and detects the drone. The bounding box centroid maps to a distance of 110 meters. The camera is known to have ± 5 meters of thermal noise. This creates a second, narrower Gaussian curve centered at 110m.

3. The Multiplication (Posterior)

We now have two conflicting pieces of data: the physics model says 100m, the camera says 110m. Bayes' theorem handles this conflict seamlessly by multiplying the two curves.

Because the camera's curve (Likelihood) is narrower (meaning it has less variance/uncertainty) than the physics prediction curve (Prior), the multiplication heavily favors the camera. The resulting Posterior curve will not peak perfectly in the middle at 105m. Instead, the mathematical peak will shift closer to the more "certain" data source—perhaps peaking at 108 meters.

Furthermore, because we have successfully fused two independent sources of information, the new Posterior curve will be narrower and taller than both the Prior and the Likelihood. Mathematically, Bayes' theorem guarantees that fusing data always reduces overall system uncertainty.

> **Deep Dive: The Birth of the Kalman Gain**
>
> We just stated that multiplying two Gaussian bell curves miraculously creates a third, perfectly shaped Gaussian curve. But why? And how does that relate to writing C++ tracking code?If you multiply the algebraic equations of these two curves, the resulting formula for the new Mean and Variance is exactly the 1D Kalman Filter Update Equation. For the step-by-step algebraic proof showing exactly how the Kalman Gain (K) is derived from this multiplication, refer to **Appendix D: The Gaussian Multiplication Proof and the Origins of the Kalman Gain**.

The Engineering Dilemma (The Integration Problem)

The Evidence term $p(\mathrm{z})$ requires integrating the likelihood across the entire infinite state space: $p(\mathrm{z}) = \int p(\mathrm{z} \mid \mathrm{x}) p(\mathrm{x}) d\mathrm{x}$. For non-linear camera projections, this integral has no closed-form analytical solution. It is impossible to calculate on a CPU. This mathematical roadblock is exactly why the Cubature Kalman Filter uses numerical cubature points to approximate this integral, rather than attempting to solve Bayes' Theorem analytically.

4.2 Expectation algebra

To write tracking algorithms, we don't manipulate entire PDF curves; we manipulate their statistical properties using **Expectation Algebra**.

The Expected Value ($E[\mathrm{x}]$) is the mathematical center of mass of the PDF. In tracking, this is our state estimate, denoted as $\hat{\mathrm{x}}$ or μ.

The Covariance (P) measures how much the random variable is expected to spread out from its mean:

$$P = E[(\mathrm{x} - \hat{\mathrm{x}})(\mathrm{x} - \hat{\mathrm{x}})^T]$$

Expectation is a linear operator. This means that if we apply a linear matrix transformation to our state (like multiplying our state by the State Transition Matrix F), the expectation flows through cleanly.

> **Deep Drive: Expectation Algebra and Covariance Propagation**
>
> The rigorous mathematical proof showing exactly how covariance propagates through linear matrices—resulting in the famous Kalman Filter equation $P_{k|k-1} = FP_{k-1}F^T + Q$—is fundamental to estimation theory. To keep this chapter focused on implementation, the complete derivation is provided in **Appendix C: Expectation Algebra and Covariance Propagation**.

4.3 Gaussian distributions and their properties (multivariate normal distribution).

The Mathematical Concept

While Bayes' theorem theoretically works for any PDF shape, maintaining arbitrary, lopsided probability curves in software requires massive particle filters that choke embedded CPUs.

The Kalman Filter family makes a bold mathematical assumption: all uncertainties are Gaussian (Normal) distributions. Governed by the Central Limit Theorem, this is a safe assumption because the sum of many independent random noises (thermal jitter, wind, vibration) naturally tends toward a Gaussian shape.

A Multivariate Gaussian is entirely defined by just two parameters: its mean vector μ (the state estimate) and its covariance matrix P (the uncertainty). The PDF is evaluated as:

$$\mathcal{N}(\mathrm{x}; \mu, P) = \frac{1}{\sqrt{(2\pi)^n |P|}} \exp\left(-\frac{1}{2}(\mathrm{x}-\mu)^T P^{-1}(\mathrm{x}-\mu)\right)$$

Where:

- n is the dimension of the state vector.
- $|P|$ is the determinant of the covariance matrix.
- P^{-1} is the inverse of the covariance matrix.

The Engineering Connection to Chapter 2 Look closely at the Gaussian equation above. To evaluate a target's likelihood, the CPU must calculate the determinant $|P|$ and the inverse P^{-1}. If floating-point drift causes your covariance matrix to lose positive definiteness (as discussed in Chapter 2.2), the determinant $|P|$ becomes negative. The term $\sqrt{|P|}$ then attempts to take the square root of a negative number, resulting in an immediate software crash. This proves why rigorous Cholesky regularization is non-negotiable in production tracking.

4.4 Markov processes and transition probabilities.

The Mathematical Concept

A **Markov Process** (specifically a discrete-time Markov Chain) is a mathematical framework for modeling systems that transition from one state to another over time.

The defining rule of a **Markov Process** is the **Markov Property** (or "memoryless" property): The probability of transitioning to any future state depends entirely on the current state, and completely ignores the historical path taken to get there.

If a system has a set of possible states $S = \{s_1, s_2, \dots, s_n\}$, the Markov Property is written mathematically as:

$$P(X_{k+1} = s_j \mid X_k = s_i, X_{k-1} = s_h, \dots) = P(X_{k+1} = s_j \mid X_k = s_i)$$

These transition probabilities are organized into a square Transition Probability Matrix, denoted as Π (or P).

$$\Pi = [p_{ij}] \quad \text{where} \quad p_{ij} = P(M_j(k) \mid M_i(k-1))$$

For a 2-state system, it looks like this:

$$\Pi = \begin{bmatrix} p_{11} & p_{12} \\ p_{21} & p_{22} \end{bmatrix}$$

- p_{11} is the probability of staying in State 1.
- p_{12} is the probability of switching from State 1 to State 2.

- Because the system must be in one of the states at the next time step, every row in a Markov matrix must sum to exactly 1.0. ($p_{11} + p_{12} = 1.0$).

In the Interacting Multiple Model (IMM) architecture, we assume the target's decision to maneuver is a Markov Process. The target is in a specific kinematic "mode" M (e.g., M_1 = Constant Velocity, M_2 = High-Agility Turn).

Application in Target Tracking

In the IMM-CKF architecture, we are running multiple kinematic models simultaneously (e.g., a Constant Velocity (CV) model and a Coordinated Turn (CT) model). The IMM uses a Markov Process to govern how the filter switches between these physical models.

To see exactly how the Markov Transition Matrix (Π) controls the tracking filter, let's walk through a single frame of video (one time step, k) using concrete numbers.

0. The Initial State (Frame $k-1$):

In the previous frame, the drone was flying straight. Our filter was highly confident it was in CV mode

- Previous Probabilities: $\mu_1 = 0.80$ (80% CV), $\mu_2 = 0.20$ (20% CT).
- Previous State Estimates (Simplified to 1D Velocity for this example): $\hat{x}_1 = 100$ m/s, $\hat{x}_2 = 90$ m/s.

1. Defining the "Inertia" (Π Matrix)

As an engineer, you define the Transition Matrix to give the target physical inertia. For example: - State 1 = Constant Velocity (CV) - State 2 = Coordinated Turn (CT)

You might define your Π matrix as:

$$\Pi = \begin{bmatrix} 0.95 & 0.05 \\ 0.10 & 0.90 \end{bmatrix}$$

- Row 1 means: "If the drone is flying straight, there is a 95% chance it will keep flying straight, and a 5% chance it will initiate a turn."
- Row 2 means: "If the drone is actively turning, there is a 90% chance it will keep turning, and a 10% chance it will flatten out into straight flight."

2. The Interaction Step (Mixing)

Before the visual measurements even arrive, the IMM uses the Markov matrix to mix the tracking data. It calculates the Mixing Probabilities ($\mu_{i|j}$), which ask: "Given that we end up in Mode j, what is the probability we came from Mode i?"

$$\mu_{i|j} = \frac{1}{\bar{c}_j} p_{ij} \mu_i$$

Where μ_i is the current probability of the model from the previous frame, p_{ij} is the Markov transition scalar, and $\bar{c}_j$ is a normalizer.

The filter then calculates a "mixed" initial state vector and covariance matrix for every model. This prevents the CT model from losing track of the target while it sits idle during long periods of straight flight.

A. Calculate the Predicted Mode Probabilities ($\bar{c}_j$):

What is the prior probability of being in each mode today, based purely on yesterday's modes and our Markov matrix?

$$\bar{c}_1 = (p_{11} \times \mu_1) + (p_{21} \times \mu_2) = (0.95 \times 0.80) + (0.10 \times 0.20) = 0.76 + 0.02 = 0.78$$
$$\bar{c}_2 = (p_{12} \times \mu_1) + (p_{22} \times \mu_2) = (0.05 \times 0.80) + (0.90 \times 0.20) = 0.04 + 0.18 = 0.22$$

(Note: $0.78 + 0.22 = 1.0$. The math holds).

B. Calculate the Mixing Weights ($\mu_{i|j}$):

If we end up in Model 1 today, how much of yesterday's Model 1 and Model 2 should we mix into it?

$$\mu_{1|1} = (0.95 \times 0.80)/0.78 = 0.974$$
$$\mu_{2|1} = (0.10 \times 0.20)/0.78 = 0.026$$

C. Calculate the Mixed Initial States:

We blend the kinematic states together. The new starting velocity for Model 1 is heavily weighted by its own history, but it absorbs a tiny bit (2.6%) of Model 2's history.

$$\hat{x}_{01} = (0.974 \times 100) + (0.026 \times 90) = 99.74 \text{ m/s}$$

(The CKF Covariance matrices are mixed using similar weighted math, pooling their uncertainties together).

3. The Filtering Step (CKF)

The individual Cubature Kalman Filters now run independently. The CV model predicts a straight line; the CT model predicts a curve. Both generate cubature points and compare them to the incoming visual bounding box. Each filter calculates its own Likelihood (Λ_j)—essentially, how well its prediction matched reality.

Now, the two separate Cubature Kalman Filters run their standard prediction and update steps using the newly mixed initial states.

Let's assume the drone just executed a violent evasive turn.

- The CV Filter predicts the drone kept flying straight. It compares its prediction to the new camera bounding box. The error is massive. It outputs a tiny Measurement Likelihood: $\Lambda_1 = 0.01$.
- The CT Filter predicts a curve. It compares its prediction to the camera box. It's a very close match! It outputs a high Measurement Likelihood: $\Lambda_2 = 0.40$.

Let's say the filters output their newly calculated velocities as $\hat{x}_{new,1} = 95$ m/s and $\hat{x}_{new,2} = 85$ m/s.

4. The Model Probability Update

This is where Bayes' Theorem and the Markov process meet. The overall probability of each mode (μ_j) is updated by multiplying its Markov-predicted likelihood ($\bar{c}_j$) by its visual measurement likelihood (Λ_j):

$$\mu_j = \frac{1}{c}\Lambda_j \bar{c}_j$$

If the target executes a violent maneuver, the CV model's visual likelihood drops to zero. The CT model's likelihood spikes. The IMM instantly shifts the dominant probability μ to the CT model, and the final combined output trajectory curves perfectly with the target.

This is where Bayes' Theorem executes the mode switch. We update the overall probability of each model by multiplying the Markov Prediction $(\bar{c})$ by the Visual Likelihood (Λ).

A. Multiply Prior × Likelihood:

$$\mu_1^* = \Lambda_1 \times \bar{c}_1 = 0.01 \times 0.78 = 0.0078$$
$$\mu_2^* = \Lambda_2 \times \bar{c}_2 = 0.40 \times 0.22 = 0.0880$$

B. Normalize (Calculate Evidence c):

$$c = 0.0078 + 0.0880 = 0.0958$$

C. The Final Mode Probabilities:

$$\mu_1 = 0.0078/0.0958 = 0.081(8.1\%CV)$$
$$\mu_2 = 0.0880/0.0958 = 0.919(91.9\%CT)$$

The Engineering Result: In a single frame, the system mathematically realized the target was maneuvering. The Markov transition matrix allowed the probability to seamlessly violently flip from 80% CV to 91.9% CT without resetting the tracking pipeline

5. Estimate Combination

The final output of the IMM—the data actually sent to the weapons system or gimbal—is a weighted sum of the two filters, governed by our new mode probabilities.

$$\hat{x}_{final} = (\mu_1 \times \hat{x}_{new,1}) + (\mu_2 \times \hat{x}_{new,2})$$
$$\hat{x}_{final} = (0.081 \times 95) + (0.919 \times 85)$$
$$\hat{x}_{final} = 7.695 + 78.115 = 85.81 \text{ m/s}$$

Because the probability flipped to 91.9% CT, the final system output almost entirely trusts the Coordinated Turn model's estimate, perfectly adapting to the evasive maneuver.

C++ Implementation: IMM Markov Mixing

```
#include <Eigen/Dense>
#include <iostream>

using namespace Eigen;

class MarkovProcess {
public:
    // Calculates the predicted mode probabilities based on the Markov Transition Matrix
```

```
    // current_probs: The likelihood of each mode from the previous frame
    // transition_matrix: The Pi matrix defining the maneuver inertia
    static VectorXd predictModeProbabilities(const VectorXd& current_probs,
                                             const MatrixXd& transition_matrix) {

        // Ensure inputs are valid
        if (current_probs.rows() != transition_matrix.rows()) {
            throw std::invalid_argument("Dimension mismatch in Markov mixing.");
        }

        // The Markov prediction is a simple matrix-vector multiplication
        // mu_predicted = Pi^T * mu_current
        VectorXd predicted_probs = transition_matrix.transpose() * current_probs;

        // Normalize to ensure total probability strictly equals 1.0
        double sum = predicted_probs.sum();
        if (sum > 0) {
            predicted_probs /= sum;
        }

        return predicted_probs;
    }
};
```

Part II: Dynamic Systems and The Optimal Estimator

Chapter 5

Control System Fundamentals

In Part I, we established the mathematical physics to model a target's trajectory (Chapter 3) and the probability theory to handle uncertainty (Chapter 4). Now, we must fuse these concepts into a unified framework that a computer can systematically solve.

This framework is called **State-Space** Representation. It is the absolute core of modern control theory and the foundation upon which all optimal estimators—including the Kalman Filter family—are built.

5.1 State-space representation of dynamic systems.

The Mathematical Concept

In classical control theory (like PID controllers), systems are often modeled using transfer functions in the frequency domain. However, modern aerospace tracking requires tracking multiple variables simultaneously (position, velocity, target size) in the time domain.

State-space representation models a physical system as a set of first-order differential equations written in matrix-vector form. For a linear, continuous-time system, the state-space equations are:

1. The System Equation (How the target moves):

$$\dot{\mathbf{x}}(t) = A\mathbf{x}(t) + B\mathbf{u}(t) + \mathbf{w}(t)$$

2. The Measurement Equation (What the sensor sees):

$$\mathbf{z}(t) = H\mathbf{x}(t) + D\mathbf{u}(t) + \mathbf{v}(t)$$

Where: - $\mathbf{x}$ is the State Vector (kinematics). - $\mathbf{u}$ is the Control Vector (steerage, throttle). - $\mathbf{w}$ is Process Noise and $\mathbf{v}$ is Measurement Noise. - A, B, H, and D are the system matrices mapping these relationships.

To run this on a digital CPU, we must discretize these continuous equations (as discussed via Runge-Kutta in Chapter 3). The discrete-time linear state-space model used by the Kalman Filter is:

$$\mathbf{x}_{k+1} = F\mathbf{x}_k + B\mathbf{u}_k + \mathbf{w}_k$$

$$\mathrm{z}_k = H\mathrm{x}_k + \mathrm{v}_k$$

(Note: F is the discrete State Transition Matrix we derived earlier, and H is the Measurement Matrix).

The Engineering Dilemma: Uncooperative Targets

If you are writing the navigation filter for your own UAV, the state-space model is fully realized. You know exactly what control inputs (u_k) the autopilot is commanding.

However, in target tracking, we are observing an uncooperative object. We do not have access to the enemy pilot's stick inputs or the drone's motor PWM signals. Therefore, the control term $B\mathrm{u}_k$ is completely unknown to us.

To solve this, tracking engineers drop the $B\mathrm{u}_k$ term entirely. We are forced to assume that any intentional maneuver made by the target is just random statistical Process Noise (w_k). This is exactly why tracking evasive targets is mathematically harder than navigating your own vehicle, and why we require the Interacting Multiple Model (IMM) to dynamically swap out F matrices when a maneuver occurs.

5.2 Derivation of the Discrete State Extrapolation Equation

To understand the cap beteen the differential equations and the discrete time linear state space model, we must mathematically derive how a continuous physical system is translated into discrete code. This requires a two-part proof: first, discretizing the continuous differential equations of motion, and second, applying the expectation operator to form the optimal state prediction.

Continuous-to-Discrete System Discretization

We begin with the continuous-time linear differential state equation describing the true physics of the system:

$$\dot{\mathrm{x}}(t) = A\mathrm{x}(t) + B\mathrm{u}(t) + \mathrm{w}_c(t)$$

Where A is the continuous system matrix, B is the continuous control input matrix, $\mathrm{u}(t)$ is the control vector, and $\mathrm{w}_c(t)$ is continuous zero-mean white noise.

Step 1: Isolate the State Variables

We begin with the standard continuous-time state equation describing our dynamic system:

$$\dot{\mathrm{x}}(t) = A\mathrm{x}(t) + B\mathrm{u}(t) + \mathrm{w}_c(t)$$

To solve for the state vector $\mathrm{x}(t)$, we move all terms containing $\mathrm{x}(t)$ to the left-hand side of the equation:

$$\dot{\mathrm{x}}(t) - A\mathrm{x}(t) = B\mathrm{u}(t) + \mathrm{w}_c(t)$$

Step 2: Introduce the Matrix Integrating Factor

In scalar calculus, an equation of the form $\dot{x}(t) - ax(t) = f(t)$ is solved by multiplying the entire equation by an integrating factor e^{-at}. In linear systems theory, we apply the exact same logic using matrices. We define our matrix integrating factor as the matrix exponential:

$$e^{-At}$$

The matrix exponential possesses a crucial property: it commutes with its own generating matrix, meaning $Ae^{-At} = e^{-At}A$. Consequently, its time derivative follows standard scalar behavior:

$$\frac{d}{dt}\left(e^{-At}\right) = -Ae^{-At} = -e^{-At}A$$

We premultiply every term in our isolated state equation by this integrating factor:

$$e^{-At}\dot{\mathrm{x}}(t) - e^{-At}A\mathrm{x}(t) = e^{-At}B\mathrm{u}(t) + e^{-At}\mathrm{w}_c(t)$$

Step 3: Apply the Matrix Product Rule in Reverse

Look closely at the left-hand side of our equation: $e^{-At}\dot{\mathrm{x}}(t) - e^{-At}A\mathrm{x}(t)$. This expression perfectly matches the expanded form of the matrix product rule for differentiation.

Recall that the derivative of a product of a matrix function and a vector function is:

$$\frac{d}{dt}\left(e^{-At}\mathrm{x}(t)\right) = e^{-At}\dot{\mathrm{x}}(t) + \left(\frac{d}{dt}e^{-At}\right)\mathrm{x}(t)$$

Substituting our known derivative $\frac{d}{dt}(e^{-At}) = -e^{-At}A$ into the product rule yields:

$$\frac{d}{dt}\left(e^{-At}\mathrm{x}(t)\right) = e^{-At}\dot{\mathrm{x}}(t) - e^{-At}A\mathrm{x}(t)$$

Because this exactly matches the left-hand side of our state equation, we can collapse those two independent terms into a single total derivative:

$$\frac{d}{dt}\left(e^{-At}\mathrm{x}(t)\right) = e^{-At}B\mathrm{u}(t) + e^{-At}\mathrm{w}_c(t)$$

Step 4: Integrate Over the Definite Interval

We want to solve this system across a discrete time step from an initial time t_n to a future time t_{n+1}. To do this, we integrate both sides of the equation with respect to a dummy time variable τ over the definite bounds $[t_n, t_{n+1}]$:

$$\int_{t_n}^{t_{n+1}} \frac{d}{d\tau}\left(e^{-A\tau}\mathrm{x}(\tau)\right)d\tau = \int_{t_n}^{t_{n+1}} e^{-A\tau}B\mathrm{u}(\tau)d\tau + \int_{t_n}^{t_{n+1}} e^{-A\tau}\mathrm{w}_c(\tau)d\tau$$

By the Fundamental Theorem of Calculus, the integral of a total derivative on the left-hand side simply evaluates to the function itself evaluated at the upper and lower limits:

$$\left[e^{-A\tau}\mathrm{x}(\tau)\right]_{t_n}^{t_{n+1}} = \int_{t_n}^{t_{n+1}} e^{-A\tau}B\mathrm{u}(\tau)d\tau + \int_{t_n}^{t_{n+1}} e^{-A\tau}\mathrm{w}_c(\tau)d\tau$$

Evaluating the limits yields:

$$e^{-At_{n+1}}\mathrm{x}(t_{n+1}) - e^{-At_n}\mathrm{x}(t_n) = \int_{t_n}^{t_{n+1}} e^{-A\tau}B\mathrm{u}(\tau)d\tau + \int_{t_n}^{t_{n+1}} e^{-A\tau}\mathrm{w}_c(\tau)d\tau$$

Step 5: Isolate the Future State Vector

To solve explicitly for the future state vector $\mathrm{x}(t_{n+1})$, we first move the initial state term to the right-hand side:

$$e^{-At_{n+1}}\mathrm{x}(t_{n+1}) = e^{-At_n}\mathrm{x}(t_n) + \int_{t_n}^{t_{n+1}} e^{-A\tau}B\mathrm{u}(\tau)d\tau + \int_{t_n}^{t_{n+1}} e^{-A\tau}\mathrm{w}_c(\tau)d\tau$$

Finally, we isolate $\mathrm{x}(t_{n+1})$ by premultiplying the entire equation by the inverse of the leading matrix exponential, which is $e^{At_{n+1}}$:

$$\mathrm{x}(t_{n+1}) = e^{At_{n+1}}e^{-At_n}\mathrm{x}(t_n) + e^{At_{n+1}}\int_{t_n}^{t_{n+1}} e^{-A\tau}B\mathrm{u}(\tau)d\tau + e^{At_{n+1}}\int_{t_n}^{t_{n+1}} e^{-A\tau}\mathrm{w}_c(\tau)d\tau$$

Step 6: Combine the Matrix Exponentials

Because matrix exponentials of the same matrix commute ($e^X e^Y = e^{X+Y}$ if X and Y commute), we can algebraically combine the terms.

For the first term:

$$e^{At_{n+1}}e^{-At_n} = e^{A(t_{n+1}-t_n)}$$

For the integral terms, because the matrix $e^{At_{n+1}}$ is independent of the integration variable τ, we can pass it directly inside the integral signs:

$$e^{At_{n+1}}\int_{t_n}^{t_{n+1}} e^{-A\tau}(\dots)d\tau = \int_{t_n}^{t_{n+1}} e^{At_{n+1}}e^{-A\tau}(\dots)d\tau = \int_{t_n}^{t_{n+1}} e^{A(t_{n+1}-\tau)}(\dots)d\tau$$

Substituting these simplified exponents back into our equation completes the rigorous derivation:

$$\mathrm{x}(t_{n+1}) = e^{A(t_{n+1}-t_n)}\mathrm{x}(t_n) + \int_{t_n}^{t_{n+1}} e^{A(t_{n+1}-\tau)}B\mathrm{u}(\tau)d\tau + \int_{t_n}^{t_{n+1}} e^{A(t_{n+1}-\tau)}\mathrm{w}_c(\tau)d\tau$$

Applying a Zero-Order Hold (ZOH) assumption states that the control input vector remains constant over the interval Δt ($\mathrm{u}(\tau) = \mathrm{u}_n$). Performing a change of variables letting $\lambda = t_{n+1} - \tau$ (implying $d\tau = -d\lambda$) shifts the limits of integration and simplifies the expression to:

$$\mathrm{x}_{n+1} = e^{A\Delta t}\mathrm{x}_n + \left(\int_0^{\Delta t} e^{A\lambda}d\lambda B\right)\mathrm{u}_n + \mathrm{w}_n$$

This yields the discrete-time **True State Equation**:

$$\mathrm{x}_{n+1} = F\mathrm{x}_n + G\mathrm{u}_n + \mathrm{w}_n$$

Where the discrete matrices are analytically defined via the matrix exponential: * **State Transition Matrix:** $F = e^{A\Delta t}$ * **Discrete Input Matrix:** $G = \int_0^{\Delta t} e^{A\lambda}d\lambda B$ * **Discrete Process Noise Vector:** $\mathrm{w}_n = \int_{t_n}^{t_{n+1}} e^{A(t_{n+1}-\tau)}\mathrm{w}_c(\tau)d\tau$

The Conditional Expectation Proof

The equation above governs the true state vector x_{n+1}. To compute the optimal state prediction (the extrapolated estimate $\hat{x}_{n+1|n}$), we must evaluate the conditional expectation given all historical measurements up to the current step n (denoted as Z_n):

$$\hat{x}_{n+1|n} = E[x_{n+1} \mid Z_n]$$

Substituting the true discrete state equation inside the expectation operator yields:

$$\hat{x}_{n+1|n} = E[Fx_n + Gu_n + w_n \mid Z_n]$$

Utilizing the mathematical linearity of the expectation operator, we distribute it across the individual terms:

$$\hat{x}_{n+1|n} = FE[x_n \mid Z_n] + GE[u_n \mid Z_n] + E[w_n \mid Z_n]$$

We evaluate each term based on fundamental probability axioms: 1. The conditional expectation of the true state given historical data is our current updated state estimate: $E[x_n \mid Z_n] = \hat{x}_{n|n}$. 2. The control vector u_n is a known deterministic input, meaning: $E[u_n \mid Z_n] = u_n$. 3. The discrete process noise w_n is a zero-mean white process completely uncorrelated with past measurement history Z_n, meaning its expected value vanishes: $E[w_n \mid Z_n] = 0$.

Combining these evaluations yields the final, optimal discrete state extrapolation equation used in the Kalman Filter prediction loop:

$$\hat{x}_{n+1|n} = F\hat{x}_{n|n} + Gu_n$$

5.3 Observability and controllability.

Before deploying an estimation algorithm, an engineer must answer two fundamental questions: Can I steer this system? And can I actually see what I am trying to track?

Controllability

Controllability asks: Given the system matrices F and B, can we find a control sequence u that drives the system from any initial state to any desired final state in a finite amount of time? Because we drop the B matrix in uncooperative tracking, controllability is primarily a concern for the interceptor's guidance algorithms, not the state estimator itself.

Observability

Observability is the single most critical theorem for a tracking engineer. It asks: Given a sequence of sensor measurements $z_0, z_1, \dots, z_k$, can we uniquely deduce the internal state x_0 of the system?If a system is unobservable, the Kalman Filter will mathematically fail to converge. The covariance matrix (P) for the unobservable states will grow toward infinity, eventually crashing the filter.

The Observability Theorem:

A discrete linear time-invariant system is completely observable if and only if the Observability Matrix (O) has full column rank (rank n, where n is the dimension of the state vector):

$$O = \begin{bmatrix} H \\ HF \\ HF^2 \\ \vdots \\ HF^{n-1} \end{bmatrix}$$

Deep Dive: The Observability Proof

If you want to understand exactly why multiplying the measurement matrix by the transition matrix $n-1$ times mathematically guarantees that a target can be tracked, the rigorous linear algebra proof demonstrating how full column rank allows for unique state recovery is provided in **Appendix E: The Proof of Linear Observability**.

The Engineering Dilemma: The Monocular Camera

Observability is the exact mathematical reason why estimating 3D kinematics from a single 2D camera is so difficult.

A monocular visual sensor only provides bearing angles (or 2D pixel coordinates). It provides absolutely zero range (depth) information. If an aircraft flies directly toward the camera, its bearing does not change. Because multiple physical states (a small drone close up, or a massive airliner far away) can produce the exact same sequence of pixel measurements, the 3D position is mathematically unobservable.

To make the system observable, tracking engineers must inject prior knowledge. In our architecture, we include the target's physical Width (W) and Height (H) in our state vector. By utilizing a neural network bounding box, the change in pixel dimensions over time acts as a geometric proxy for range, fulfilling the rank requirement of the Observability Matrix and allowing the filter to converge on a 3D solution.

5.4 Process noise vs. measurement noise.

The entire soul of the Kalman Filter lives in the mathematical tension between two covariance matrices: the Process Noise (Q) and the Measurement Noise (R). Tuning these matrices is the most difficult art in state estimation.

Measurement Noise (R)

The R matrix represents v_k. It defines how much we distrust our sensors.

- If you are fusing data from a heavily degraded IR microbolometer experiencing thermal blooming, the bounding boxes will flutter. You must inflate R.
- **Engineering Reality**: R is not just physical sensor noise; it also absorbs software pipeline errors. Consider a low-level memory bug—such as an incorrect bytes-per-pixel calculation when copying NV12 video buffer planes into your AI inference engine. This memory misalignment will warp or jitter the resulting bounding box. The tracking filter doesn't know you have a C++ pointer bug; it just sees massive statistical variance. Accurately modeling R prevents these upstream pipeline glitches from immediately tearing your track apart.

Process Noise (Q)

The Q matrix represents w_k. It defines how much we distrust our internal kinematic physics (the F matrix).

- In a Constant Velocity (CV) model, we assume acceleration is zero. But physics dictates that wind gusts and minor pilot inputs exist. The Q matrix injects just enough uncertainty into the velocities to keep the filter "open" to sudden changes.

The Tuning Dilemma

- **High Q, Low R:** The filter trusts the camera completely and ignores its own physics. The track will tightly follow the bounding box, but it will jitter violently with every pixel flutter.
- **Low Q, High R:** The filter trusts its physics completely and ignores the camera. The track will be beautifully smooth, but if the target suddenly executes a sharp turn, the track will lag behind and completely miss the maneuver.

C++ Implementation: Constructing the Process Noise Matrix (Q)

Unlike R, which is often a static diagonal matrix, Q must be derived analytically based on how continuous-time noise integrates over the discrete time step Δt. A standard approach for vision tracking is the Continuous White Noise Acceleration (CWNA) model, which assumes acceleration is a continuous white noise process with a spectral density magnitude of q.

For a 1D position/velocity system, the discrete Q matrix evaluates to:

$$Q = q \begin{bmatrix} \frac{\Delta t^3}{3} & \frac{\Delta t^2}{2} \\ \frac{\Delta t^2}{2} & \Delta t \end{bmatrix}$$

Here is how we map this efficiently into our 8-state CV model in C++:

```
#include <Eigen/Dense>
#include <cmath>

using namespace Eigen;

class NoiseModels {
public:
    // Generates the Process Noise Covariance Matrix (Q) for the 8-state CV model
    // dt: Time step
    // noise_magnitude: The tuning parameter (q) dictating expected maneuverability
    static MatrixXd getContinuousWhiteNoiseAccelerationQ(double dt, double noise_magnitude) {
        MatrixXd Q = MatrixXd::Zero(8, 8);

        double dt2 = dt * dt;
        double dt3 = dt2 * dt;

        // Positional Variance (dt^3 / 3)
        double pos_var = noise_magnitude * (dt3 / 3.0);
        // Velocity Variance (dt)
        double vel_var = noise_magnitude * dt;
        // Position-Velocity Covariance (dt^2 / 2)
        double covar = noise_magnitude * (dt2 / 2.0);

        // X-Axis
```

```
        Q(0, 0) = pos_var; Q(3, 3) = vel_var;
        Q(0, 3) = covar;   Q(3, 0) = covar;

        // Y-Axis
        Q(1, 1) = pos_var; Q(4, 4) = vel_var;
        Q(1, 4) = covar;   Q(4, 1) = covar;

        // Z-Axis
        Q(2, 2) = pos_var; Q(5, 5) = vel_var;
        Q(2, 5) = covar;   Q(5, 2) = covar;

        // Bounding Box Width/Height (Typically assigned tiny static drift values)
        Q(6, 6) = 1e-4;
        Q(7, 7) = 1e-4;

        return Q;
    }
};
```

Chapter 6

The Bayesian Filtering Framework

In Chapter 4, we explored Bayes' Theorem as a static concept—a way to fuse a single prediction with a single measurement. However, a target tracking system does not solve a single math problem; it processes a continuous, never-ending stream of video frames.

To track a dynamic target, we must turn Bayes' Theorem into a continuous loop. This is the **Recursive Bayes Filter**. It is the theoretical heartbeat of every tracking architecture, from the simplest 1D Kalman Filter to the most advanced Interacting Multiple Model Cubature Kalman Filter (IMM-CKF).

6.1 The recursive Bayes filter: Prediction and Update steps.

The recursive framework operates on a simple philosophy: *Today's Posterior becomes tomorrow's Prior*. Instead of recalculating the target's state from the very beginning of the flight sequence every time a new frame arrives, the filter condenses all historical knowledge into the current state estimate, takes one step forward in time, and processes the new measurement.

Let Z_k denote the set of all sensor measurements from time 0 up to time k. The recursive loop is divided into two distinct mathematical phases:

Phase 1: The Prediction Step (Time Update)

Before the camera captures frame k, the filter must predict where the target will be, based entirely on where it was in frame $k-1$ and our kinematic models.

Mathematically, we are calculating the **Prior PDF:** $p(\mathrm{x}_k \mid Z_{k-1})$.

This is achieved using the **Chapman-Kolmogorov Equation**:

$$p(\mathrm{x}_k \mid Z_{k-1}) = \int p(\mathrm{x}_k \mid \mathrm{x}_{k-1}) p(\mathrm{x}_{k-1} \mid Z_{k-1}) d\mathrm{x}_{k-1}$$

- $p(\mathrm{x}_{k-1} \mid Z_{k-1})$ is the final, updated state from the previous frame.
- $p(\mathrm{x}_k \mid \mathrm{x}_{k-1})$ is the State Transition Model (our kinematic physics and Process Noise Q from Chapter 5).
- **The Physical Reality:** During the prediction step, uncertainty always *grows*. Because we are moving blindly forward in time without new sensor data, process noise expands the covariance matrix. The target's "uncertainty bubble" inflates.

Deep Dive: The Chapman-Kolmogorov Derivation

We just stated that the Prediction Step relies on the Chapman-Kolmogorov equation to push the target's state forward in time. But where does this continuous integral actually come from? If you want to see how it is mathematically derived from the Law of Total Probability and the Markov Assumption, refer to **Appendix G: The Chapman-Kolmogorov Equation**.

Phase 2: The Update Step (Measurement Update)

The camera shutter fires and detection model does inference, We receive a new bounding box measurement, Z_k . We must now fuse this new data with our prediction to find the true state.

Mathematically, we are calculating the **Posterior PDF**: $p(x_k \mid Z_k)$.

This is achieved using **Bayes' Theorem**:

$$p(x_k \mid Z_k) = \frac{p(z_k \mid x_k)p(x_k \mid Z_{k-1})}{p(z_k \mid Z_{k-1})}$$

- $p(x_k \mid Z_{k-1})$ is the Prediction we just generated in Phase 1.
- $p(z_k \mid x_k)$ is the Likelihood (our Measurement Model and Sensor Noise R).
- The denominator $p(z_k \mid Z_{k-1})$ is the Evidence, the normalizing constant ensuring the total probability equals 1.0.
- **The Physical Reality:** During the update step, uncertainty always *shrinks*. The infusion of real-world data pins the target down, collapsing the covariance matrix.

If you log the determinant of your covariance matrix ($|P|$) during a live intercept, you will see it literally "breathe"—expanding during the prediction step, and contracting sharply during the update step.

6.2 Why closed-form solutions are rare (the integration problem).

If the recursive Bayes equations are the ultimate, optimal way to track a target, why do we need hundreds of different Kalman Filter variations? Why can't we just write a C++ function to solve these two equations directly?

The answer lies in the terrifying mathematical machinery hiding inside the denominator of the Update step (the Evidence). To calculate the total probability of the measurement, we must integrate the numerator across the entire infinite state space:

$$p(z_k \mid Z_{k-1}) = \int p(z_k \mid x_k)p(x_k \mid Z_{k-1})dx_k$$

The Mathematical Roadblock

An integral calculates the area under a continuous curve. To solve an integral analytically (a "closed-form" solution), you must be able to find the anti-derivative of the functions inside it.

1. **The Linear Exception:** If—and *only* if—your kinematic models are perfectly linear (e.g., the Constant Velocity matrix F), and your measurement models are perfectly linear (the H matrix), and all noises are perfectly Gaussian, then the math is beautiful. The integrals of linear Gaussians result in more linear Gaussians. The integrals vanish algebraically, collapsing into the simple, static matrix multiplications of the standard **Kalman Filter (KF)**.

2. **The Non-Linear Reality:** In modern aerospace tracking, systems are highly non-linear. The Coordinated Turn (CT) model uses sine and cosine functions to couple velocities. Visual AI sensors project a 3D world onto a 2D pixel plane using highly non-linear pinhole camera geometry.

When you shove trigonometric functions and perspective divisions inside a Gaussian exponent, it creates a wildly warped probability distribution. **This non-linear integral has no analytical solution.** It is mathematically impossible to solve with algebra.

> **Deep Dive: The Calculus of Non-Linear Integration**
>
> It is one thing to say an integral is unsolvable; it is another to see it fail on paper. If you want to look at the exact calculus of why this happens—specifically using the non-linear bearing angle geometry ($\arctan(Y/X)$) of a monocular tracking camera—refer to **Appendix H: The Analytical Impossibility of Non-Linear Integration**.

The Engineering Dilemma

If we cannot solve the integral algebraically, the CPU must approximate it numerically.

Historically, engineers solved this by cheating the physics: the **Extended Kalman Filter (EKF)** uses Jacobian matrices (calculus derivatives) to forcibly flatten the non-linear physics into linear tangents, essentially pretending the problem is linear so the standard integrals work. However, for highly evasive targets and complex camera projections, this forced linearization introduces massive truncation errors, causing the filter's covariance to collapse and the track to diverge.

To achieve superior accuracy without analytical integrals, we must stop trying to linearize the math, and instead mathematically approximate the probability distribution itself.

This exact integration roadblock is the birthplace of the **Cubature Kalman Filter (CKF)**. Instead of attempting impossible calculus, the CKF uses the Spherical-Radial cubature rule to carefully select a deterministic set of $2n$ points (where n is the state dimension). By pushing these specific points through the true, uncorrupted non-linear equations, the CKF computes the exact expected value of the unsolvable integral with incredible accuracy and low CPU overhead.

> **Deep Dive: The Spherical-Radial Derivation**
>
> We just stated that converting to spherical coordinates miraculously collapses an infinite Gaussian integral down to exactly $2n$ points. But how does the calculus actually prove this? And how do we prove that the radius of the points must be exactly $\sqrt{n}$ to perfectly capture the system's variance? For the rigorous mathematical proof involving Gaussian-Laguerre quadrature and moment-matching, refer to **Appendix A: Derivation of the Spherical-Radial Cubature Rule**.

Part III: The Classic Standard Kalman Filter (KF)

Chapter 7

Derivation of the Standard KF

In Chapter 6, we hit a mathematical wall. The denominator of Bayes' Theorem (the Evidence integral) is analytically impossible to solve for non-linear systems. The Cubature Kalman Filter (CKF) will eventually be our numerical solution to that problem.

However, before we can understand the CKF, we must look at the mathematical miracle that occurs when a dynamic system is perfectly linear. When the math behaves, the infinite integrals of Bayes' Theorem beautifully collapse into a set of five simple matrix equations. This is the **Standard Kalman Filter (KF)**.

7.1 The Linear Gaussian Assumption

To derive the standard KF from Bayesian principles, we must make two absolute, uncompromising assumptions about our target tracking system:

1. **Hypothesis A: Linearity:** The State Transition Matrix (F) and the Measurement Matrix (H) must be perfectly linear. A constant velocity drone moving across a 2D radar screen is linear. A drone executing a banked turn measured by a monocular camera's bearing angle (arctan) is non-linear.

- **The State Transition Model:** $\mathrm{x}_k = F\mathrm{x}_{k-1} + \mathrm{w}_k$
- **The Measurement Model:** $\mathrm{z}_k = H\mathrm{x}_k + \mathrm{v}_k$

  (*A constant velocity drone moving across a 2D radar screen is linear. A drone executing a banked turn measured by a monocular camera's non-linear bearing angle* (arctan) *violates this assumption*).

2. **Hypothesis B:Gaussian Noise:** The Process Noise (Q) and the Measurement Noise (R) must be normally distributed bell curves with a mean of zero. They cannot be heavily skewed by systematic sensor bias.

- **Process Noise:** $\mathrm{w}_k \sim \mathcal{N}(0, Q)$
- **Measurement Noise:** $\mathrm{v}_k \sim \mathcal{N}(0, R)$

(*This means the noise must be truly random. It cannot be heavily skewed or offset by a systematic sensor bias*).

The Mathematical Miracle:

If—and only if—these two conditions are met, a profound rule of statistics takes over: *Any linear transformation of a Gaussian distribution results in another perfect Gaussian distribution.* Because everything in the pipeline remains a

perfect bell curve, we never actually have to solve the impossible integrals of the Recursive Bayes Filter. We only have to track two parameters: the Mean ($\hat{x}$) and the Covariance (P).

7.2 The Minimum Mean Square Error (MMSE) Estimator

Before we derive the equations, we must answer a critical question: What makes the Kalman Filter "optimal"?

In estimation theory, optimality is defined by a loss function. We want an estimator that penalizes errors. The most common penalty is the squared error: if our prediction is off by 2 meters, the penalty is 4; if it is off by 10 meters, the penalty is 100. Squaring the error heavily punishes large deviations.

To understand why we define "optimality" using a loss function, we have to step away from the pure math for a second and look at the philosophy of engineering.

When you build a tracking system, your filter is constantly generating a state estimate (x^). Because sensors have noise, your estimate will never be exactly equal to the true state (x). You will always be wrong by some amount.

Since you cannot be perfect, you have to decide how you want to be wrong. A Loss Function (or Cost Function), denoted as L(x,x^), is the mathematical penalty you assign to your filter for making an error.

By changing the rules of the penalty, you completely change what the filter considers to be the "optimal" guess. Here are the three classic loss functions in estimation theory:

1. **The Quadratic Loss: Minimum Mean Square Error (MMSE)**

- **The Penalty**: $L = (x - \hat{x})^2$
- **The Philosophy**: "Small errors are acceptable, but large errors are catastrophic." Because the error is squared, being off by 1 meter adds a penalty of 1. Being off by 10 meters adds a massive penalty of 100.
- **The Optimal Point**: To minimize this specific penalty across a probability distribution, the math dictates that your best guess is the Mean (the center of mass/average).
- **The Engineering Use**: This is what the Kalman Filter uses. In aerospace, an interceptor missile can easily adjust its fins to correct a 1-meter tracking error. But a 10-meter error means the target escapes the missile's seeker cone entirely. We heavily punish large deviations.

2. **The Absolute Loss: Mean Absolute Error (MAE)**

- **The Penalty** : $L = |x - \hat{x}|$
- **The Philosophy**: "All errors are penalized proportionally." Being off by 10 meters is exactly ten times worse than being off by 1 meter. There is no exponential explosion of the penalty.
- **The Optimal Point**: To minimize this penalty, the math dictates that your best guess is the Median (the exact middle value, where 50% of the probability is to the left and 50% is to the right).
- **The Engineering Use**: This is used in robust estimation where you expect massive sensor outliers. If your radar occasionally glitches and reports the drone is 5,000 miles away, squaring that error (MMSE) would destroy your filter. Absolute loss ignores the extreme leverage of the outlier.

3. **The Uniform Loss: Maximum A Posteriori (MAP)**

- **The Penalty**: "Hit or Miss." The penalty is 0 if you are exactly right, and 1 if you are wrong by any amount.
- **The Philosophy**: "I only care about being exactly right. A miss by 1 inch is identical to a miss by a mile."
- **The Optimal Point**: To minimize this penalty, your best guess is the Mode (the absolute highest peak of the probability distribution).
- **The Engineering Use**: This is heavily used in classification and computer vision (e.g., "Is this bounding box a Drone or a Bird?"). You cannot be "average" between a bird and a drone; you must pick the single most likely category.

The Gaussian Miracle (Why Kalman Chose MMSE)

If different loss functions result in different "optimal" answers, why is the Kalman Filter locked into the MMSE?

This is where the magic of the **Linear Gaussian Assumption (Section 7.1)** comes into play. For a perfectly symmetrical, normal Gaussian bell curve, the Mean, the Median, and the Mode are all located at the exact same physical point (the center peak).

For a linear Kalman filter, it mathematically does not matter which of the three loss functions you choose. By solving for the MMSE, Rudolf Kalman essentially solved for all of them simultaneously. However, the squared error (x2) is mathematically much easier to differentiate and minimize via calculus than an absolute value (|x|), which is why the MMSE is the gold standard for linear derivation.

The **Minimum Mean Square Error (MMSE)** estimator is the mathematical function that finds the state estimate $\hat{x}$ that minimizes the expected value of the squared error:

$$J = E\left[(x - \hat{x})^T (x - \hat{x})\right]$$

For any generic probability distribution, proving the MMSE is difficult. However, for a Gaussian distribution, the MMSE estimate is simply the **mean** (the peak of the bell curve). Because Bayes' Theorem fused our linear models into a perfect Posterior Gaussian, finding the peak of that Posterior gives us the mathematically guaranteed lowest possible tracking error.

The Engineering Dilemma: Multimodal Failure

The MMSE is brilliant, but it is entirely dependent on the Gaussian assumption. Consider tracking a UAV that flies behind a dense cloud. You predict it might keep flying straight (Hypothesis A), or it might turn left to hide in a valley (Hypothesis B).

Your true probability distribution is now *multimodal*—it has two distinct peaks, like a camel's back. If you blindly apply an MMSE estimator (like the Kalman Filter) to a multimodal distribution, it will average the two peaks together. It will calculate the "optimal" mean state as being located perfectly in the empty sky between the straight path and the valley path. The MMSE will confidently track a target that physically does not exist. This is why we will eventually need the IMM-CKF architecture to manage multiple distinct models.

7.3 Derivation of the Standard Kalman Filter:

With the MMSE and Linear assumptions established, we can derive the Kalman Filter directly from the recursive Bayes equations.

We start with the Prediction Step (the Chapman-Kolmogorov equation). Because our transition model $\mathbf{x}_k = F\mathbf{x}_{k-1} + \mathbf{w}_k$ is linear and Gaussian, we simply apply the Expectation operator ($E[\cdot]$) to find the new Mean and Covariance:

1. ** Predicted Mean:**

$$\hat{\mathbf{x}}_{k|k-1} = E[F\mathbf{x}_{k-1} + \mathbf{w}_k]$$

Because the noise has a mean of zero ($E[\mathbf{w}_k] = 0$), this simplifies to:

$$\hat{\mathbf{x}}_{k|k-1} = F\hat{\mathbf{x}}_{k-1|k-1}$$

The Update Step (Bayes' Theorem) requires fusing the predicted state with the measurement $\mathbf{z}_k$. We calculate the Residual (Innovation), which is the difference between what the sensor saw and what we predicted it would see:

$$\mathbf{y}_k = \mathbf{z}_k - H\hat{\mathbf{x}}_{k|k-1}$$

To minimize the MMSE loss function, we want to add a fraction of this residual to our predicted state. This optimal fraction is the **Kalman Gain** (K).

> **Deep Dive: The Algebraic Derivation of the Kalman Gain**
>
> In Chapter 4, we showed geometrically how the 1D Kalman Gain is born from multiplying two scalar Gaussian equations. Scaling this proof up to multidimensional matrices requires taking the derivative of the MMSE loss function and setting it to zero. For the rigorous, step-by-step matrix calculus proving the derivation of the multidimensional Kalman Gain, refer to **Appendix I: Matrix Derivation of the Kalman Filter**.

7.4 The Standard Algorithm and C++ Implementation

The derivations result in the five legendary equations of the Standard Linear Kalman Filter. Every tracking engineer must know these by heart.

Prediction Step (Time Update):

1. State Prediction: $\hat{\mathbf{x}}_{k|k-1} = F\hat{\mathbf{x}}_{k-1|k-1}$
2. Covariance Prediction: $P_{k|k-1} = FP_{k-1|k-1}F^T + Q$

Update Step (Measurement Update):

3. Innovation Covariance & Kalman Gain:

$$S_k = HP_{k|k-1}H^T + R$$

$$K_k = P_{k|k-1}H^T S_k^{-1}$$

4. State Update: $\hat{\mathbf{x}}_{k|k} = \hat{\mathbf{x}}_{k|k-1} + K_k(\mathbf{z}_k - H\hat{\mathbf{x}}_{k|k-1})$
5. Covariance Update: $P_{k|k} = (I - K_kH)P_{k|k-1}$

High-Performance C++ Implementation Translating these five equations into code is incredibly straightforward using a modern linear algebra library. Here is a highly optimized Eigen implementation for an edge-deployed UAV interceptor.

Notice that we compute the inverse of the S matrix (`S.inverse()`). Because S has the dimensions of the measurement space (e.g., a 2×2 matrix for an X/Y radar hit), this inversion is computationally cheap, making the linear KF incredibly fast.

```
#include <Eigen/Dense>

using namespace Eigen;

class LinearKalmanFilter {
private:
    VectorXd x; // State estimate
    MatrixXd P; // Estimate covariance

    MatrixXd F; // State transition matrix
    MatrixXd Q; // Process noise covariance
    MatrixXd H; // Measurement matrix
    MatrixXd R; // Measurement noise covariance
    MatrixXd I; // Identity matrix

public:
    // Constructor to initialize matrix dimensions
    LinearKalmanFilter(int state_dim, int meas_dim) {
        x = VectorXd::Zero(state_dim);
        P = MatrixXd::Identity(state_dim, state_dim);
        F = MatrixXd::Identity(state_dim, state_dim);
        Q = MatrixXd::Identity(state_dim, state_dim);
        H = MatrixXd::Zero(meas_dim, state_dim);
        R = MatrixXd::Identity(meas_dim, meas_dim);
        I = MatrixXd::Identity(state_dim, state_dim);
    }

    // Phase 1: Prediction Step
    void predict() {
        x = F * x;
        P = F * P * F.transpose() + Q;
    }

    // Phase 2: Update Step
    void update(const VectorXd& z) {
        // Calculate Innovation
        VectorXd y = z - (H * x);

        // Calculate Innovation Covariance (S)
        MatrixXd S = H * P * H.transpose() + R;
```

```
        // Calculate Kalman Gain (K)
        MatrixXd K = P * H.transpose() * S.inverse();

        // Update State
        x = x + (K * y);

        // Update Covariance using the numerically stable Joseph Form
        // P = (I - K*H) * P
        MatrixXd I_KH = I - (K * H);
        P = I_KH * P;
    }

    // Setters for matrices omitted for brevity...
};
```

Chapter 8

Tuning and Diagnostics

No tracking filter survives first contact with real-world sensor data. A Kalman Filter derived on a whiteboard assumes that we perfectly know the Process Noise (Q) and the Measurement Noise (R). In reality, these matrices are almost always educated guesses.

If your filter equations are perfectly coded but your Q and R matrices are poorly tuned, the filter will either violently jitter with sensor noise or sluggishly trail behind an accelerating target. This chapter bridges the gap between pure mathematics and practical engineering, detailing how to tune a filter, how to mathematically prove it is tuned correctly, and how to prevent floating-point rounding errors from destroying your covariance matrices.

8.1 The Tug-of-War: Q vs. R

Tuning a tracking filter is essentially a tug-of-war between your kinematic model (F) and your sensor data (z). The rope they are pulling on is the Kalman Gain (K).

$$K_k = P_{k|k-1}H^T(HP_{k|k-1}H^T + R)^{-1}$$

Look at the denominator: the Measurement Noise R. Now remember that $P_{k|k-1}$ is directly driven by the Process Noise Q.

- **High R, Low Q (Trust the Model):** If you tell the filter that your sensors are terrible (large R) and your physical model is perfect (small Q), the Kalman Gain approaches zero. The filter ignores incoming measurements and blindly flies along its predicted trajectory. The output will be incredibly smooth, but if the target maneuvers, the filter will **lag** severely and lose the track.
- **Low R, High Q (Trust the Sensor):** If you tell the filter your sensors are flawless (small R) and your model is highly uncertain (large Q), the Kalman Gain approaches H^{-1}. The filter ignores its physics model and snaps the state estimate directly to the noisy sensor measurements. The filter will perfectly track rapid maneuvers, but the output will be violently **jittery**.

The Engineering Reality: R is usually known. You can look at a radar's hardware datasheet and find its baseline variance (e.g., $\sigma = 5$ meters). Therefore, the art of filter tuning almost entirely consists of artificially scaling the Process Noise matrix Q to balance lag and jitter.

8.2 Innovation Analysis and the NIS Statistic

Manual tuning by "eyeballing" the lag and jitter on a screen is dangerous. We need a rigorous, mathematical diagnostic to tell us if our Q matrix is correctly balanced against our R matrix.

This is achieved using **Innovation Analysis**. The Innovation (or Residual), y_k, is the difference between what the sensor measured and what the filter predicted:

$$\mathrm{y}_k = \mathrm{z}_k - H\hat{\mathrm{x}}_{k|k-1}$$

The filter also calculates how much variance we *expect* to see in this innovation, defined as the Innovation Covariance (S_k):

$$S_k = HP_{k|k-1}H^T + R$$

If the filter is perfectly tuned, the actual error (y_k) should be statistically consistent with the expected error (S_k). We measure this consistency using the **Normalized Innovation Squared (NIS)** statistic:

$$\epsilon_k = \mathrm{y}_k^T S_k^{-1} \mathrm{y}_k$$

The Mathematics:

Because y_k is assumed to be a zero-mean Gaussian, normalizing it by its covariance S_k transforms the NIS (ϵ_k) into a **Chi-Square (χ^2) distribution**. The degrees of freedom (m) of this distribution equal the number of dimensions in your measurement vector (e.g., $m = 2$ for an X/Y radar hit).

If you plot the NIS values over time during a tracking run:

1. **NIS is consistently too high:** The actual errors are much larger than the filter expects. The filter is overconfident in its model. You need to **increase Q**.
2. **NIS is consistently too low:** The actual errors are much smaller than the filter expects. The filter is underconfident. You need to **decrease Q**.

The NIS is also the primary mechanism for **Measurement Gating**. By checking a Chi-Square table, we know that for a 2D measurement, 95% of all valid sensor hits will yield an NIS less than 5.99. If a radar blip produces an NIS of 25.0, the filter instantly knows it is a false alarm (clutter) and mathematically rejects it before it corrupts the state.

8.3 Filter Divergence and Adaptive Covariance Matching

Filter Divergence occurs when the estimation error grows exponentially over time, but the filter's internal covariance matrix (P) shrinks toward zero. The filter becomes completely blind to reality, adamantly believing it knows exactly where the target is, while the true target flies away.

Divergence is usually caused by unmodeled target maneuvers (e.g., a target suddenly pulling 9Gs when the filter assumes constant velocity).

To prevent divergence, advanced tracking systems use **Adaptive Covariance Matching**. Instead of hard-coding Q and R before the flight, the filter monitors the NIS in real-time. If the NIS spikes (indicating a sudden maneuver), the filter dynamically recalculates and injects massive amounts of Process Noise (Q) into the system on the fly to force the Kalman Gain to open up and catch the maneuvering target.

Deep Dive: The Mathematics of Adaptive Tuning

Calculating new Q and R matrices dynamically requires taking moving-window averages of the innovation sequence and solving backward through the covariance equations. For the rigorous mathematical derivation of the Myers-Tapley Covariance Matching algorithm, refer to **Appendix J: Adaptive Covariance Matching**.

8.4 Numerical Stability and the Joseph Form

Even if your filter is perfectly tuned, it can still diverge and crash due to computer science limitations.

The Covariance Matrix (P) is a mathematical representation of physical uncertainty. Therefore, by definition, it must be **Symmetric** and **Positive Definite** (all its eigenvalues must be strictly greater than zero). You cannot have "negative uncertainty."

The standard covariance update equation is:

$$P_{k|k} = (I - K_k H)P_{k|k-1}$$

The Engineering Dilemma:

This equation requires subtracting a matrix from the Identity matrix. In 32-bit or 64-bit floating-point C++ environments, repeated subtraction over thousands of iterations causes microscopic rounding errors (truncation). Over time, these truncation errors accumulate. The P matrix will eventually lose its symmetry, develop negative eigenvalues, and instantly crash your software with a `NaN` (Not a Number) failure during the next Cholesky decomposition.

To mathematically guarantee numerical stability in production software, engineers use the **Joseph Form** update equation (derived in Appendix J):

$$P_{k|k} = (I - K_k H)P_{k|k-1}(I - K_k H)^T + K_k R K_k^T$$

While it requires more matrix multiplications (costing slightly more CPU time), the Joseph form adds two strictly positive-definite matrices together ($APA^T + BRB^T$). Because it utilizes addition rather than subtraction, it is mathematically impossible for rounding errors to cause negative eigenvalues.

C++ Implementation:

Here is the production-ready C++ code implementing both NIS gating and the Joseph Form stability update.

```
#include <Eigen/Dense>

using namespace Eigen;

class RobustKalmanFilter {
private:
    VectorXd x; // State estimate
    MatrixXd P; // Covariance
    MatrixXd H; // Measurement Matrix
    MatrixXd R; // Measurement Noise
```

```
    MatrixXd I; // Identity Matrix

    // Chi-Square 95% threshold for 2 Degrees of Freedom (e.g., X, Y radar)
    const double CHI_SQUARE_95_2DOF = 5.991;

public:
    // ... Constructor and Predict step omitted for brevity ...

    // Update Step with NIS Gating and Joseph Form stability
    bool updateRobust(const VectorXd& z) {
        // 1. Calculate Innovation (y) and Innovation Covariance (S)
        VectorXd y = z - (H * x);
        MatrixXd S = H * P * H.transpose() + R;
        MatrixXd S_inv = S.inverse();

        // 2. Innovation Analysis (Calculate NIS)
        // NIS = y^T * S^-1 * y
        double nis = y.transpose() * S_inv * y;

        // 3. Measurement Gating
        // If NIS exceeds statistical threshold, reject the measurement!
        if (nis > CHI_SQUARE_95_2DOF) {
            // Measurement is likely clutter or a severe glitch.
            // Do not update the state. Return false indicating rejection.
            return false;
        }

        // 4. Calculate Kalman Gain
        MatrixXd K = P * H.transpose() * S_inv;

        // 5. Update State
        x = x + (K * y);

        // 6. Joseph Form Covariance Update (Guarantees Numerical Stability)
        // P = (I - K*H) * P * (I - K*H)^T + K * R * K^T
        MatrixXd I_KH = I - (K * H);
        P = I_KH * P * I_KH.transpose() + K * R * K.transpose();

        return true; // Successfully fused measurement
    }
};
```

Part IV: Conquering Non-Linearity

Chapter 9

The Extended Kalman Filter (EKF)

In Part III, we proved that the Standard Kalman Filter is the optimal minimum-mean-square-error (MMSE) estimator, but only if the system is perfectly linear.

In aerospace and robotics, perfect linearity does not exist. A drone banking into a coordinated turn utilizes sine and cosine functions. A ground-based radar tracking a satellite measures slant range (a square root function) and elevation angles (an arctangent function).

The Extended Kalman Filter (EKF) was the aerospace industry's first great triumph over non-linear geometry. It powered the Apollo navigation computers and remains one of the most widely deployed algorithms in embedded systems today.

9.1 Linearizing non-linear functions using the Jacobian

In a non-linear system, we replace our constant matrices (F and H) with generic, non-linear mathematical functions, denoted as $f(\cdot)$ and $h(\cdot)$.

- **Non-Linear State Transition:** $x_k = f(x_{k-1}) + w_k$
- **Non-Linear Measurement:** $z_k = h(x_k) + v_k$

Recall the statistical rule that governed Chapter 7.1: *Any linear transformation of a Gaussian distribution results in another perfect Gaussian distribution.* The rule that governs Chapter 9 is: *A non-linear transformation of a Gaussian is NOT a Gaussian.*

If you push a perfectly bell-shaped Gaussian probability distribution through an arctangent function, the output is a warped, skewed, asymmetric lump. Because the Kalman Filter requires the tracking covariance to remain a perfect, symmetric ellipse, non-linear functions mathematically break the recursive loop.

Linearization via Calculus

To fix this, the EKF essentially cheats the geometry. It accepts that it cannot push a covariance matrix through a curve, so it uses calculus to turn the curve into a straight line.

This is achieved using a **First-Order Taylor Series Expansion**. By calculating the partial derivatives of the non-linear function evaluated exactly at our current state estimate, we find the multi-dimensional tangent line.

This matrix of partial derivatives is called the **Jacobian**. We must calculate two Jacobians during every single time step:

1. **The State Transition Jacobian (F_k):**

$$F_k = \left.\frac{\partial f(x)}{\partial x}\right|_{x=\hat{x}_{k-1|k-1}}$$

2. **The Measurement Jacobian (H_k):**

$$H_k = \left.\frac{\partial h(x)}{\partial x}\right|_{x=\hat{x}_{k|k-1}}$$

By evaluating the derivative at our current best guess of the target's location, we create a temporary linear matrix (F_k or H_k) that acts as a "flat mirror" of the curved space at that exact coordinate.

The EKF Algorithm

The genius of the EKF is that it splits the tracking math in half.

1. **The State:** We push the mean state vector ($\hat{x}$) through the true, uncorrupted, non-linear functions (f and h) to ensure our physical location is as accurate as possible.
2. **The Covariance:** We push the uncertainty matrix (P) through the flattened, linear Jacobian matrices (F_k and H_k) to ensure the math remains a perfect Gaussian.

Prediction Step:

1. State: $\hat{x}_{k|k-1} = f(\hat{x}_{k-1|k-1})$
2. Covariance: $P_{k|k-1} = F_k P_{k-1|k-1} F_k^T + Q$

Update Step:

3. Innovation & Gain:

$$S_k = H_k P_{k|k-1} H_k^T + R$$
$$K_k = P_{k|k-1} H_k^T S_k^{-1}$$

4. State: $\hat{x}_{k|k} = \hat{x}_{k|k-1} + K_k(z_k - h(\hat{x}_{k|k-1}))$
5. Covariance: $P_{k|k} = (I - K_k H_k) P_{k|k-1}$

9.2 Mathematical proof and limitations of the EKF (truncation errors)

If the EKF is so brilliant, why did the aerospace industry spend millions of dollars inventing the Unscented and Cubature Kalman Filters to replace it? The EKF has a fatal mathematical flaw: **Truncation Error**.

When we use a Taylor Series to create our tangent line (Jacobian), we delete all the higher-order derivatives (the acceleration of the curve, the jerk, etc.). We truncate the math.

A tangent line is only an accurate representation of a curve if you are extremely close to the exact point of the tangent. If your target is flying smoothly and your sensor is highly accurate (your Covariance P is tiny), the EKF works perfectly because your entire "uncertainty bubble" fits safely near the tangent point.

However, if your target pulls a high-G evasive maneuver, or if your sensor goes blind and your Covariance P expands massively, your uncertainty bubble spreads out. At the edges of that wide bubble, the straight tangent line is nowhere near the true curved geometry.

When the EKF pushes a wide covariance through a tangent line that doesn't match reality, the filter calculates the wrong Kalman Gain, updates in the wrong direction, and suffers catastrophic **Filter Divergence**.

Engineering Example & C++ Code: Radar Tracking

Let's look at the classic EKF engineering test case: A 2D radar tracking an aircraft. The aircraft flies in Cartesian coordinates (X, Y), so our state vector is $x = [x, y, v_x, v_y]^T$. However, the radar measures in Polar coordinates: slant range (r) and bearing angle (θ). Our measurement vector is $z = [r, \theta]^T$.

The non-linear measurement function $h(x)$ mapping the Cartesian state to the Polar sensor is:

$$h(x) = \begin{bmatrix} \sqrt{x^2 + y^2} \\ \arctan\left(\frac{y}{x}\right) \end{bmatrix}$$

Because this is non-linear, we must calculate the 2×4 Measurement Jacobian matrix (H_k) by taking the partial derivatives of r and θ with respect to x and y.

> **Deep Dive: EKF Matrix Derivations and the Radar Jacobian**
>
> We have stated that the EKF requires a discrete State Transition Jacobian (F_k) and a Measurement Jacobian (H_k). But how do we actually discretize a continuous-time non-linear physics model to find F_k? And what is the exact step-by-step calculus required to derive the 2×4 Polar-to-Cartesian radar Jacobian? For the rigorous mathematical proofs and derivative calculations, refer to **Appendix K: EKF Derivations: Matrices and Jacobians**.

C++ Implementation

Notice in the code below how the true non-linear $h(x)$ is used to calculate the innovation vector `y`, but the linear Jacobian `H_j` is used to calculate the covariance `S` and the Kalman Gain `K`.

```
#include <Eigen/Dense>
#include <cmath>

using namespace Eigen;

class RadarEKF {
private:
    VectorXd x; // State: [x, y, vx, vy]
    MatrixXd P; // Covariance
    MatrixXd R; // Radar Noise (Range variance, Bearing variance)

public:
    // ... Constructor and Linear Prediction Step omitted for brevity ...

    void updateRadar(double range, double bearing) {
        // Extract current state estimates
        double px = x(0);
        double py = x(1);
```

```
        // Prevent divide-by-zero if target is exactly on top of the radar
        double r2 = px*px + py*py;
        if (r2 < 0.0001) r2 = 0.0001;
        double r = std::sqrt(r2);

        // 1. Calculate the Measurement Jacobian (H_j)
        MatrixXd H_j = MatrixXd::Zero(2, 4);
        H_j(0, 0) = px / r;
        H_j(0, 1) = py / r;
        // Velocities do not affect position measurement, so H_j(0,2) and H_j(0,3) are 0

        H_j(1, 0) = -py / r2;
        H_j(1, 1) = px / r2;

        // 2. Calculate Predicted Measurement using True Non-Linear h(x)
        VectorXd z_pred(2);
        z_pred(0) = r;
        z_pred(1) = std::atan2(py, px);

        // 3. Calculate Innovation
        VectorXd z_meas(2);
        z_meas << range, bearing;
        VectorXd y = z_meas - z_pred;

        // Normalize bearing innovation to stay within [-pi, pi]
        while (y(1) > M_PI) y(1) -= 2.0 * M_PI;
        while (y(1) < -M_PI) y(1) += 2.0 * M_PI;

        // 4. Update Math (Using the Jacobian H_j!)
        MatrixXd S = H_j * P * H_j.transpose() + R;
        MatrixXd K = P * H_j.transpose() * S.inverse();

        x = x + (K * y);
        MatrixXd I = MatrixXd::Identity(4, 4);
        P = (I - K * H_j) * P; // (Use Joseph form in production)
    }
};
```

Chapter 10

The Unscented Kalman Filter (UKF)

In 1997, Simon Julier and Jeffrey Uhlmann published a paper that fundamentally changed aerospace estimation. They identified the fatal flaw of the Extended Kalman Filter (EKF) and proposed a completely new philosophy for handling non-linear geometry.

Their philosophy was summarized in a single, powerful sentence: *"It is easier to approximate a probability distribution than it is to approximate an arbitrary non-linear function or transformation."*

Instead of using calculus to flatten a curved physical space into a straight line (the EKF approach), the Unscented Kalman Filter (UKF) leaves the physical curves completely uncorrupted. Instead, it approximates the Gaussian uncertainty bubble itself using a deterministic set of sample points.

10.1 The Unscented Transform (UT)

The core mechanism of the UKF is the **Unscented Transform (UT)**. Jeffrey Uhlmann initially proposed the unscented transform (UT) as a component of his PhD thesis [7]; however, it is predominantly known from [3]. If we have a state vector x (with mean $\hat{\mathrm{x}}$ and covariance P) that must pass through a highly non-linear function $\mathrm{y} = g(\mathrm{x})$, we execute the UT using four rigorous steps.

Step 1: Select a set of points from the input distribution.

We cannot pass a continuous matrix (P) through a curve, so we must discretize it. The UT selects exactly $2n + 1$ deterministic vectors called **Sigma Points** ($\mathcal{X}$), where n is the dimension of the state. One point is placed exactly at the mean. The remaining $2n$ points are spread symmetrically outward along the principal axes of the covariance ellipsoid.

To dictate how far these points spread out, we use a composite scaling factor, $\lambda = \alpha^2(n + \kappa) - n$. (Where α determines the spread, and κ is a secondary scaling parameter).

To find the axes of the ellipse, we take the Cholesky Decomposition of the covariance matrix ($L = \sqrt{P}$). The Sigma Points are generated as:

$$\mathcal{X}_0 = \hat{\mathrm{x}}$$

$$\mathcal{X}_i = \hat{\mathrm{x}} + \left(\sqrt{(n + \lambda)P}\right)_i \quad \text{for } i = 1 \dots n$$

$$\mathcal{X}_{i+n} = \hat{\mathrm{x}} - \left(\sqrt{(n + \lambda)P}\right)_i \quad \text{for } i = 1 \dots n$$

(*Where subscript i denotes the i-th column of the lower-triangular matrix L*).

Step 2: Propagate each selected point through the non-linear function.

This is the genius of the UKF. We abandon Taylor series derivatives entirely. We simply take our $2n+1$ Sigma Points and push them through the true, uncorrupted non-linear physics or camera projection equations.

$$\mathcal{Y}_i = g(\mathcal{X}_i) \quad \text{for } i = 0 \dots 2n$$

This produces a new set of transformed points $(\mathcal{Y})$ that perfectly map to the warped, non-linear output distribution.

Step 3: Compute sigma point weights.

Because our points were artificially spread out based on our tuning parameters (α, κ), they are not equal in value. We must assign specific mathematical weights (W) to them to properly reconstruct the statistics later. We calculate separate weights for reconstructing the Mean (m) and the Covariance (c):

- Weight for the central point (Mean): $W_0^{(m)} = \frac{\lambda}{n+\lambda}$
- Weight for the central point (Covariance): $W_0^{(c)} = \frac{\lambda}{n+\lambda} + (1 - \alpha^2 + \beta)$
- Weights for all $2n$ outer points: $W_i^{(m)} = W_i^{(c)} = \frac{1}{2(n+\lambda)}$

(*Note: β incorporates prior knowledge of the distribution. For a Gaussian distribution, $\beta = 2$ is optimal*).

Step 4: Approximate the sample mean and covariance of the output distribution.

We reconstruct the final, transformed probability distribution by taking a weighted sum of the propagated points.

- The new Mean is the weighted sum of the points:

$$\hat{\mathrm{y}} = \sum_{i=0}^{2n} W_i^{(m)} \mathcal{Y}_i$$

- The new Covariance is the weighted sum of their squared deviations from the new mean:

$$P_y = \sum_{i=0}^{2n} W_i^{(c)} [\mathcal{Y}_i - \hat{\mathrm{y}}][\mathcal{Y}_i - \hat{\mathrm{y}}]^T$$

Deep Dive: The Mathematical Proof of UKF Superiority

Why does this 4-step algorithm work better than the EKF's calculus? The Unscented Transform mathematically guarantees that the calculated mean and covariance perfectly match the true non-linear distribution up to the 3rd order of a Taylor series expansion, whereas the EKF only matches the 1st order. For the formal proof of this precision, refer to **Appendix L: Taylor Series Verification of the Unscented Transform**.

10.2 The UKF Algorithm

To build a full tracking filter, we simply apply the 4-step UT process to the standard Kalman Filter equations.

Prediction Step:

1. Generate Sigma Points $(\mathcal{X}_{k-1})$ from previous state $\hat{\mathrm{x}}_{k-1|k-1}$ and $P_{k-1|k-1}$.
2. Propagate through the non-linear physics model: $\mathcal{X}_{k|k-1}^{(i)} = f(\mathcal{X}_{k-1}^{(i)})$.
3. Approximate Predicted Mean: $\hat{\mathrm{x}}_{k|k-1} = \sum_{i=0}^{2n} W_i^{(m)} \mathcal{X}_{k|k-1}^{(i)}$

4. Approximate Predicted Covariance: $P_{k|k-1} = \sum_{i=0}^{2n} W_i^{(c)} [\mathcal{X}_{k|k-1}^{(i)} - \hat{\mathrm{x}}_{k|k-1}][\dots]^T + Q$

Update Step:

5. Generate *new* Sigma Points using the newly predicted $P_{k|k-1}$.
6. Propagate points through non-linear sensor model: $\mathcal{Z}_k^{(i)} = h(\mathcal{X}_{k|k-1}^{(i)})$.
7. Approximate Predicted Measurement: $\hat{\mathrm{z}}_k = \sum_{i=0}^{2n} W_i^{(m)} \mathcal{Z}_k^{(i)}$
8. Calculate Innovation Covariance (S) and Cross-Covariance (P_{xz}):

$$S_k = \sum_{i=0}^{2n} W_i^{(c)} [\mathcal{Z}_k^{(i)} - \hat{\mathrm{z}}_k][\dots]^T + R$$

$$P_{xz} = \sum_{i=0}^{2n} W_i^{(c)} [\mathcal{X}_{k|k-1}^{(i)} - \hat{\mathrm{x}}_{k|k-1}][\mathcal{Z}_k^{(i)} - \hat{\mathrm{z}}_k]^T$$

9. Calculate Kalman Gain & Update:

$$K_k = P_{xz} S_k^{-1}$$

$$\hat{\mathrm{x}}_{k|k} = \hat{\mathrm{x}}_{k|k-1} + K_k(\mathrm{z}_k - \hat{\mathrm{z}}_k)$$

$$P_{k|k} = P_{k|k-1} - K_k S_k K_k^T$$

The Engineering Dilemma: The Tuning Nightmare

Despite its mathematical brilliance, the UKF introduces a massive software engineering dilemma for embedded aerospace systems.

Look closely at the formula for the central covariance weight in Step 3: $W_0^{(c)} = \frac{\lambda}{n+\lambda} + (1 - \alpha^2 + \beta)$.

Because α is typically set to a very small number (e.g., 10^{-3}), the composite parameter λ becomes a large **negative** number. This means the central covariance weight $W_0^{(c)}$ frequently becomes negative.

In Chapter 2.2, we established that a covariance matrix must remain strictly Positive Definite (no negative eigenvalues). When you multiply the center point by a negative weight during the Step 4 Covariance reconstruction sum, you are mathematically *subtracting* variance.

If the tracking environment is highly volatile—such as a target pulling a sudden evasive maneuver combined with a dropped camera frame—the outer sigma points spread wide. The negative central weight pulls the core of the covariance matrix inward. Under these high-stress conditions, standard 32-bit floating-point rounding errors will cause the covariance matrix P to lose positive definiteness.

During the very next iteration, the Cholesky Decomposition ($\sqrt{P}$) will attempt to take the square root of a negative eigenvalue, resulting in a fatal `NaN` crash.

The UKF requires rigorous, heuristic parameter tuning (α, β, κ) specifically tailored to your state dimension (n) and your expected noise profile to avoid this. If the system scales to higher dimensions, tuning the UKF to remain stable becomes a mathematical nightmare.

C++ Code: The Unscented Transform

```
#include <Eigen/Dense>
#include <cmath>
#include <vector>

using namespace Eigen;
```

```
class UnscentedTransform {
public:
    // Generates UKF Sigma Points and Weights
    static void computeSigmaPoints(
        const VectorXd& x, const MatrixXd& P,
        double alpha, double beta, double kappa,
        MatrixXd& sigmaPoints, VectorXd& Wm, VectorXd& Wc)
    {
        int n = x.size();
        int num_points = 2 * n + 1;

        // Resize output matrices
        sigmaPoints.resize(n, num_points);
        Wm.resize(num_points);
        Wc.resize(num_points);

        // 1. Calculate Scaling Parameter (Lambda)
        double lambda = (alpha * alpha) * (n + kappa) - n;

        // 2. Calculate Weights
        Wm(0) = lambda / (n + lambda);
        // DANGER: Wc(0) can easily become negative depending on alpha!
        Wc(0) = (lambda / (n + lambda)) + (1.0 - (alpha * alpha) + beta);

        for (int i = 1; i < num_points; ++i) {
            Wm(i) = 1.0 / (2.0 * (n + lambda));
            Wc(i) = Wm(i);
        }

        // 3. Cholesky Decomposition (Square Root of Covariance)
        // Note: Production code MUST wrap this in a regularization check
        // to handle the non-positive-definite errors caused by negative Wc(0).
        LLT<MatrixXd> llt(P);
        MatrixXd L = llt.matrixL();
        MatrixXd scaledL = std::sqrt(n + lambda) * L;

        // 4. Generate Sigma Points
        sigmaPoints.col(0) = x; // Center point

        for (int i = 0; i < n; ++i) {
            sigmaPoints.col(i + 1)     = x + scaledL.col(i);
            sigmaPoints.col(i + 1 + n) = x - scaledL.col(i);
        }
    }
};
```

Chapter 11

The Cubature Kalman Filter (CKF)

In Chapter 10, we established that approximating a Gaussian probability distribution with deterministic points (the UKF) is vastly superior to approximating a non-linear function with calculus (the EKF).

However, we also uncovered the UKF's fatal engineering flaw: the arbitrary scaling parameters (α, β, κ). In higher-dimensional state spaces—such as a 9-state Coordinated Turn model running on an embedded Jetson architecture—the UKF's central covariance weight often plunges into negative values. If the target executes a violent maneuver, this negative weight mathematically subtracts variance, causing the covariance matrix to lose positive-definiteness and instantly crashing the Cholesky decomposition loop.

To build a truly resilient tracking architecture, we need a filter that possesses the non-linear accuracy of the UKF, but with mathematically guaranteed numerical stability.

In 2009, Simon Haykin and Ienkaran Arasaratnam published the **Cubature Kalman Filter (CKF)**. By abandoning arbitrary scaling parameters and returning to the strict roots of multi-dimensional integration, they created the definitive non-linear filter for modern state estimation.

11.1 The spherical-radial cubature rule.

The fundamental mathematical challenge of non-linear filtering is solving the multi-dimensional Gaussian-weighted integral:

$$I(f) = \int_{\mathbb{R}^n} f(\mathrm{x}) \mathcal{N}(\mathrm{x}; 0, I) d\mathrm{x}$$

Instead of relying on the Unscented Transform's arbitrary scaling parameters, the Cubature Kalman Filter (CKF) solves this integral rigorously using the Spherical-Radial Integration Rule.

As conceptually introduced in Chapter 1.4, converting Cartesian coordinates into spherical coordinates cleanly fractures this seemingly impossible multidimensional integral into two independent, solvable parts: a Spherical integral (which accounts for direction) and a Radial integral (which accounts for distance).

To execute its prediction and update loops, the CKF algorithm utilizes this exact rule. The spherical integral is solved using a fully symmetric spherical cubature rule, while the radial integral is solved using a 1st-degree Gauss-Laguerre quadrature. Because of the spherical symmetry, this 1st-degree radial approximation is mathematically proven to be exact for all 3rd-degree polynomials.

Deriving the Cubature Points

To refresh your memory on how this transformation physically generates the evaluation points, we look to the foundational paper by Arasaratnam and Haykin. They derive the Cubature points by splitting the integration problem:

1. **The Spherical Rule**: The authors set up symmetric moment-matching equations for the surface of an n-dimensional unit sphere (specifically evaluating $f(\mathrm{y}) = 1$ and $f(\mathrm{y}) = y_1^2$). The only mathematical solution that perfectly balances these equations yields a uniform weight of $A_n/2n$ (where A_n is the surface area) and forces the points to sit exactly on the coordinate axis intersections ($u^2 = 1$).
2. **The Radial Rule**: Next, they apply a 1st-degree Gauss-Laguerre quadrature to match the variance of the standard Gaussian curve. To perfectly capture the variance across n dimensions, the math forces the radial distance to equal exactly $\sqrt{n}$.

When you scale the directional unit-sphere points by this exact radial distance, you produce the final, deterministic **Cubature Points** used by the filter:

$$\xi_i = \sqrt{n}[1]_i \quad \text{for } i = 1, 2, \dots, 2n$$

(Where $[1]_i$ represents the i-th standard basis vector, capturing both the positive and negative axis intersections).

11.2 Mathematical derivation of cubature points.

By applying this 3rd-degree rule, the continuous Bayesian integral completely collapses into a finite, discrete summation over exactly $m = 2n$ points, where n is the dimension of the state vector.

These points (the **Cubature Points**, denoted as ξ_i) are mathematically proven to reside exactly on the intersections of the multi-dimensional axes and an uncertainty sphere of radius $\sqrt{n}$:

$$\xi_i = \sqrt{n}[1]_i \quad \text{for } i = 1, 2, \dots, 2n$$

(*Where $[1]_i$ is the i-th standard basis unit vector, covering both the positive and negative axes*).

Because the integral's mass is distributed evenly across the surface of the sphere, the weight (W_i) assigned to every single point is identical:

$$W_i = \frac{1}{2n} \quad \text{for } i = 1, 2, \dots, 2n$$

The final integrated expectation of the non-linear function is simply the equally weighted average of the propagated points:

$$I(f) \approx \frac{1}{2n} \sum_{i=1}^{2n} f(\xi_i)$$

(*For a refresher on the rigorous mathematical derivation from Gaussian-Laguerre quadrature, refer back to **Appendix A: Derivation of the Spherical-Radial Cubature Rule***).

11.3 Comparative analysis: CKF vs. UKF in high-dimensional state spaces.

The philosophical difference between the UKF and the CKF dictates their survival in edge computing.

- **The UKF** uses $2n + 1$ points. It places a heavily weighted point at the exact center of the distribution, and spreads the remaining points outward based on arbitrary tuning scalars.
- **The CKF** uses exactly $2n$ points. It leaves the center entirely empty, pushing all evaluation points to the "shell" of the uncertainty sphere.

By eliminating the central point, the CKF completely eradicates the need for the UKF's negative central scaling weight. In the CKF, every single weight is strictly positive ($\frac{1}{2n}$). Because the summation of positive semi-definite outer products multiplied by strictly positive weights can never yield a negative number, the CKF mathematically guarantees that the Covariance Matrix (P) remains positive-definite indefinitely.

> **Deep Dive: Mathematical Proof of Covariance Stability**
>
> We just stated that eliminating negative weights guarantees that the covariance matrix will never collapse. But how does linear algebra formally prove this? By applying the definition of positive-definiteness ($\mathrm{v}^T P \mathrm{v} > 0$) to the cubature summation, we can mathematically prove why the CKF survives where the UKF fails. For the rigorous step-by-step linear algebra proof, refer to **Appendix M: Proof of CKF Numerical Stability and Positive Definiteness**.

11.4 The Square-Root Cubature Kalman Filter (SCKF)

While the standard CKF proves that the *math* remains positive-definite, the physical hardware of a 32-bit CPU executing the standard update equation ($P_{k|k} = P_{k|k-1} - KSK^T$) will still accumulate microscopic rounding errors over millions of cycles.

To permanently eradicate numerical instability, Haykin and Arasaratnam introduced the **Square-Root Cubature Kalman Filter (SCKF)**.

Instead of propagating the full covariance matrix (P), the SCKF only propagates its lower-triangular square-root factor (S), where $P = SS^T$. By exclusively updating S using **QR Decomposition (Triangularization)** and **Least-Squares**, the SCKF:

1. Completely avoids the highly unstable Cholesky square-root operation during the filtering loop.
2. Mathematically forces the implicit P matrix to remain perfectly symmetric.
3. Effectively doubles the computational precision of the filter's arithmetic.

If A is a matrix composed of our weighted, centered cubature points, the QR decomposition factors it into an orthogonal matrix Q and an upper-triangular matrix R. We simply take the transpose of R to find our new lower-triangular square-root factor S:

$$S = \mathrm{Tria}(A) = R^T$$

Engineering Example: Angular Models & SCKF Algorithm

A common failure point in visual state estimation occurs when processing raw bearing angles from a camera. If a filter linearly averages angular cubature points (e.g., averaging 1° and 359°), it evaluates to 180°, causing massive track divergence.

To solve this, the SCKF must utilize **Directional Statistics**. We convert the angular points into 2D Cartesian vectors, average them, and use the atan2 function to recover the true mean angle.

C++ Implementation: The SCKF Update Loop

Here is a production-ready Eigen implementation of the SCKF Update Step, explicitly utilizing QR decomposition (`HouseholderQR`) to update the square-root covariance matrix (S) directly without ever computing P.

```
#include <Eigen/Dense>
#include <Eigen/QR>
#include <cmath>

using namespace Eigen;

class SquareRootCKF {
private:
    // Helper function to wrap angles to [-pi, pi]
    static double wrapAngle(double angle) {
        return std::fmod(angle + M_PI, 2.0 * M_PI) - M_PI;
    }

public:
    // SCKF Update step assuming an Angular Measurement
    // x: State estimate vector
    // S: Square-root factor of the covariance matrix (Lower Triangular)
    static void updateSCKF(
        VectorXd& x, MatrixXd& S,
        double z_meas, double noise_R,
        const MatrixXd& X_points) // Cubature points from Predict step
    {
        int n = x.size();
        int m = 2 * n; // Number of points
        double weight = 1.0 / m;

        VectorXd Z_points(m);
        double sum_sin = 0.0, sum_cos = 0.0;

        // 1. Propagate points and compute Mean using Directional Statistics
        for (int i = 0; i < m; ++i) {
            Z_points(i) = std::atan2(X_points(1, i), X_points(0, i)); // Example: Bearing
            sum_sin += weight * std::sin(Z_points(i));
            sum_cos += weight * std::cos(Z_points(i));
        }
        double z_pred = std::atan2(sum_sin, sum_cos);

        // 2. Create Centered Matrices
        MatrixXd Z_centered(1, m);
        MatrixXd X_centered(n, m);
```

```
        for (int i = 0; i < m; ++i) {
            Z_centered(0, i) = wrapAngle(Z_points(i) - z_pred) / std::sqrt(m);
            X_centered.col(i) = (X_points.col(i) - x) / std::sqrt(m);
        }

        // 3. Estimate Square-Root of Innovation Covariance (S_zz) via QR
        // Tria( [Z_centered, S_R] )
        double S_R = std::sqrt(noise_R); // Square root of measurement noise
        MatrixXd A_zz(1, m + 1);
        A_zz << Z_centered, S_R;

        // QR Decomposition -> Tria returns lower triangular (R transpose)
        HouseholderQR<MatrixXd> qr_zz(A_zz.transpose());
        MatrixXd S_zz = qr_zz.matrixQR().triangularView<Upper>().transpose();

        // Since measurement is 1D, S_zz is a 1x1 matrix (a scalar)
        double S_zz_scalar = S_zz(0,0);

        // 4. Estimate Cross-Covariance
        MatrixXd P_xz = X_centered * Z_centered.transpose();

        // 5. Calculate Kalman Gain using Least-Squares division
        // W = (P_xz / S_zz^T) / S_zz
        VectorXd W = (P_xz / S_zz_scalar) / S_zz_scalar;

        // 6. Update State
        double y_innov = wrapAngle(z_meas - z_pred);
        x = x + (W * y_innov);

        // 7. Update Square-Root Covariance (S) via QR Decomposition
        // Tria( [X_centered - W * Z_centered, W * S_R] )
        MatrixXd A_upd(n, m + 1);
        A_upd << (X_centered - W * Z_centered), (W * S_R);

        HouseholderQR<MatrixXd> qr_upd(A_upd.transpose());
        MatrixXd R_upd = qr_upd.matrixQR().topRows(n).triangularView<Upper>();
        S = R_upd.transpose(); // New Lower-Triangular Square-Root Covariance!
    }
};
```

Chapter 12

Advanced Frontiers

Before we conclude our exploration of non-linear filtering, we must address the absolute theoretical limit of state estimation.

The Extended, Unscented, and Cubature Kalman Filters all share one fundamental constraint: they are trapped within the Gaussian domain. They mathematically force the tracking uncertainty to remain a symmetric ellipsoid (a mean and a covariance).

But what if a target's probability distribution is completely arbitrary? What if a UAV flies behind a mountain, and the probability of it emerging on the left side is 30%, while the probability of it emerging on the right side is 70%? A Gaussian filter will average this bimodal distribution and confidently predict the target is inside the solid rock of the mountain.

To track non-Gaussian, multi-modal distributions, we must abandon matrices entirely and step into the frontier of Monte Carlo estimation.

12.1 The Particle Filter (PF)

The Particle Filter (PF) abandons the concept of a covariance matrix. Instead, it approximates an arbitrary probability density function (PDF) by flooding the state space with thousands of discrete, randomly generated samples called **Particles**.

Each particle i consists of a state vector $x^{(i)}$ (a hypothesis of where the target is) and a corresponding weight $w^{(i)}$ (the probability that this specific hypothesis is correct). The total probability distribution is simply the weighted sum of these thousands of points.

The PF Algorithm (Sequential Importance Resampling)

1. **Predict**: When the time step advances, every single particle is pushed through the non-linear kinematic equations ($x_k = f(x_{k-1})$). Crucially, random process noise is uniquely injected into every particle, causing the cloud of particles to spread out like a swarm of bees.
2. **Update (Weighting)**: When a camera measurement z_k arrives, the filter calculates the likelihood of that measurement for every single particle. Particles that land near the camera's bounding box are assigned massive weights; particles far away are assigned near-zero weights.
3. **Resampling**: Because particles far from the target become mathematically useless, the filter routinely

executes a "Darwinian" resampling step. It deletes the low-weight particles and clones the high-weight particles, focusing all computational power on the most likely region of space.

The Engineering Dilemma: The Curse of Dimensionality

If the Particle Filter can track any arbitrary distribution perfectly, why do we use the Cubature Kalman Filter for aerospace tracking?

The answer is CPU meltdown.

The number of particles required to successfully map a probability space grows exponentially with the dimension of the state vector. For a simple 2D tracking problem (X, Y), 500 particles might suffice. But for a 9-state Coordinated Turn model, you might need 50,000 to 100,000 particles to prevent the swarm from missing the target entirely.

Pushing 100,000 particles through complex camera projection geometry, calculating 100,000 likelihood functions, and executing a massive array-sorting algorithm for resampling at 60 Frames Per Second (16ms per frame) is physically impossible on Size, Weight, and Power (SWaP) constrained embedded architectures like an Nvidia Jetson Nano.

> **Deep Dive: The Mathematics of Particle Degeneracy**
>
> If you fail to run the Resampling step, the Particle Filter will suffer from "Weight Degeneracy," where all but one particle drops to a mathematical weight of exactly zero, crashing the estimator. For the mathematical proof of weight degeneracy and the formula for Effective Sample Size (N_{eff}), refer to **Appendix N: Particle Filter Degeneracy and Effective Sample Size**.

C++ Implementation: Systematic Resampling

For engineers who do deploy Particle Filters on high-compute ground stations, the bottleneck is often the Resampling step. Here is a highly optimized $O(N)$ Systematic Resampling algorithm that clones high-weight particles without requiring expensive array sorting.

```
#include <vector>
#include <random>

struct Particle {
    Eigen::VectorXd state;
    double weight;
};

void systematicResampling(std::vector<Particle>& particles) {
    int N = particles.size();
    std::vector<Particle> new_particles;
    new_particles.reserve(N);

    // Calculate cumulative sum of weights
    std::vector<double> cdf(N);
    cdf[0] = particles[0].weight;
    for (int i = 1; i < N; ++i) {
        cdf[i] = cdf[i - 1] + particles[i].weight;
    }
```

```
    // Generate a random starting point between 0 and 1/N
    std::random_device rd;
    std::mt19937 gen(rd());
    std::uniform_real_distribution<> dis(0.0, 1.0 / N);
    double u = dis(gen);

    int i = 0;
    // Step through the CDF like a roulette wheel with N equally spaced spokes
    for (int j = 0; j < N; ++j) {
        double u_j = u + (double)j / N;
        while (u_j > cdf[i] && i < N - 1) {
            i++;
        }
        // Clone the particle that the spoke landed on, resetting its weight
        Particle p = particles[i];
        p.weight = 1.0 / N;
        new_particles.push_back(p);
    }
    particles = new_particles;
}
```

12.2 The Ensemble Kalman Filter (EnKF)

If your state vector is exceptionally massive—such as a 10,000-state vector modeling ocean currents or a swarm of 500 UAVs—even the Cubature Kalman Filter will fail. Maintaining a $10,000 \times 10,000$ covariance matrix P requires gigabytes of RAM and is impossible to invert.

The Ensemble Kalman Filter (EnKF) is a hybrid approach. It uses a small swarm of particles (an "ensemble" of maybe 100 points) to represent the target. However, instead of using arbitrary particle weights, it assumes the swarm follows a Gaussian distribution. It calculates the empirical covariance of those 100 points and feeds that approximation into the standard Kalman Filter update equations.

While heavily utilized in meteorology and fluid dynamics, the EnKF is generally too imprecise for the strict, millimeter-accurate requirements of single-target optical weapons tracking, cementing the CKF as our tool of choice.

Chapter 13

The Engineer's Matrix

We have now reached the end of Part IV. We have journeyed from the perfectly linear Standard Kalman Filter (KF), through the calculus of the Extended Kalman Filter (EKF), into the deterministic points of the Unscented (UKF) and Cubature Kalman Filters (CKF), and finally looked at the Monte Carlo brute force of the Particle Filter (PF).

No single filter is the "best" for every application. Engineering is the art of compromise. The table below serves as your master decision matrix, comparing the computational costs, accuracy, and tuning nightmares of every filter.

The Non-Linear Filter Comparison Matrix

Filter	Core Mechanism	Non-Linear Accuracy	Point Evaluations per Step	Tuning Complexity	Stability Risk
KF	Linear Matrix Algebra	Fails (Requires Linearity)	1 (Just the Mean)	Low (Just Q and R)	Low
EKF	Calculus (Jacobians)	1st Order Taylor Series	1 (Just the Mean)	Medium	High (Truncation Error)
UKF	Unscented Transform	3rd Order Taylor Series	$2n + 1$ Sigma Points	Extreme (α, β, κ)	High (Negative Weights)
CKF	Spherical-Radial Cubature	3rd Order Taylor Series	$2n$ Cubature Points	Low (Just Q and R)	Low (Always Positive Definite)
PF	Monte Carlo Integration	Exact (Non-Gaussian)	$1,000$ to $100,000+$	High (Resampling Logic)	High (CPU/Memory Overload)

The Aerospace Decision Guide

When architecting a new tracking system, ask yourself the following questions in order:

1. **Is your physical system entirely linear?**(e.g., Tracking a slow-moving ground vehicle on a 2D map using Cartesian radar coordinates).

- **Decision**: Use the **Standard KF**. Do not waste CPU cycles generating cubature points if the matrix math is already exact.

2. **Are you tracking a highly agile target with a monocular camera on SWaP-constrained hardware?**(e.g., An interceptor drone tracking a maneuvering fixed-wing aircraft using nested Euler angles).

- **Decision**: Use the **Square-Root Cubature Kalman Filter (SCKF)**. The non-linearities will destroy the EKF, the UKF will require months of frustrating manual parameter tuning, and the PF will melt your embedded CPU. The SCKF offers mathematical stability with zero tuning parameters.

3. **Are you tracking a target where the probability is heavily multi-modal (non-Gaussian)?** (e.g., Terrain-referenced navigation, or tracking a target moving through a dense urban city block where it can turn down 4 distinct streets).

- **Decision**: Use the Particle Filter (PF). A Gaussian filter will average the four streets together and confidently predict the target is inside a building. You must use a PF and accept the heavy computational cost.

With the Cubature Kalman Filter firmly established as our primary non-linear engine, we are now ready to tackle the final challenge of target tracking. We have a mathematically stable filter, but what happens when the target fundamentally changes its behavior mid-flight? In Part V, we will wrap our CKF inside the **Interacting Multiple Model (IMM)** architecture to track targets that fight back.

Part V: Maneuvering Targets and Multiple Models

Chapter 14

Interacting Multiple Model (IMM) Estimation

In the preceding chapters of this text, our primary adversary was non-linear geometry. Whether deriving Jacobians for the Extended Kalman Filter or distributing cubature points across a sphere for the Cubature Kalman Filter, we operated under a single, fragile assumption: we know the physical kinematic equations governing the target.

In optical and radar weapons tracking, this assumption collapses the moment a target detects an incoming threat. A commercial drone loitering in a stable hover (Constant Velocity) will instantly pull a violent, high-G banking evasive maneuver (Coordinated Turn or Constant Acceleration).

If an engineer deploys a single dynamic filter, they face an impossible tuning dilemma. If the Process Noise (Q) is tuned low, the filter outputs a beautifully smooth track during hovering, but completely loses the target when it maneuvers. If Q is artificially inflated to catch the turn, the filter tracks the maneuver but jitters violently during stable flight.

To track targets that fundamentally change their behavior mid-flight, we must move beyond single-model estimation. We must wrap our non-linear filters inside a hybrid dynamic architecture known as the Interacting Multiple Model (IMM) estimator.

14.1 The Maneuvering Target Dilemma & Markov Chains

When tracking an agile adversary, we face two distinct types of uncertainty:

1. **Continuous Kinematic Uncertainty**: The standard Gaussian noise corrupting position, velocity, and sensor hits (handled by Kalman filters).
2. **Discrete Structural Uncertainty**: The abrupt, non-continuous switches between completely different flight modes.

To capture structural uncertainty, we define a discrete set of r kinematic models (e.g., Model 1 = Constant Velocity, Model 2 = Coordinated Turn). We mathematically assume that the target switches between these models according to a **Discrete-Time Markov Chain**.

The probability of the target transitioning from Model i at time $k-1$ to Model j at time k is governed by a known, **static Transition Probability Matrix**, denoted as Π:

$$\Pi = \begin{bmatrix} \pi_{11} & \pi_{12} & \dots & \pi_{1r} \\ \pi_{21} & \pi_{22} & \dots & \pi_{2r} \\ \vdots & \vdots & \ddots & \vdots \\ \pi_{r1} & \pi_{r2} & \dots & \pi_{rr} \end{bmatrix}$$

Where the transition probabilities satisfy $\sum_{j=1}^{r} \pi_{ij} = 1$. If a drone is loitering smoothly, π_{11} might be 0.95, reflecting a 95% chance it continues loitering, and a 5% chance ($\pi_{12} = 0.05$) it snaps into an evasive banking turn.

14.2 The Four-Step IMM Architecture

The brilliant insight of the IMM estimator (originally established by Blom and Bar-Shalom) is that it does not force the computer to hard-switch between models. Instead, it runs all r filters in parallel and executes a highly intricate, 4-step probabilistic mixing cycle during every single camera frame.

Let $\mu_i(k-1)$ represent the established probability that Model i was active during the previous frame.

- **Step 1: The Interaction (Mixing) Step**

Before executing the standard filter predictions, we calculate the normalization constants (c_j) and the mixing probabilities ($\mu_{i|j}$), which represent the probability that the target switched from Model i to Model j:

$$c_j = \sum_{i=1}^{r} \pi_{ij} \mu_i(k-1)$$

$$\mu_{i|j}(k-1|k-1) = \frac{\pi_{ij} \mu_i(k-1)}{c_j}$$

We now calculate the mixed initial state vector ($\bar{\mathrm{x}}_j$) and mixed initial covariance matrix ($\bar{P}_j$) specifically tailored for every filter j:

$$\bar{\mathrm{x}}_j(k-1|k-1) = \sum_{i=1}^{r} \mu_{i|j}(k-1|k-1) \hat{\mathrm{x}}_i(k-1|k-1)$$

$$\bar{P}_j(k-1|k-1) = \sum_{i=1}^{r} \mu_{i|j}(k-1|k-1) \left[P_i(k-1|k-1) + \mathrm{d}_{ij} \mathrm{d}_{ij}^T \right]$$

(Where the spread-of-the-means vector is defined as $\mathrm{d}_{ij} = \hat{\mathrm{x}}_i(k-1|k-1) - \bar{\mathrm{x}}_j(k-1|k-1)$).

- **Step 2: Mode-Matched Filtering**

We now take our mixed inputs ($\bar{\mathrm{x}}_j, \bar{P}_j$) and feed them individually into our r parallel tracking filters (which can be linear KFs, EKFs, or Cubature Kalman Filters).

Each filter executes its own standard prediction and measurement update step, yielding a newly updated state $\hat{\mathrm{x}}_j(k|k)$, covariance $P_j(k|k)$, and an innovation covariance $S_j(k)$. Crucially, each filter also evaluates the raw Gaussian Likelihood (Λ_j) of the incoming camera measurement z_k:

$$\Lambda_j(k) = \frac{1}{\sqrt{|2\pi S_j(k)|}} \exp\left(-\frac{1}{2}\mathrm{y}_j^T(k) S_j^{-1}(k) \mathrm{y}_j(k)\right)$$

(Where $\mathrm{y}_j(k)$ is the residual innovation vector of filter j).

- **Step 3: Mode Probability Update**

We use these raw measurement likelihoods to update the overall operational probability (μ_j) of each flight mode using Bayes' rule:

$$\mu_j(k) = \frac{\Lambda_j(k) c_j}{c}$$

Where the total system scale factor is $c = \sum_{j=1}^{r} \Lambda_j(k) c_j$. If the target suddenly banks, the Coordinated Turn filter's predicted measurement will land squarely on the camera hit, generating a massive likelihood Λ_{CT}, instantly spiking the mode probability $\mu_{CT} \to 1.0$.

- **Step 4: State Output Combination**

For final telemetry output and visual rendering, we collapse the r separate Gaussian branches into a single global state estimate ($\hat{\mathrm{x}}$) and global covariance (P) using moment matching:

$$\hat{\mathrm{x}}(k|k) = \sum_{j=1}^{r} \mu_j(k) \hat{\mathrm{x}}_j(k|k)$$

$$P(k|k) = \sum_{j=1}^{r} \mu_j(k) \left[P_j(k|k) + \left(\hat{\mathrm{x}}_j(k|k) - \hat{\mathrm{x}}(k|k)\right)(\dots)^T\right]$$

Deep Dive: The Linear Algebra of Gaussian Collapse

Why must we execute Step 1 and Step 4 during every iteration? If we simply ran r independent filters across k time steps without mixing, a target switching between r models would generate a massive tree of r^k possible hypothesis branches. At 60Hz over a 10-second engagement, the computer would have to track 2^{600} simultaneous matrices—crashing any computer on Earth. The IMM achieves optimal $O(r)$ performance using a Generalized Pseudo-Bayesian (GPB2) moment-matching collapse. For the rigorous mathematical proof of this variance-preserving collapse, refer to **Appendix O: Moment Matching Reduction of Gaussian Mixtures**.

Engineering Example: Loitering vs. Evasive UAV

Consider an optical tracking payload mounted on an interceptor vehicle. We must track an enemy UAV that alternates between stable hovering and rapid evasive turns. We define a 2-model IMM:

1. **Model 1 (CV)**: Constant Velocity in 2D Cartesian space $[x, y, v_x, v_y]^T$. Tuned with tiny process noise ($q_{CV} = 0.01\mathrm{m}^2/\mathrm{s}^3$).
2. **Model 2 (CT)**: Coordinated Turn with unknown turn rate $[x, y, v_x, v_y, \Omega]^T$. Tuned with high process noise ($q_{CT} = 2.5\mathrm{m}^2/\mathrm{s}^3$) to absorb violent angular accelerations.

We program the Markov transition matrix to heavily favor remaining in the current operational state:

$$\Pi = \begin{bmatrix} 0.95 & 0.05 \\ 0.10 & 0.90 \end{bmatrix}$$

(Note: $\pi_{21} = 0.10$ indicates that once the target enters a turn, there is a 10% chance per frame it snaps back out into straight-line flight).

C++ Code: The IMM Mixing Engine

Here is a highly modular, production-ready Eigen implementation of the core IMM mixing, probability updating, and collapsing engine. This engine acts as the master controller that can wrap around any linear or non-linear filter class.

```
#include <Eigen/Dense>
#include <vector>
#include <cmath>

using namespace Eigen;

struct SubFilterState {
    VectorXd x;         // State vector
    MatrixXd P;         // Covariance matrix
    double likelihood;  // Measurement likelihood from filter update
};

class InteractingMultipleModel {
private:
    int num_models;
    MatrixXd Pi;        // Markov Transition Probability Matrix (r x r)
    VectorXd mu;        // Current Mode Probabilities (r x 1)
    VectorXd c;         // Normalization constants (r x 1)
    MatrixXd mu_mix;    // Mixing Probabilities matrix (r x r)

public:
    InteractingMultipleModel(const MatrixXd& transition_matrix, const VectorXd& initial_probs) {
        Pi = transition_matrix;
        mu = initial_probs;
        num_models = mu.size();
        c.resize(num_models);
        mu_mix.resize(num_models, num_models);
    }

    // STEP 1: Execute IMM Mixing before passing data to sub-filters
    void mixStates(const std::vector<SubFilterState>& prev_states,
                   std::vector<SubFilterState>& mixed_states)
    {
        int state_dim = prev_states[0].x.size();

        // 1. Calculate normalization constants c_j
        c = Pi.transpose() * mu;
```

```
    // 2. Calculate mixing probabilities mu_{i|j}
    for (int i = 0; i < num_models; ++i) {
        for (int j = 0; j < num_models; ++j) {
            mu_mix(i, j) = (Pi(i, j) * mu(i)) / c(j);
        }
    }

    // 3. Calculate Mixed Initial States and Covariances
    for (int j = 0; j < num_models; ++j) {
        VectorXd x_bar = VectorXd::Zero(state_dim);
        for (int i = 0; i < num_models; ++i) {
            x_bar += mu_mix(i, j) * prev_states[i].x;
        }

        MatrixXd P_bar = MatrixXd::Zero(state_dim, state_dim);
        for (int i = 0; i < num_models; ++i) {
            VectorXd diff = prev_states[i].x - x_bar;
            P_bar += mu_mix(i, j) * (prev_states[i].P + diff * diff.transpose());
        }

        mixed_states[j].x = x_bar;
        mixed_states[j].P = P_bar;
    }
}

// STEPS 3 & 4: Update Mode Probabilities and Collapse Output
void combineOutput(const std::vector<SubFilterState>& updated_states,
                   VectorXd& global_x, MatrixXd& global_P)
{
    int state_dim = updated_states[0].x.size();
    double total_c = 0.0;

    // Step 3: Update Mode Probabilities using Likelihoods
    for (int j = 0; j < num_models; ++j) {
        mu(j) = updated_states[j].likelihood * c(j);
        total_c += mu(j);
    }
    mu = mu / total_c; // Normalize probabilities to sum to 1.0

    // Step 4: State Output Combination (Gaussian Collapse)
    global_x = VectorXd::Zero(state_dim);
    for (int j = 0; j < num_models; ++j) {
        global_x += mu(j) * updated_states[j].x;
    }

    global_P = MatrixXd::Zero(state_dim, state_dim);
    for (int j = 0; j < num_models; ++j) {
```

```
            VectorXd diff = updated_states[j].x - global_x;
            global_P += mu(j) * (updated_states[j].P + diff * diff.transpose());
        }
    }

    VectorXd getModeProbabilities() const { return mu; }
};
```

Chapter 15

The Synthesis: IMM-CKF

In Chapter 11, we established the Cubature Kalman Filter (CKF) as an optimal numerical solution for handling continuous non-linear geometry. In Chapter 14, we established the Interacting Multiple Model (IMM) architecture to handle discrete structural uncertainty—specifically, a target suddenly changing its flight profile.

Now, we fuse them together.

The **IMM-CKF architecture** represents the synthesis of these two powerful paradigms. It seamlessly manages both the discrete uncertainty of which maneuver the target is performing, and the continuous uncertainty of the target's exact spatial coordinates within that maneuver. This chapter details how to embed the CKF inside the IMM framework, how it handles severe non-linear transitions, and the mathematical proof of its convergence.

15.1 Embedding the Cubature Kalman Filter within the IMM framework.

The standard IMM framework is filter-agnostic; it simply requires a bank of sub-filters that can produce a state estimate, a covariance matrix, and a measurement likelihood (Λ).

When tracking highly non-linear targets (such as utilizing bearings-only camera measurements or complex aerodynamic flight models), linear Kalman Filters and Extended Kalman Filters (EKFs) suffer from truncation errors that artificially deflate their likelihood calculations. If a sub-filter calculates an inaccurate likelihood due to linearization errors, the IMM mode-mixer will assign the wrong probabilities, and the entire architecture will collapse.

Embedding the CKF ensures that the likelihoods fed into the IMM Markov mixer are rigorously accurate up to the third-order Taylor series expansion.

The Embedded CKF Cycle: For every mode j in the IMM (where $j = 1, \dots, r$), the mixed initial state $\bar{\mathrm{x}}_j(k-1|k-1)$ and mixed covariance $\bar{P}_j(k-1|k-1)$ are passed to a dedicated CKF.

1. **Cubature Point Generation**: The CKF generates $2n$ deterministic cubature points $\mathcal{X}_{i,j}$ around the mixed state $\bar{\mathrm{x}}_j$.
2. **Non-Linear State Propagation**: The points are passed through the specific non-linear kinematic model $f_j(\mathrm{x})$ assigned to mode j (e.g., a non-linear Coordinated Turn model).

3. **Non-Linear Measurement Propagation**: The predicted points are then passed through the non-linear measurement function $h(\mathrm{x})$ (e.g., an arctangent bearing calculation).
4. **Likelihood Extraction**: After computing the innovation $\nu_j = \mathrm{z}_k - \hat{\mathrm{z}}_j$ and the innovation covariance S_j, the CKF computes the Gaussian likelihood:

$$\Lambda_j = \frac{1}{\sqrt{|2\pi S_j|}} \exp\left(-\frac{1}{2}\nu_j^T S_j^{-1} \nu_j\right)$$

By relying on the spherical-radial cubature rule to accurately model the innovation covariance S_j, the IMM-CKF ensures that the mode probabilities (μ_j) update correctly even during the most severe geometric non-linearities.

15.2 Handling highly non-linear maneuvers (e.g., transitioning from a linear trajectory to a sharp, high-G coordinated turn).

To understand the power of the IMM-CKF, consider an engineering example: A UAV transitioning from a linear trajectory to a sharp, high-G coordinated turn.

We define a 2-model IMM-CKF:

- **Model 1 (CV)**: A linear Constant Velocity model $[X, Y, V_x, V_y]^T$.
- **Model 2 (CT)**: A highly non-linear Coordinated Turn model $[X, Y, V_x, V_y, \omega]^T$, where ω is the turn rate. The velocities are coupled via sine and cosine functions: $V_{x,k+1} = V_{x,k}\cos(\omega\Delta t) - V_{y,k}\sin(\omega\Delta t)$.

The Transition Event: While the UAV flies straight, the CV model perfectly predicts the target's position. Its innovation (ν_{CV}) is near zero, and its likelihood (Λ_{CV}) is exceptionally high. The IMM assigns ~95% probability to the CV mode. The CT model, meanwhile, calculates a near-zero turn rate $(\omega \approx 0)$ and maintains a low background probability.

Suddenly, the UAV banks hard and pulls a 5G turn.

- **The CV Filter Failure**: The CV model blindly projects the target moving in a straight line. The incoming sensor measurement reveals the target has aggressively curved away. The CV innovation (ν_{CV}) spikes massively. The exponential term in the Gaussian likelihood equation drives Λ_{CV} asymptotically toward zero.
- **The CT Filter Success**: The CT model's cubature points naturally capture the non-linear sine/cosine velocity coupling. The CT filter perfectly predicts the curved trajectory. Its innovation (ν_{CT}) remains small, yielding a high likelihood (Λ_{CT}).

Within a single tracking frame, the IMM's Markov mixer multiplies these new likelihoods against the transition matrix. The probability weight instantly shifts from 95% CV to 95% CT. Because the CKF accurately preserved the non-linear covariance of the turn without requiring analytical Jacobians, the transition is mathematically smooth, and the combined state output effortlessly tracks the high-G maneuver.

Deep Dive: SWaP-Constrained Agility (The Acceleration Surrogate)

While the Coordinated Turn (CT) or Constant Acceleration (CA) models are mathematically rigorous, real-world embedded interceptors often lack the computational overhead (Size,

Weight, and Power) to execute them. As a lightweight alternative, engineers use the Acceleration Surrogate: running a secondary Constant Velocity model with a massively inflated Process Noise (Q) matrix to absorb the high-G maneuver. For the rigorous mathematical proof demonstrating how this drives the Kalman Gain to bypass the kinematic memory, refer to **Appendix P: Proof of Acceleration Surrogate via Process Noise Inflation**.

15.3 Dynamic Sensor Transitions and Noise Adapation

Beyond physical kinematic maneuvers, a tactical tracker must seamlessly handle sensors that alter their fundamental noise profiles mid-flight. The IMM-CKF architecture allows the measurement covariance matrix (R_k) to be dynamically scaled without destabilizing the Markov mixer.

If a tracking payload toggles from a Visible camera to an Infrared (IR) sensor mid-flight, the target's physical velocity does not change, but the measurement uncertainty space degrades drastically due to the lower resolution and thermal properties of the new sensor. By allowing the CKF to dynamically reinflate its R_k matrix at the moment of the hardware toggle, the IMM gracefully de-weights the raw visual measurements, relying more heavily on its kinematic memory to bridge the transition.

Deep Dive: Multi-Spectral Transitions and Thermal Blooming

Uncooled Vanadium Oxide (VOx) microbolometers in IR payloads measure heat absorption. When tracking a fast-moving drone, the thermal inertia of the pixels causes the heat signature to smear—a phenomenon known as Thermal Blooming. The IMM-CKF must dynamically inflate R_k to survive this transition. For the thermodynamic derivation of this scaling factor, refer to **Appendix Q: Derivation of the Thermal Blooming Coefficient** (β_{th}).

15.4 Complete algorithm walkthrough and mathematical proof of convergence.

The complete IMM-CKF algorithm follows a rigorous, recursive four-step cycle:

1. **Interaction (Mixing)**: Calculate mixing probabilities $\mu_{i|j}$ using the Markov transition matrix Π. Mix the r previous states and covariances to generate r new starting conditions.
2. **CKF Filtering**: Run r parallel Cubature Kalman Filters. Generate cubature points, propagate through non-linear $f_j(\cdot)$ and $h_j(\cdot)$, and calculate the mode-conditioned likelihoods Λ_j.
3. **Mode Probability Update**: Update the overall mode probabilities using Bayes' theorem: $\mu_j(k) = \frac{1}{c}\Lambda_j \sum_{i=1}^{r} \pi_{ij}\mu_i(k-1)$.
4. **Combination**: Collapse the r Gaussian distributions into a single global state $\hat{\mathrm{x}}$ and covariance P using moment matching for final output.

Mathematical Proof of Convergence

A critical question in IMM estimation is: Does the filter mathematically guarantee convergence to the true target mode? Let the true, unknown operating mode of the target at time k be $M_{true} = q$. We wish to prove that as measurements Z_k accumulate, the estimated probability of mode q, $\mu_q(k)$, converges to 1, while all other mode probabilities $\mu_j(k)$ (for $j \neq q$) converge to 0.

By Bayes' rule, the probability of mode j given the measurement history Z_k is proportional to its likelihood:

$$\mu_j(k) = P(M_j|Z_k) \propto p(z_k|Z_{k-1}, M_j)P(M_j|Z_{k-1})$$

Let us look at the likelihood ratio between the true mode q and any incorrect mode j:

$$L_{q,j}(k) = \frac{\mu_q(k)}{\mu_j(k)} = \frac{\Lambda_q(k)P(M_q|Z_{k-1})}{\Lambda_j(k)P(M_j|Z_{k-1})}$$

Because the CKF evaluates the true non-linear expectation, the innovation for the true mode q will be a zero-mean white noise sequence with covariance S_q. The likelihood Λ_q evaluates to a stable maximum.

For the incorrect mode j, the kinematic mismatch induces a deterministic, non-zero bias b_j in the innovation: $\nu_j \sim \mathcal{N}(\mathrm{b}_j, S_j)$.

The ratio of the likelihoods becomes:

$$\frac{\Lambda_q}{\Lambda_j} = \sqrt{\frac{|S_j|}{|S_q|}} \frac{\exp\left(-\frac{1}{2}\nu_q^T S_q^{-1}\nu_q\right)}{\exp\left(-\frac{1}{2}(\nu_q + \mathrm{b}_j)^T S_j^{-1}(\nu_q + \mathrm{b}_j)\right)}$$

As the target continues to maneuver in mode q, the deterministic bias b_j in the mismatched filter grows exponentially with time. Because the bias term is squared inside a negative exponential, the denominator shrinks rapidly toward zero:

$$\lim_{\mathrm{b}_j \to \infty} \exp\left(-\frac{1}{2}(\dots)\mathrm{b}_j\right) = 0$$

Therefore, the likelihood ratio $L_{q,j}(k) \to \infty$. This mathematically forces the normalized probability of the true mode, μ_q, to converge to exactly 1.0. The IMM-CKF is theoretically proven to successfully identify the correct non-linear maneuver, provided the true mode is accurately represented within the filter bank.

C++ Implementation: The IMM-CKF Filter Bank

The following C++ implementation demonstrates the generic synthesis of the IMM mixing logic with an embedded Cubature Kalman Filter. This structure utilizes Eigen to pass the mixed states into a generic CubatureKalmanFilter class (implementation of the CKF math is assumed from Chapter 11).

```
#include <Eigen/Dense>
#include <vector>
#include <cmath>
#include <iostream>

using namespace Eigen;

// Forward declaration of a generic CKF class (from Chapter 11)
class CubatureKalmanFilter {
public:
    VectorXd x;
    MatrixXd P;
```

```
    // Function pointers or enums defining the non-linear physics for this specific mode
    int mode_type;

    CubatureKalmanFilter(int mode) : mode_type(mode) {}

    // Executes the CKF Prediction and Update, returns the Gaussian Likelihood
    double predictAndUpdate(const VectorXd& z_meas, double dt) {
        // 1. Generate Cubature Points
        // 2. Propagate through f(x) and h(x)
        // 3. Compute Innovation (nu) and Innovation Covariance (S)
        // 4. Update x and P

        // Mocked variables for demonstration
        int m = z_meas.size();
        VectorXd nu = VectorXd::Zero(m); // Innovation
        MatrixXd S = MatrixXd::Identity(m, m); // Innovation Covariance

        // Gaussian Likelihood calculation
        double det_S = S.determinant();
        double exponent = -0.5 * (nu.transpose() * S.inverse() * nu).value();
        double likelihood = std::exp(exponent) / std::sqrt(std::pow(2 * M_PI, m) * det_S);

        return std::max(likelihood, 1e-15); // Prevent underflow
    }
};

class IMM_CKF_System {
private:
    int num_models;
    MatrixXd Pi;        // Markov Transition Matrix
    VectorXd mu;        // Mode Probabilities
    std::vector<CubatureKalmanFilter> filters;

public:
    IMM_CKF_System(const MatrixXd& transition_matrix, const VectorXd& initial_probs) {
        Pi = transition_matrix;
        mu = initial_probs;
        num_models = mu.size();

        // Initialize parallel CKF models (e.g., 0 = CV, 1 = CT)
        for (int i = 0; i < num_models; ++i) {
            filters.push_back(CubatureKalmanFilter(i));
            // Initialize state dimensions here in production...
        }
    }

    void step(const VectorXd& z_meas, double dt, VectorXd& global_x, MatrixXd& global_P) {
        int state_dim = 5; // Example dimension for [X, Y, Vx, Vy, omega]
```

```
VectorXd c = Pi.transpose() * mu;
MatrixXd mu_mix = MatrixXd::Zero(num_models, num_models);

// 1. Mixing Probabilities
for (int i = 0; i < num_models; ++i) {
    for (int j = 0; j < num_models; ++j) {
        mu_mix(i, j) = (Pi(i, j) * mu(i)) / c(j);
    }
}

// 2. Mixed Initial States for the CKFs
std::vector<VectorXd> x_mixed(num_models, VectorXd::Zero(state_dim));
std::vector<MatrixXd> P_mixed(num_models, MatrixXd::Zero(state_dim, state_dim));

for (int j = 0; j < num_models; ++j) {
    for (int i = 0; i < num_models; ++i) {
        x_mixed[j] += mu_mix(i, j) * filters[i].x;
    }
    for (int i = 0; i < num_models; ++i) {
        VectorXd diff = filters[i].x - x_mixed[j];
        P_mixed[j] += mu_mix(i, j) * (filters[i].P + diff * diff.transpose());
    }
    // Assign mixed states to the sub-filters BEFORE they run
    filters[j].x = x_mixed[j];
    filters[j].P = P_mixed[j];
}

// 3. Mode-Matched CKF Filtering & Likelihood Extraction
VectorXd likelihoods(num_models);
double total_c = 0.0;

for (int j = 0; j < num_models; ++j) {
    // The CKF executes the Spherical-Radial rules internally
    likelihoods(j) = filters[j].predictAndUpdate(z_meas, dt);

    // 4. Mode Probability Update (Bayes' Rule)
    mu(j) = likelihoods(j) * c(j);
    total_c += mu(j);
}
mu = mu / total_c; // Normalize

// 5. Global State Output Combination
global_x = VectorXd::Zero(state_dim);
global_P = MatrixXd::Zero(state_dim, state_dim);

for (int j = 0; j < num_models; ++j) {
    global_x += mu(j) * filters[j].x;
```

```
        }
        for (int j = 0; j < num_models; ++j) {
            VectorXd diff = filters[j].x - global_x;
            global_P += mu(j) * (filters[j].P + diff * diff.transpose());
        }
    }
};
```

Part VI: Virtual-Real-World Vision-and-Telemetry-Based Defense Interceptor Products Architecture and Implementation

Chapter 16

System Architecture and Multispectral Data Fusion

Up until this point, we have assumed that our filter magically receives perfect, synchronized measurements. In a real-world defense interceptor, the reality is chaotic. Video frames arrive at 60Hz, gimbal encoders blast serial data at 100Hz, the vehicle autopilot updates at a sluggish 10Hz, and the target is actively attempting to evade amidst a swarm of similar objects.

This chapter details the production software architecture required to host the IMM-CKF tracking engine on an **NVIDIA Jetson devices**. We will trace the data pipeline from the bare-metal FPGA video fusion layer, through the Deep Learning inference engines (YOLOv8 and OSNet), and into the kinematic tracking mathematics, explicitly addressing the harsh realities of multi-platform tactical deployments.

16.1 The Asynchronous Hardware Hub and Multi-Threaded Design

The Jetson Orin NX serves as the central nervous system of the interceptor. Because hardware data arrives at vastly different rates (video at 60Hz, gimbal at 100Hz, autopilot at 10Hz), a single-threaded loop would be catastrophic. If the CPU waits 16ms for a YOLOv8 inference to complete, it will drop multiple 100Hz serial packets from the gimbal, permanently corrupting the target's geometric projection.

To survive this, the software architecture is rigidly divided into **four concurrent CPU threads**, communicating exclusively through thread-safe memory ringbuffers.

Thread 1: The Zero-Copy Video & AI Pipeline

- **Hardware Interfacing**: An FPGA acts as the video fuse, grabbing raw 1080p Visible and 640p Infrared (IR) streams. Captures YUV422 video via CSI/PCIe using native V4L2 with mmap pinned memory, completely bypassing bloated middleware like OpenCV or GStreamer. Uses Jetson hardware (NvBufSurface and VIC) for zero-copy colorspace conversion to both FP16 RGB/BGR and NV12.
- **AI Inference**: Executes the TensorRT YOLOv8 detection and OSNet Re-ID inference.
- **Storage**: Packages the zero-copy frame pointer, YOLO bounding boxes, OSNet embeddings, and NV12 frame with frame index and timestamped pushing them into the Historical Frame Ringbuffer.

Thread 2: The Gimbal Telemetry Pipeline

- **Hardware Interfacing**: Listens to the RS232 serial port at 100Hz.
- **Storage**: Parses the Serial packets (IMU, Pan, Tilt, and Camera Zoom states) and pushes them into the **Gimbal Telemetry Ringbuffer**, timestamped at the exact microsecond of arrival.

Thread 3: The Autopilot Telemetry Pipeline

- **Hardware Interfacing**: Listens to the RS422 serial port at 10Hz
- **Storage**: Parses incoming Mavlink packets from the Pixhawk (Vehicle Roll, Pitch, Yaw, GPS position, and velocity), pushing them into the Autopilot Telemetry Ringbuffer.

Thread 4: The IMM-CKF Master Tracker Pipeline

Fusion: This is the master estimation thread. It wakes up when a new frame is deposited into the Historical buffer. It queries the Gimbal and Autopilot buffers to extract the exact physical attitudes that match the frame's timestamp. It then executes the data association logic, runs the IMM-CKF matrices, and outputs the final, stabilized target box to the system.

The Latency Dilemma and Ground Station Locks: This highly decoupled ringbuffer architecture solves the datalink latency problem. When a human operator at a remote ground station manually selects a target to lock, datalink latency means the video frame index they clicked is already obsolete by hundreds of milliseconds. The master Tracker Thread handles this by reading the incoming frame_index from the datalink, winding back time to query the Historical Frame Ringbuffer for the target's exact visual embedding from the past, initializing the IMM-CKF in the past, and mathematically "fast-forwarding" to catch up to live time.

16.2 The AI Perception Engine: A 4-Stage Association Pipeline

Before the IMM-CKF can track a target, the Tracker Thread must extract the target from the pixels and disambiguate it from similar objects. This forms a rigorous four-stage perception and data association pipeline.

- **Stage 1: Detection (YOLO with NVM, P2, and TensorRT)**

The FP16 NvBufSurface video frame is fed directly into a highly optimized YOLO architecture accelerated by NVIDIA TensorRT. Because tactical engagements often begin at extreme slant ranges, standard YOLO networks fail to detect targets smaller than 10x10 pixels. We utilize the P2 Feature Map (a higher-resolution, shallower neural layer) combined with NVM (Non-Volatile Memory) TensorRT plugins to guarantee sub-millisecond inference on tiny airborne and ground targets.

> Output: A batch of bounding boxes, Class IDs (Person, Car, Drone, Military Vehicle), and confidence scores.

- **Stage 2: Re-Identification (OSNet with TensorRT) for Similar Object Disambiguation**

In tactical environments, the most severe tracking threat is the presence of multiple objects that look identical to the target from the ground station's perspective—such as an enemy drone flying into a larger swarm, or a targeted truck merging into a convoy of identical vehicles. When this happens, YOLO will detect multiple valid bounding boxes. The IMM-CKF kinematic prediction alone might struggle to distinguish between the real target and a similar object pulling a parallel maneuver.

We feed the cropped pixel patches of YOLO's detections into OSNet (Omni-Scale Network), which is also compiled into an optimized TensorRT engine for maximum GPU throughput. OSNet extracts a rich, 512-

dimensional numerical embedding vector (a mathematical "fingerprint" of the target's visual texture). These embeddings are saved directly into the Historical Frame Ringbuffer for future association

The Military Retraining Imperative

Off-the-shelf YOLOv8 and OSNet weights (trained on public datasets like COCO or ImageNet) are entirely insufficient for defense applications. Both networks must be rigorously retrained with specialized military datasets. > * YOLOv8 must be retrained on proprietary infrared and visible datasets to detect specific missiles, UAVs, and military vehicles against diverse operational backgrounds (sky clutter, maritime environments, urban terrain).

- **OSNet** must be retrained using a military-specific triplet-loss configuration, teaching the network to mathematically differentiate the microscopic visual and thermal signatures of otherwise identical chassis (e.g., distinguishing the specific heat distribution, battle damage, or payload configuration of the primary target drone from an identical swarm drone flying right next to it).

  Deep Dive: Data Association and the Kinematic Fallback Before passing a YOLO measurement to the IMM-CKF, the software calculates the Cosine Distance between the OSNet embedding of the new detection and the historically saved embedding of the tracked target. If the target crosses paths with another vehicle of the exact same make and model, the OSNet vector leverages subtle, target-specific texture fingerprints learned during retraining to reject the incorrect object.

  But what if the visual fingerprints are mathematically identical? If two perfectly identical, pristine mass-produced drones cross paths, OSNet's cosine similarity will fail to distinguish them. At this critical juncture, the AI perception layer has failed, and the architecture must rely entirely on the IMM-CKF. The filter uses its predicted innovation covariance matrix (S_k) to calculate the Mahalanobis distance for each bounding box. The IMM-CKF isolates the correct target based purely on its spatial velocity vector, physical momentum, and trajectory history, proving why neural networks must always be fused with rigorous Newtonian kinematics

- **Stage 3: Extract Kinematic Mahalanobis Distance (IMM-CKF)**

While OSNet handles visual textures, the system simultaneously queries the IMM-CKF for its physical predictions. The filter uses its predicted innovation covariance matrix (S_k) to calculate the Mahalanobis distance (D_m) for each candidate bounding box. This distance evaluates how well each detection aligns with the target's predicted spatial velocity vector, physical momentum, and trajectory history—ensuring the neural networks are firmly tethered to rigorous Newtonian kinematics.

- **Stage 4: Make the Final Decision via Dynamic Weighting**

To make the final decision on which bounding box to track, the architecture fuses the AI perception layer with the IMM-CKF kinematic layer using a dynamically weighted cost function.

Because the OSNet Cosine Similarity (S_{vis}) is bounded between $[0, 1]$ and the IMM-CKF Mahalanobis distance (D_m) is bounded between $[0, \infty)$, we first map the distance into a Kinematic Similarity Score using the Gaussian exponential:

$$S_{kin} = \exp(-0.5 \cdot D_m^2)$$

The architecture then calculates the "Visual Ambiguity Margin"—the difference in similarity between the top two visual detections. If the margin is large, the target is visually unique, and the dynamic weight (α)

is set to 0.8 to heavily favor the AI. If the margin drops below 0.05 (e.g., the target merges with an identical swarm drone), the system detects visual ambiguity and drops α to 0.2.

The final tracking decision is determined by evaluating every bounding box against the fusion equation:

$$\text{Fusion Score} = \alpha S_{vis} + (1 - \alpha) S_{kin}$$

By dynamically shifting the weight, the system seamlessly transitions from relying on visual textures to relying strictly on physical momentum whenever the neural networks become confused.

16.3 The Vision-to-Kinematic Bridge

Once the correct target bounding box is isolated, the architecture must project it from 2D pixels into 3D Earth-fixed space (North-East-Down, or NED) to feed the IMM-CKF. This requires rigorous, frame-by-frame mathematical compensation for the payload's physical hardware states

1. **Camera Zoom In/Out** (The Dynamic Intrinsic Matrix): As the camera zooms, the field of view narrows, meaning the effective focal length changes dynamically. The Jetson parses the 100Hz STM32 serial packets to update the focal length variables ($f_x(t)$,$f_y(t)$) in real time. We define the dynamic intrinsic camera matrix $K(t)$ as:

$$K(t) = \begin{bmatrix} f_x(t) & 0 & c_x \\ 0 & f_y(t) & c_y \\ 0 & 0 & 1 \end{bmatrix}$$

Let the YOLO bounding box center be the pixel coordinate $\mathrm{p} = [u, v, 1]^T$. The normalized 3D ray in the local camera coordinate frame ($\vec{r}_c$) is calculated via the inverse intrinsic matrix:

$$\vec{r}_c = K(t)^{-1}\mathrm{p} = \begin{bmatrix} (u - c_x)/f_x(t) \\ (v - c_y)/f_y(t) \\ 1 \end{bmatrix}$$

If the tracker does not update $f_x(t)$ mathematically during a zoom-in event, the target will artificially appear to accelerate across the frame, causing the IMM-CKF to incorrectly predict a violent maneuver.

2. **Kinematic Compensation for Gimbal and Vehicle Rotation** The local camera ray $\vec{r}_c$ is physically meaningless to an interceptor flying in global space. It must be rotated through the active mechanical linkages of the system.

- R_{gimbal}: A 3D rotation matrix constructed from the STM32's 100Hz Pan and Tilt encoders.
- $R_{vehicle}$: A 3D rotation matrix constructed from the Pixhawk's 10Hz Euler angles (Roll, Pitch, Yaw), mapping the airframe chassis to Earth-fixed NED coordinates.

3. **Cross-Platform Installation Variations** (R_{mount}) A tracker might be mounted upright on the top of an armored ground vehicle, or suspended upside-down beneath the wing of a drone. Instead of maintaining distinct codebases for different military branches, the architecture utilizes a static orientation matrix, R_{mount}, loaded from a boot config file. For example, an upside-down drone mount simply applies a 180-degree roll:

$$R_{mount} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & -1 \end{bmatrix}$$

The Global Line-of-Sight Equation The true 3D unit direction vector pointing from the interceptor to the target in the Earth-fixed NED coordinate frame ($\vec{r}_{NED}$) is computed by chaining these rotation matrices together:

$$\vec{r}_{NED} = R_{vehicle}(\text{10Hz}) \cdot R_{mount}(\text{Static}) \cdot R_{gimbal}(\text{100Hz}) \cdot \vec{r}_c$$

16.4 Fusing the IMM-CKF Architecture

With the target geometrically compensated, the data is handed to the IMM-CKF tracking engine. This engine solves the remaining multi-domain tactical challenges:

1. **High-G Movement**: As proven in Chapter 15.2, an 11-state Constant Acceleration model amplifies visual pixel jitter, destroying track stability. Instead, the IMM handles high-G movement using an inflated-noise Constant Velocity (CV) model. During a violent evasive turn, the IMM seamlessly shifts probability weight to this "Agility Branch," mathematically bypassing the linear kinematic memory and snapping the track to the new trajectory.
2. **Toggle IR/Visible Sensor on the Fly**: When the FPGA signals a switch from a high-resolution Visible sensor to a lower-resolution IR sensor, the measurement uncertainty (R_k) degrades drastically due to quantization drop and thermal blooming. The IMM-CKF dynamically scales its Measurement Noise matrix on the exact frame of the toggle:

$$R_{IR} = \alpha R_{Vis}$$

(Where α is derived from resolution ratios and the Thermal Blooming Coefficient, β_{th}, defined in Appendix Q).

Mathematical Impact: Recall the Kalman Gain equation: $K_k = P_{k|k-1}H^T(HP_{k|k-1}H^T + R_k)^{-1}$. By instantly inflating R_k, the denominator grows massively, shrinking the Kalman Gain. This mathematically forces the CKF to temporarily ignore the fuzzy IR pixel data and rely heavily on its internal momentum ($P_{k|k-1}$) until the IR track stabilizes, preventing mechanical whiplash in the gimbal control loop.

3. **Target States (Taking off vs. Airborne)**: To determine target range from a monocular 2D camera, the IMM maintains domain-specific kinematic models.

- **The Ground Model**: Clamps the target's Z-velocity to zero ($V_z = 0$). Assuming a flat Earth ($Z_{NED} = 0$) and knowing the interceptor's current GPS altitude (H_{UAV}), the filter calculates the precise 3D target location using Geometric Intersection along the line-of-sight vector ($\vec{r}_{NED}$ derived in section 16.3):

$$\text{Slant Range } d = \frac{-H_{UAV}}{\vec{r}_{NED,z}}$$

$$\mathrm{P}_{target} = \mathrm{P}_{UAV} + d \cdot \vec{r}_{NED}$$

- **The Airborne Model**: Unconstrained in all 3 axes, estimating range entirely from dynamic motion parallax and scale priors.

Mathematical Proof of Domain Transition: When an airborne target (e.g., a drone) takes off, its physical Z_{NED} coordinate becomes negative (gaining altitude). The Ground Model, mathematically forced to predict $Z_{NED} = 0$, will generate a massive measurement innovation ($\nu_g = z_{meas} - \hat{z}_g$). Because the IMM Likelihood function is an exponential decay based on innovation squared ($\Lambda_g \propto \exp(-\frac{1}{2}\nu_g^T S_g^{-1} \nu_g)$), the Ground Model's likelihood crashes toward zero. Simultaneously, the unconstrained Airborne Model correctly predicts the vertical ascent, maintaining a high likelihood ($\Lambda_a \approx 1$). The Markov mixing equations autonomously transfer the track probability from the ground to the air within milliseconds, seamlessly tracking the liftoff without dropping the lock.

16.5 C++ Code: The Asynchronous Software Architecture

The following C++ architecture demonstrates the high-level multithreaded control loop running on the Jetson Orin NX. It explicitly shows the four distinct threads (std::thread), the asynchronous Ringbuffers, and the master Tracker loop seamlessly fusing OSNet Cosine Similarity and IMM-CKF Mahalanobis distance.

```
#include <Eigen/Dense>
#include <vector>
#include <cmath>
#include <thread>
#include <atomic>
#include <mutex>
#include "nvbufsurface.h" // Jetson Multimedia API (Zero-Copy)

// Data Structures for the Ringbuffers
struct BoundingBox { float x, y, w, h; int class_id; };
struct FeatureEmbedding { Eigen::VectorXf vec; };

struct HistoricalFrameData {
    uint64_t timestamp_ms;
    uint32_t frame_index;
    NvBufSurface* nv12_surf; // Replacing OpenCV for true zero-copy
    std::vector<BoundingBox> yolo_boxes;
    std::vector<FeatureEmbedding> osnet_embeddings;
};

// Forward declarations of Thread-Safe Ringbuffers and Sub-Systems
class GimbalRingBuffer;
class AutopilotRingBuffer;
class HistoricalFrameBuffer;
class YOLOv8_Inference;
class OSNet_ReID;
class IMM_CKF_System;
class NVEnc_Streamer;

class MultiThreadedTrackerPipeline {
```

```
private:
    // Thread-Safe Buffers
    GimbalRingBuffer* gimbal_buf;
    AutopilotRingBuffer* autopilot_buf;
    HistoricalFrameBuffer* history_buf;

    // Core Engine Pointers
    YOLOv8_Inference* yolo;
    OSNet_ReID* reid;
    IMM_CKF_System* imm_ckf;
    NVEnc_Streamer* h265_streamer;

    // Thread Control
    std::thread vision_thread;
    std::thread gimbal_thread;
    std::thread autopilot_thread;
    std::thread tracker_thread;
    std::atomic<bool> system_running{true};

    // Target Lock State
    FeatureEmbedding locked_target_features;
    std::atomic<bool> is_target_locked{false};
    std::mutex lock_mutex;

    float calculateCosineSimilarity(const FeatureEmbedding& a, const FeatureEmbedding& b) {
        float dot = a.vec.dot(b.vec);
        return (a.vec.norm() == 0 || b.vec.norm() == 0) ? 0.0f : dot / (a.vec.norm() * b.vec.norm());
    }

    // ============================================================================
    // THREAD 1: Zero-Copy Video Pipeline (60 Hz)
    // ============================================================================
    void visionThreadLoop() {
        while (system_running) {
            uint64_t timestamp_ms;
            uint32_t frame_index;
            // Native V4L2 mmap capture into zero-copy NvBufSurface
            NvBufSurface* nv12_surf = captureV4L2Frame(timestamp_ms, frame_index);

            // AI Stage: TensorRT YOLOv8 Detection and OSNet Extraction
            std::vector<BoundingBox> detections = yolo->runInference(nv12_surf);
            std::vector<FeatureEmbedding> embeddings;
            for (const auto& det : detections) {
                embeddings.push_back(reid->extractFeatures(nv12_surf, det));
            }

            // Store into Ringbuffer for the Tracker Thread
            history_buf->push(timestamp_ms, frame_index, nv12_surf, detections, embeddings);
```

```
        // Encode & Stream Downlink bypassing CPU (Hardware NVENC)
        h265_streamer->encodeAndStreamUDP(nv12_surf, timestamp_ms);
    }
}

// ============================================================================
// THREAD 2: Gimbal Telemetry (100 Hz)
// ============================================================================
void gimbalThreadLoop() {
    while (system_running) {
        // Read RS232 Serial from STM32 Controller
        GimbalPacket packet = readRS232Port();
        gimbal_buf->push(packet.timestamp_ms, packet.pan, packet.tilt, packet.zoom_focal_length);
    }
}

// ============================================================================
// THREAD 3: Autopilot Telemetry (10 Hz)
// ============================================================================
void autopilotThreadLoop() {
    while (system_running) {
        // Read RS422 Serial from Pixhawk
        MavlinkPacket packet = readMavlinkPort();
        autopilot_buf->push(packet.timestamp_ms, packet.roll, packet.pitch, packet.yaw, packet.veloc
    }
}

// ============================================================================
// THREAD 4: IMM-CKF Master Tracker (Triggered by Vision Thread)
// ============================================================================
void trackerThreadLoop() {
    while (system_running) {
        // Wait and pop the latest fully processed frame bundle
        HistoricalFrameData frame_data = history_buf->popLatest();
        if (!is_target_locked) continue;

        // 1. Synchronize Geometric Bridge using exact frame timestamp
        Eigen::Matrix3d R_gimbal = gimbal_buf->getAttitudeAt(frame_data.timestamp_ms);
        Eigen::Matrix3d R_vehicle = autopilot_buf->getAttitudeAt(frame_data.timestamp_ms);
        Eigen::Matrix3d R_mount; // Loaded statically
        double focal_length = gimbal_buf->getZoomStateAt(frame_data.timestamp_ms);
        Eigen::Matrix3d R_total = R_vehicle * R_mount * R_gimbal;

        // 2. Data Association via Dynamic Weighting (OSNet + IMM-CKF)
        BoundingBox best_match;
        int best_match_idx = -1;
        float highest_fusion_score = -1.0f;
```

```
bool match_found = false;

float top1_sim = 0.0f, top2_sim = 0.0f;
std::vector<float> visual_scores(frame_data.yolo_boxes.size());

std::lock_guard<std::mutex> lock(lock_mutex); // Protect locked_target_features

for (size_t i = 0; i < frame_data.yolo_boxes.size(); ++i) {
    float sim = calculateCosineSimilarity(locked_target_features, frame_data.osnet_embeddings
    visual_scores[i] = sim;
    if (sim > top1_sim) { top2_sim = top1_sim; top1_sim = sim; }
    else if (sim > top2_sim) { top2_sim = sim; }
}

// Calculate Dynamic Visual Weight
float alpha = (top1_sim - top2_sim < 0.05f) ? 0.2f : 0.8f;

for (size_t i = 0; i < frame_data.yolo_boxes.size(); ++i) {
    if (visual_scores[i] < 0.4f) continue;

    Eigen::VectorXd z_cand(4);
    z_cand << frame_data.yolo_boxes[i].x, frame_data.yolo_boxes[i].y,
              frame_data.yolo_boxes[i].w, frame_data.yolo_boxes[i].h;

    // Extract Kinematic Mahalanobis distance
    double m_dist = imm_ckf->calculateMahalanobisDistance(z_cand);
    float kinematic_score = std::exp(-0.5 * m_dist * m_dist);

    // Dynamic Fusion Equation
    float fusion_score = (alpha * visual_scores[i]) + ((1.0f - alpha) * kinematic_score);

    if (fusion_score > highest_fusion_score) {
        highest_fusion_score = fusion_score;
        best_match = frame_data.yolo_boxes[i];
        best_match_idx = i;
        match_found = true;
    }
}

// 3. Execute IMM-CKF Update
Eigen::VectorXd global_x;
Eigen::MatrixXd global_P;

if (match_found) {
    locked_target_features.vec = 0.9 * locked_target_features.vec + 0.1 * frame_data.osnet_em
    Eigen::VectorXd z_meas(4);
    z_meas << best_match.x, best_match.y, best_match.w, best_match.h;
```

```
                imm_ckf->step(z_meas, R_total, focal_length, global_x, global_P);
            } else {
                imm_ckf->predictOnly(global_x, global_P);
            }

            publishTargetState(global_x, global_P);
        }
    }

public:
    void startSystem() {
        vision_thread = std::thread(&MultiThreadedTrackerPipeline::visionThreadLoop, this);
        gimbal_thread = std::thread(&MultiThreadedTrackerPipeline::gimbalThreadLoop, this);
        autopilot_thread = std::thread(&MultiThreadedTrackerPipeline::autopilotThreadLoop, this);
        tracker_thread = std::thread(&MultiThreadedTrackerPipeline::trackerThreadLoop, this);
    }

    void processGroundStationLock(uint32_t clicked_frame_index, const BoundingBox& clicked_box) {
        std::lock_guard<std::mutex> lock(lock_mutex);
        HistoricalFrameData old_data = history_buf->getFrameByIndex(clicked_frame_index);
        locked_target_features = reid->extractFeatures(old_data.nv12_surf, clicked_box);
        is_target_locked = true;
        // imm_ckf->fastForwardTrack(...) initializes filter in the past and steps forward.
    }
};
```

Chapter 17

Hardware Deployment Considerations

The transition from a high-fidelity MATLAB or Python simulation to a production C++ implementation on an NVIDIA Device is where most industrail tracking projects fail. The math that runs flawlessly in an environment with infinite clock cycles and gigabytes of RAM often shatters when subjected to the reality of real-time embedded systems. This chapter addresses the transition from "mathematically correct" to "tactically deployable."

17.1 Hard Real-Time Determinism for IMM-CKF Estimators

Deploying complex estimators like the IMM-CKF on embedded systems is fundamentally an exercise in time-budget management. If the IMM-CKF overruns its time budget, the entire sensor fusion pipeline desynchronizes, leading to catastrophic tracking failure.

The Deterministic Pipeline Design:

1. **Memory Pre-Allocation and Static Pools**: Dynamic memory allocation (malloc/new) is strictly forbidden in the main tracking loop. The OS allocator is non-deterministic and can introduce unpredictable latency spikes during garbage collection. We utilize static memory pools defined at system boot to house all filter state matrices. By pre-allocating these blocks in contiguous physical memory, we ensure $O(1)$ access times and prevent memory fragmentation that would eventually cause an allocation failure during high-stress flight maneuvers.
2. **Pinned (Page-Locked) Memory for Heterogeneous Compute:** When offloading cubature point propagation to the GPU or running target classification via TensorRT, the host-to-device (H2D) transfer overhead becomes the primary bottleneck. Standard system RAM is "pageable," meaning the OS can move it around at any time, forcing the driver to perform an implicit, slow copy to a staging buffer before sending data to the GPU.

- We implement Pinned (Page-Locked) Memory for our estimator state buffers. By using *cudaHostAlloc*, we lock the memory in physical RAM, allowing the DMA (Direct Memory Access) controller to stream data directly from the estimator to the GPU/DLA without CPU intervention.
- This creates a high-bandwidth, low-latency conduit that allows the IMM-CKF to push cubature point updates to the GPU at wire speed, ensuring that the latency between the estimator and the hardware acceleration remains in the sub-microsecond range.

3. **Fixed-Point vs. Floating-Point**: We utilize Eigen's optimized intrinsic routines, ensuring that the CKF state update executes in constant time.
4. **Interrupt Latency and Thread Pinning**: To prevent the Linux kernel's scheduler from preempting the tracking loop, we use *pthread_setaffinity_np* to pin the tracking thread to a specific dedicated CPU core.

17.2 Exploiting Silicon: Parallelization for Matrix Operations

The primary bottleneck in a Cubature Kalman Filter is the propagation of the **Cubature Points**. For an n-dimensional state, you must calculate $2n$ points, each requiring its own non-linear update.

1. **Tensor-Level Parallelism:** Instead of calculating the $2n$ Cubature points on the CPU, we offload the point generation and the non-linear state propagation to the Jetson's GPU using custom CUDA kernels. Since each cubature point update is independent of the others, this is a perfectly parallel problem.
2. **Memory-Bound Optimization:** Modern embedded silicon is rarely compute-bound; it is **memory-bandwidth bound**. Moving large matrices between the CPU cache and the GPU's global memory (VRAM) consumes more time than the math itself.

- We utilize Unified Memory with cudaMemPrefetchAsync to keep the CKF state matrices residing in the L2 cache as much as possible.
- We implement Tiled Matrix Multiplication in our CUDA kernels to maximize cache reuse, preventing the system from stalling while waiting for data from the slower LPDDR5 RAM.

17.3 Managing Thermal Throttling in Tactical Flight

The interceptor will be exposed to extreme ambient temperatures. If the system enters a thermal throttling state, the clock frequency drops.

Tactical Adaptive Throttling:

- **Graceful Degradation**: When the hardware thermal sensor detects a crossing of the 85°C threshold, the software triggers an autonomous transition from a high-fidelity 11-state CKF model to a lower-fidelity 6-state constant velocity model. By reducing the matrix dimensionality from 11×11 to 6×6, we reduce the computational load by approximately 40%.
- **The "Cool-Down" Logic**:The system monitors the rate of thermal change. If the temperature rise is precipitous, it throttles the neural network inference frequency before it throttles the kinematics, prioritizing the "Math" (the IMM-CKF) over the "Vision" (YOLO/OSNet), as the tracker can coast on inertia for short periods, but it cannot survive an estimator stall.

17.4 The Failure-State Logic

No system is perfect. In a highly contested tactical environment, targets will vanish behind terrain, deploy sophisticated countermeasures, or execute erratic maneuvers that temporarily blind the perception engine. Chapter 17 concludes by outlining the "Safe-State" handler—a deterministic recovery architecture that executes when the IMM-CKF diverges or the neural network loses confidence.

Allowing a diverged filter to output corrupted velocity vectors to the autopilot will cause erratic interceptor behavior or gimbal runaway. The system must fail gracefully and recover actively through a four-stage process:

1. **Divergence Detection**: The system must mathematically recognize its own failure before it issues a bad command. The Safe-State handler is triggered by three concurrent health checks:

- **Sustained Visual Loss**: YOLOv8 fails to detect any bounding box matching the target's class for X consecutive frames.
- **Mahalanobis Spike**: The Mahalanobis distance of all candidate bounding boxes exceeds the 99% chi-square confidence interval, indicating the target has defied the kinematic prediction (e.g., a catastrophic collision or extreme spoofing).
- **Covariance Explosion**: The mathematical trace (the sum of the diagonal elements) of the global covariance matrix P exceeds a predefined safety threshold, indicating total uncertainty.

2. **The Coasting Phase**: Upon detecting divergence, the system immediately enters "Coast Mode." The IMM-CKF halts all measurement updates. It relies purely on the kinematic prediction step ($x_{k|k-1}$) to coast the target along its last known trajectory. Crucially, the software clamps the covariance growth; without bounds, process noise (Q) will inflate P toward infinity, eventually triggering a floating-point overflow that crashes the Jetson.

3. **Active Re-Acquisition**: If coasting fails to yield a valid OSNet match within a defined time window (e.g., 2.0 seconds), the interceptor switches from "Track Mode" to "Search Mode."

- The tracker commands the STM32 gimbal controller to zoom out (widening the Field of View).
- The gimbal initiates a localized raster scan centered on the final predicted coasting coordinate.
- The OSNet Cosine Similarity threshold is temporarily relaxed to cast a wider net for the lost target's visual fingerprint.

4. **Filter Re-Initialization**: When the target is finally re-acquired by YOLO and validated by OSNet, the system cannot simply resume standard filtering. The old covariance matrix is polluted with massive uncertainty from the coasting phase. The IMM-CKF triggers a "Hard Reset," dynamically collapsing the mixed states $\bar{x}$, purging the stale velocity memory, and overwriting the initial covariance $\bar{P}$ with a pre-calibrated re-acquisition matrix to absorb the sudden tracking transient smoothly.

17.5 The Zero-Copy Video Pipeline: Eliminating CPU Overhead

In an tracking scenario, capturing high-resolution video and pushing it through the IMM-CKF estimator and neural network classifier is a high-bandwidth task. If your video frames travel from the capture device → CPU RAM → GPU, you have already lost the "real-time" race due to bus saturation and cache pollution.

1. **The NvBufSurface Advantage**: We utilize NVIDIA's NvBufSurface memory structures to maintain frames in dedicated hardware-accessible memory. By allocating these buffers as unified memory or through the **NVBUF_MEM_SURFACE_ARRAY** type, we enable Zero-Copy operations. This means the pointer to the raw sensor data is passed directly to the processing units without copying the underlying pixels.

2. **Hardware-Accelerated Pre-processing:** Before a frame enters the classifier, it typically requires colorspace conversion (e.g., YUV to RGB) and resizing (e.g., 4K capture down to a 640 × 640 model

input).

- **VIC (Video Image Compositor):** We offload these operations to the hardware VIC engine. By using **NvBufSurfTransform**, we perform the conversion and rescale entirely on hardware. The CPU never touches the pixel values, saving thousands of cycles per frame that would otherwise be spent on SIMD-optimized manual loops.
- **Result:** The frame is "prepared" for the neural network while the CPU is busy solving the IMM-CKF kinematic equations.

3. **Hardware H.264/H.265 Encoding:** For mission logging or data-link transmission, encoding the raw sensor stream is a heavy lift. We bypass the CPU entirely by piping our processed NvBufSurface frames directly into the hardware **NVENC (NVIDIA Encoder)**.

- Because the frames are already in NvBufSurface format (and potentially residing in GPU-mapped memory), the NVENC engine pulls the data directly via DMA.
- This allows us to perform real-time, high-bitrate encoding at 60+ FPS while maintaining a near-zero CPU footprint, ensuring the primary processor remains dedicated to the flight control and estimation logic.

4. **The Holistic Zero-Copy Loop:** The production pipeline follows this deterministic flow:
    1. **Capture**: Sensor data lands in NvBufSurface (Hardware/DMA).
    2. **Pre-process**: VIC performs colorspace conversion and scaling (Hardware).
    3. **Inference**: TensorRT reads directly from the NvBufSurface (Zero-copy).
    4. **Log/Transmit**: NVENC encodes the stream for external storage (Hardware).
    5. **Estimate**: The IMM-CKF runs on the CPU/CUDA, accessing only the meta-data (bounding boxes/coordinates) extracted from step 3.

By strictly enforcing this zero-copy architecture, the CPU and memory bus are effectively "decoupled" from the heavy lifting of video processing. This isolation is what guarantees that the IMM-CKF estimator never misses an update tick, even when the interceptor is streaming multiple high-definition channels simultaneously.

Appendix

A:Derivation of the Spherical-Radial Cubature Rule

In Chapter 1.4, we introduced the Spherical-Radial Integration rule as the foundational mathematics that gives the Cubature Kalman Filter its name and its efficiency. This appendix provides the rigorous derivation proving how an infinite continuous integral collapses into exactly 2n deterministic points.

1. The Core Problem

In Bayesian estimation, we constantly need to evaluate non-linear expectations of Gaussian random variables. This requires solving an integral of the form:

$$I(f) = \int_{\mathbb{R}^n} f(\mathrm{x}) \mathcal{N}(\mathrm{x}; \boldsymbol{\mu}, P) d\mathrm{x}$$

Where $f(\mathrm{x})$ is some non-linear function (like our camera projection geometry), and $\mathcal{N}$ is a Gaussian PDF with mean $\boldsymbol{\mu}$ and covariance P.

Through standard algebraic substitution, any Gaussian can be shifted and scaled into a "Standard Normal" distribution (Mean = 0, Covariance = I). Let $\mathrm{x} = \sqrt{P}\boldsymbol{\xi} + \boldsymbol{\mu}$. The integral simplifies to:

$$I(f) = \int_{\mathbb{R}^n} f(\sqrt{P}\boldsymbol{\xi} + \boldsymbol{\mu}) \mathcal{N}(\boldsymbol{\xi}; 0, I) d\boldsymbol{\xi}$$

The core mathematical challenge is solving the right side: integrating an arbitrary function multiplied by a standard normal weight across all n-dimensional space.

2. Cartesian to Spherical-Radial Transformation

We substitute the Cartesian vector $\boldsymbol{\xi}$ with a spherical-radial coordinate system. Let:

$$\boldsymbol{\xi} = r\mathrm{y}$$

Where: * $r \geq 0$ is the radial distance (a scalar). * y is the directional vector lying on the surface of an n-dimensional unit sphere (U_n), such that $\mathrm{y}^T\mathrm{y} = 1$.

The standard normal PDF is defined as $\mathcal{N}(\boldsymbol{\xi}; 0, I) = \frac{1}{\sqrt{(2\pi)^n}} \exp\left(-\frac{1}{2}\boldsymbol{\xi}^T\boldsymbol{\xi}\right)$. Because $\boldsymbol{\xi}^T\boldsymbol{\xi} = r^2(\mathrm{y}^T\mathrm{y}) = r^2$, the exponential term simplifies brilliantly to $\exp\left(-\frac{r^2}{2}\right)$.

By transforming the differential volume element $d\boldsymbol{\xi}$ into spherical coordinates ($r^{n-1} dr d\sigma(\mathrm{y})$), the massive multidimensional integral splits into two separate, independent integrals:

$$I(f) = \frac{1}{\sqrt{(2\pi)^n}} \int_0^\infty \int_{U_n} f(r\mathrm{y}) r^{n-1} \exp\left(-\frac{r^2}{2}\right) d\sigma(\mathrm{y}) dr$$

This gives us our two target integrals: 1. **The Spherical Integral:** $S(r) = \int_{U_n} f(r\mathrm{y}) d\sigma(\mathrm{y})$ 2. **The Radial Integral:** $R = \int_0^\infty S(r) r^{n-1} \exp\left(-\frac{r^2}{2}\right) dr$

3. Solving the Spherical Integral

The spherical integral calculates the average value of the function over the surface of the sphere.

To approximate this integral numerically, we must select a finite set of points on the sphere. To capture the full symmetry of a Gaussian distribution (which is perfectly symmetric across all axes), our set of points must also be perfectly symmetric.

The most efficient fully symmetric set of points on an n-dimensional sphere are the intersections of the sphere with the Cartesian axes. For an n-dimensional space, there are n axes, each piercing the sphere in two places (positive and negative).

We define this discrete set of points as $[u]_i$. For n dimensions, there are exactly $2n$ points:

$$[u] = \left\{ \begin{bmatrix} 1 \\ 0 \\ \vdots \end{bmatrix}, \begin{bmatrix} -1 \\ 0 \\ \vdots \end{bmatrix}, \begin{bmatrix} 0 \\ 1 \\ \vdots \end{bmatrix}, \begin{bmatrix} 0 \\ -1 \\ \vdots \end{bmatrix}, \dots \right\}$$

By applying the symmetric cubature rule, the continuous spherical surface integral is approximated as a discrete sum over these $2n$ points:

$$S(r) \approx \frac{A_n}{2n} \sum_{i=1}^{2n} f(r[u]_i)$$

(*Where A_n is the surface area of the unit sphere*).

4. Solving the Radial Integral

Now we substitute our discrete spherical sum back into the radial integral:

$$I(f) \approx \frac{1}{2n} \sum_{i=1}^{2n} \left(\frac{A_n}{\sqrt{(2\pi)^n}} \int_0^\infty f(r[u]_i) r^{n-1} \exp\left(-\frac{r^2}{2}\right) dr \right)$$

We must find the correct radius r at which to evaluate these points. We do this by applying Gaussian-Laguerre quadrature (moment matching). We want to find a radius r such that the second statistical moment (the variance) of our discrete points perfectly matches the variance of the true, continuous Gaussian distribution.

The second moment of the standard normal distribution is mathematically proven to be n. Therefore, to perfectly capture the variance of the Gaussian, we set the squared radius equal to the dimension of the state space:

$$r^2 = n \implies r = \sqrt{n}$$

5. The Final Cubature Rule

By plugging $r = \sqrt{n}$ into our equations, the radial integral vanishes completely, and we are left with the final Spherical-Radial Cubature Rule:

$$I(f) \approx \frac{1}{2n} \sum_{i=1}^{2n} f(\sqrt{n}[u]_i)$$

The Engineering Conclusion: The math proves that the complex, impossible continuous integral of a non-linear Gaussian system can be calculated with incredibly high accuracy simply by taking the $2n$ axis unit vectors $([u]_i)$, scaling them outward by a radius of $\sqrt{n}$, pushing them through the non-linear function f, and averaging the result.

This deterministic calculation is what generates the **Cubature Points** (ξ_i) that you will program into your Prediction and Update steps.

B:The square root of a matrix and Cholesky decomposition

The square root of a matrix

A Matrix B is a square root of a matrix A if :

$$A = BB$$

The equation above can have several possible solutions, and there are different computation methods for finding the square root of a matrix.

However, in the context of state estimation, Uncentenced Kalman Filter(UKF) and the Cubature Kalman Filter (CKF), when we speak of the "square root" of a Covariance Matrix (P), we are usually referring to a specific spatial factorization. Because a covariance matrix is symmetric, we are looking for a matrix S such that:

$$P = SS^T$$

Where S^T is the transpose of S. While there are many ways to find an S that satisfies this equation (such as Singular Value Decomposition), the most computationally efficient and numerically stable method for tracking filters is the Cholesky Decomposition.

Positive and semi-definite Matrix Cholesky decoposition

Before we can decompose a matrix using the Cholesky algorithm, the matrix must satisfy a strict mathematical condition: it must be symmetric and positive definite (or positive semi-definite).

1. The Mathematical Definition

A symmetric $n \times n$ matrix A is Positive Definite if, for any non-zero real column vector x, the following holds true:

$$x^T Ax > 0$$

2. The Physical Tracking Definition

In target tracking, the matrix A represents our State Covariance Matrix (P). The diagonals of P are the variances of our state variables (position, velocity, etc.).

Because variance represents physical uncertainty (standard deviation squared, σ^2), it is physically impossible to have "negative uncertainty." Therefore, a valid covariance matrix will always be positive semi-definite. If numerical floating-point errors cause an eigenvalue to slip below zero, the matrix loses its

positive definiteness, and algorithms attempting to find its square root will fail mathematically, returning imaginary numbers (NaNs in C++).

Theorem: Cholesky Decomposition Proof and Derivation

Let A be an $n \times n$ symmetric, positive-definite matrix. There exists a unique lower triangular matrix L with strictly positive diagonal entries ($l_{ii} > 0$) such that

$$A = LL^T$$

A lower triangular matrix is simply a matrix where all entries above the main diagonal are zero.Now we use mathematical induction to proof this Theorem.

1. The Base Case ($n = 1$)

Let A be a 1×1 matrix, so $A = [a_{11}]$. Because A is positive-definite, for any non-zero vector x (which in this case is a scalar $x \neq 0$), $x^T Ax > 0$.

If we set $x = 1$, then $1 \cdot a_{11} \cdot 1 = a_{11} > 0$. We seek a 1×1 lower triangular matrix $L = [l_{11}]$ such that $A = LL^T$.

$$[a_{11}] = [l_{11}][l_{11}] \implies a_{11} = l_{11}^2$$

Since $a_{11} > 0$, there exists a unique, strictly positive real number $l_{11} = \sqrt{a_{11}}$. Thus, the theorem holds for $n = 1$.

2. The Inductive Hypothesis

Assume that the theorem holds for all symmetric, positive-definite matrices of size $k \times k$. This means any $k \times k$ positive-definite matrix can be uniquely decomposed into a lower triangular matrix times its transpose.

3. The Inductive Step ($n = k + 1$)

Now consider a $(k + 1) \times (k + 1)$ symmetric, positive-definite matrix A. We partition A into smaller block matrices:

$$A = \begin{bmatrix} A_{11} & a_{12} \\ a_{12}^T & a_{22} \end{bmatrix}$$

Where: - A_{11} is a $k \times k$ matrix. - a_{12} is a $k \times 1$ column vector. - a_{22} is a scalar (a 1×1 matrix).

Step A: Establishing A_{11} is Positive-Definite

Because A is symmetric and positive-definite, any principal submatrix of A must also be symmetric and positive-definite. Therefore, A_{11} is symmetric and positive-definite.

By our inductive hypothesis, because A_{11} is a $k \times k$ positive-definite matrix, there exists a unique $k \times k$ lower triangular matrix L_{11} with positive diagonals such that:

$$A_{11} = L_{11} L_{11}^T$$

Step B: Constructing L

We want to find a $(k+1) \times (k+1)$ lower triangular matrix L partitioned in the same way as A:

$$L = \begin{bmatrix} L_{11} & 0 \\ l_{21}^T & l_{22} \end{bmatrix}$$

Where:

- L_{11} is the $k \times k$ matrix from our inductive hypothesis.
- l_{21} is a $k \times 1$ column vector.
- l_{22} is a strictly positive scalar.

Let's compute LL^T:

$$LL^T = \begin{bmatrix} L_{11} & 0 \\ l_{21}^T & l_{22} \end{bmatrix} \begin{bmatrix} L_{11}^T & l_{21} \\ 0 & l_{22} \end{bmatrix} = \begin{bmatrix} L_{11}L_{11}^T & L_{11}l_{21} \\ l_{21}^T L_{11}^T & l_{21}^T l_{21} + l_{22}^2 \end{bmatrix}$$

Step C: Equating LL^T to A

By setting LL^T equal to our partitioned A, we get a system of equations:

1 $L_{11}L_{11}^T = A_{11}$
(This is already satisfied by our inductive hypothesis).

2 $L_{11}l_{21} = a_{12}$

Because L_{11} is a lower triangular matrix with strictly positive diagonal entries, its determinant (the product of its diagonals) is non-zero. This means L_{11} is invertible. Therefore, we can uniquely solve for the vector l_{21}:

$$l_{21} = L_{11}^{-1}a_{12}$$

3 $l_{21}^T l_{21} + l_{22}^2 = a_{22}$ We need to solve for the scalar l_{22}:

$$l_{22}^2 = a_{22} - l_{21}^T l_{21}$$

Step D: Proving l_{22} Exists and is Unique

For l_{22} to be a unique, strictly positive real number, we must prove that $(a_{22} - l_{21}^T l_{21}) > 0$.

Let's construct a specific $(k+1) \times 1$ test vector x:

$$x = \begin{bmatrix} y \\ -1 \end{bmatrix}$$

Where y is defined as $y = A_{11}^{-1}a_{12}$.

Because A is positive-definite, $x^T Ax > 0$ for any non-zero vector x. Let's evaluate this:

$$x^T Ax = \begin{bmatrix} y^T & -1 \end{bmatrix} \begin{bmatrix} A_{11} & a_{12} \\ a_{12}^T & a_{22} \end{bmatrix} \begin{bmatrix} y \\ -1 \end{bmatrix}$$

First, multiply the matrix A by the vector x:

$$\begin{bmatrix} A_{11} & a_{12} \\ a_{12}^T & a_{22} \end{bmatrix} \begin{bmatrix} y \\ -1 \end{bmatrix} = \begin{bmatrix} A_{11}y - a_{12} \\ a_{12}^T y - a_{22} \end{bmatrix}$$

Substitute $y = A_{11}^{-1}a_{12}$ into the top row:

$$A_{11}(A_{11}^{-1}a_{12}) - a_{12} = a_{12} - a_{12} = 0$$

Now multiply by x^T:

$$x^T Ax = \begin{bmatrix} y^T & -1 \end{bmatrix} \begin{bmatrix} 0 \\ a_{12}^T y - a_{22} \end{bmatrix} = -1 \cdot (a_{12}^T y - a_{22}) = a_{22} - a_{12}^T y$$

Substitute $y = A_{11}^{-1}a_{12}$ back into this result:

$$x^T Ax = a_{22} - a_{12}^T A_{11}^{-1} a_{12}$$

Since $x^T Ax > 0$, we now know that $a_{22} - a_{12}^T A_{11}^{-1} a_{12} > 0$.

Finally, let's look back at our term $l_{21}^T l_{21}$ from Step C. Recall that $l_{21} = L_{11}^{-1}a_{12}$.

$$l_{21}^T l_{21} = (L_{11}^{-1}a_{12})^T (L_{11}^{-1}a_{12}) = a_{12}^T (L_{11}^{-1})^T L_{11}^{-1} a_{12}$$

Since $(L_{11}^{-1})^T L_{11}^{-1} = (L_{11}L_{11}^T)^{-1} = A_{11}^{-1}$, we get:

$$l_{21}^T l_{21} = a_{12}^T A_{11}^{-1} a_{12}$$

Therefore, substituting this back into our equation for l_{22}^2:

$$l_{22}^2 = a_{22} - a_{12}^T A_{11}^{-1} a_{12}$$

Since we just proved that this exact expression is strictly greater than zero, $l_{22}^2 > 0$. Thus, $l_{22} = \sqrt{a_{22} - l_{21}^T l_{21}}$ exists and is a unique, strictly positive real number.

Conclusion

By the principle of mathematical induction, because the theorem holds for $n = 1$, and assuming it holds for k guarantees it holds for $k + 1$, we conclude that every symmetric, positive-definite matrix can be uniquely factored into $A = LL^T$.

Algorithm: Cholesky Decomposition

A lower triangular matrix is simply a matrix where all entries above the main diagonal are zero. To showcase how this factorization is deterministically achieved, let us manually expand the equation for a 3×3 matrix.Let the known symmetric covariance matrix be A:

$$A = \begin{bmatrix} a_{11} & a_{21} & a_{31} \\ a_{21} & a_{22} & a_{32} \\ a_{31} & a_{32} & a_{33} \end{bmatrix}$$

(Note: Because A is symmetric, $a_{12} = a_{21}$, $a_{13} = a_{31}$, etc.)

We want to find the unknown lower triangular matrix L:

$$L = \begin{bmatrix} l_{11} & 0 & 0 \\ l_{21} & l_{22} & 0 \\ l_{31} & l_{32} & l_{33} \end{bmatrix}$$

And its transpose L^T:

$$L^T = \begin{bmatrix} l_{11} & l_{21} & l_{31} \\ 0 & l_{22} & l_{32} \\ 0 & 0 & l_{33} \end{bmatrix}$$

If we physically multiply $L \times L^T$, we get:

$$LL^T = \begin{bmatrix} l_{11}^2 & l_{11}l_{21} & l_{11}l_{31} \\ l_{21}l_{11} & l_{21}^2 + l_{22}^2 & l_{21}l_{31} + l_{22}l_{32} \\ l_{31}l_{11} & l_{31}l_{21} + l_{32}l_{22} & l_{31}^2 + l_{32}^2 + l_{33}^2 \end{bmatrix}$$

Because $A = LL^T$, we can now set the elements of our multiplied matrix directly equal to the elements of A and solve for the unknown l values recursively, starting from the top-left corner.

Step 1: Solve the first column

$$a_{11} = l_{11}^2 \implies l_{11} = \sqrt{a_{11}}$$
$$a_{21} = l_{21}l_{11} \implies l_{21} = \frac{a_{21}}{l_{11}}$$
$$a_{31} = l_{31}l_{11} \implies l_{31} = \frac{a_{31}}{l_{11}}$$

Step 2: Solve the seconde column

$$a_{22} = l_{21}^2 + l_{22}^2 \implies l_{22} = \sqrt{a_{22} - l_{21}^2}$$
$$a_{32} = l_{21}l_{31} + l_{22}l_{32} \implies l_{32} = \frac{a_{32} - l_{21}l_{31}}{l_{22}}$$

Step 3: Solve the third column

$$a_{33} = l_{31}^2 + l_{32}^2 + l_{33}^2 \implies l_{33} = \sqrt{a_{33} - l_{31}^2 - l_{32}^2}$$

The General Algorithm

By recognizing the pattern in the algebraic proof above, we can abstract this into a set of generalized equations that a computer can run for an $n \times n$ matrix.For the diagonal elements ($j = i$): For the diagonal elements ($j = i$):

$$l_{jj} = \sqrt{a_{jj} - \sum_{k=1}^{j-1} l_{jk}^2}$$

For the off-diagonal elements ($i > j$):

$$l_{ij} = \frac{1}{l_{jj}} \left(a_{ij} - \sum_{k=1}^{j-1} l_{ik} l_{jk} \right)$$

Why Cholesky for the IMM-CKF?

In the Cubature Kalman Filter, we must generate $2n$ cubature points at every single time step for every single active IMM model. To do this, we need the matrix "square root" of the predicted state covariance matrix P.

Using the Cholesky decomposition to find $P = LL^T$ is the premier choice for embedded edge engineering for two reasons:

- **Computational Efficiency**: Standard matrix square root algorithms (like SVD or eigenvalue decomposition) have high computational overhead. Because Cholesky exploits the inherent symmetry of the covariance matrix and only solves for half the matrix (the lower triangle), it requires approximately half the floating-point operations ($O(n^3/3)$).
- **Deterministic Point Generation**: Multiplying our standard unit vectors by the lower triangular matrix L cleanly scales and rotates our cubature points directly along the principal axes of the target's uncertainty ellipse, ensuring mathematically stable propagation through non-linear measurement models.

C:Expectation Algebra and Covariance Propagation

To understand why the standard Kalman filter equations take the shape they do, one must understand how expectation (the expected value) acts as a mathematical operator.

1. Linearity of Expectation

The Expectation operator $E[\cdot]$ is perfectly linear. For any random variable x, constant matrix A, and constant vector B:

$$E[Ax + B] = AE[x] + B$$

2. Definition of Covariance

The covariance matrix P of a random variable x with mean $\mu = E[x]$ is defined as the expected value of the outer product of its deviations:

$$Cov(x) = P = E[(x - \mu)(x - \mu)^T]$$

3. The Linear Transformation Proof

In tracking, our kinematic equations (like the Constant Velocity model) apply a linear transformation to our state vector to predict the future:

$$x_{k+1} = Fx_k + w$$

Where F is the transition matrix and w is zero-mean process noise ($E[w] = 0$) with a covariance of Q.

Theorem: If a random variable x undergoes a linear transformation $y = Fx + w$, the covariance of y is $FPF^T + Q$.

Proof:

First, find the mean of our new variable y. Let the mean of x be μ_x.

$$\mu_y = E[y] = E[Fx + w]$$

By linearity of expectation:

$$\mu_y = FE[\mathrm{x}] + E[\mathrm{w}] = F\mu_x + 0$$

Now, apply the definition of covariance to y:

$$Cov(\mathrm{y}) = E[(\mathrm{y} - \mu_y)(\mathrm{y} - \mu_y)^T]$$

Substitute the definitions of y and μ_y:

$$Cov(\mathrm{y}) = E[(F\mathrm{x} + \mathrm{w} - F\mu_x)(F\mathrm{x} + \mathrm{w} - F\mu_x)^T]$$
$$Cov(\mathrm{y}) = E[(F(\mathrm{x} - \mu_x) + \mathrm{w})(F(\mathrm{x} - \mu_x) + \mathrm{w})^T]$$

Expand the outer product $(a + b)(a + b)^T = aa^T + ab^T + ba^T + bb^T$:

$$Cov(\mathrm{y}) = E[F(\mathrm{x} - \mu_x)(\mathrm{x} - \mu_x)^T F^T \quad + \quad F(\mathrm{x} - \mu_x)\mathrm{w}^T \quad + \quad \mathrm{w}(\mathrm{x} - \mu_x)^T F^T \quad + \quad \mathrm{w}\mathrm{w}^T]$$

Apply the expectation operator linearly across the terms. Because the state estimate error $(\mathrm{x} - \mu_x)$ and the random process noise w are statistically independent, the expectation of their cross-products is zero: $E[F(\mathrm{x} - \mu_x)\mathrm{w}^T] = 0$.This leaves:

$$Cov(\mathrm{y}) = F \cdot E[(\mathrm{x} - \mu_x)(\mathrm{x} - \mu_x)^T] \cdot F^T + E[\mathrm{w}\mathrm{w}^T]$$

Recognizing the original definitions of P and Q:

$$Cov(\mathrm{y}) = FPF^T + Q$$

This proof forms the mathematical basis for the Prediction Step in every linear and extended Kalman Filter ever written.

D:The Gaussian Multiplication Proof and the Origins of the Kalman Gain

In Chapter 4, we stated that Bayes' Theorem operates by multiplying the Prior probability distribution (our kinematic prediction) by the Likelihood distribution (our sensor measurement). We also stated that because both of these are Gaussian (Normal) distributions, multiplying them magically produces a third, narrower Gaussian distribution representing our updated estimate (the Posterior).

But why does multiplying two bell curves create another perfect bell curve? And more importantly for an engineer, how does this algebra actually translate into code?

By stepping through the algebraic proof of multiplying two 1D Gaussian distributions, we will witness the exact mathematical birth of the Kalman Gain (K) and the standard update equations used in every tracking filter.

1. Defining the Two Distributions

Let us define our two independent pieces of information as 1D Gaussian Probability Density Functions (PDFs):

A. The Prior (Our Physics Prediction)

- Mean (State Estimate): μ_1
- Variance (Uncertainty): σ_1^2

$$p_{prior}(x) = \frac{1}{\sqrt{2\pi\sigma_1^2}} \exp\left(-\frac{(x-\mu_1)^2}{2\sigma_1^2}\right)$$

B. The Likelihood (Our Sensor Measurement)

- Mean (Measured Value): μ_2
- Variance (Sensor Noise): σ_2^2

$$p_{likelihood}(x) = \frac{1}{\sqrt{2\pi\sigma_2^2}} \exp\left(-\frac{(x-\mu_2)^2}{2\sigma_2^2}\right)$$

2. Multiplying the Distributions

According to Bayes' theorem, to find our Posterior distribution, we multiply these two PDFs together.

$$p_{posterior}(x) \propto p_{prior}(x) \cdot p_{likelihood}(x)$$

When multiplying exponential functions, we simply add their exponents. We can safely ignore the scaling constants ($\frac{1}{\sqrt{2\pi\sigma^2}}$) for now, as they only serve to ensure the final curve's area equals 1.0. Let's focus entirely on the new exponent:

$$\exp\left(-\frac{(x-\mu_1)^2}{2\sigma_1^2}\right) \cdot \exp\left(-\frac{(x-\mu_2)^2}{2\sigma_2^2}\right) = \exp\left(-\frac{1}{2}\left[\frac{(x-\mu_1)^2}{\sigma_1^2} + \frac{(x-\mu_2)^2}{\sigma_2^2}\right]\right)$$

Now, let's group these terms by the powers of x (the x^2 terms, the x terms, and the constants):

$$x^2\left(\frac{1}{\sigma_1^2} + \frac{1}{\sigma_2^2}\right) - 2x\left(\frac{\mu_1}{\sigma_1^2} + \frac{\mu_2}{\sigma_2^2}\right) + \left(\frac{\mu_1^2}{\sigma_1^2} + \frac{\mu_2^2}{\sigma_2^2}\right)$$

3. Expanding the Exponent

To prove this results in a new Gaussian, we need to manipulate the bracketed sum into the standard Gaussian form: $\frac{(x-\mu_{new})^2}{\sigma_{new}^2}$.

Let's expand the quadratic terms inside the brackets:

$$\frac{x^2 - 2\mu_1 x + \mu_1^2}{\sigma_1^2} + \frac{x^2 - 2\mu_2 x + \mu_2^2}{\sigma_2^2}$$

Now, let's group these terms by the powers of x (the x^2 terms, the x terms, and the constants):

$$x^2\left(\frac{1}{\sigma_1^2} + \frac{1}{\sigma_2^2}\right) - 2x\left(\frac{\mu_1}{\sigma_1^2} + \frac{\mu_2}{\sigma_2^2}\right) + \left(\frac{\mu_1^2}{\sigma_1^2} + \frac{\mu_2^2}{\sigma_2^2}\right)$$

4. Completing the Square (Finding the New Mean and Variance)

We want our grouped equation to perfectly match the expanded form of our target Gaussian exponent:

$$\frac{1}{\sigma_{new}^2}(x^2 - 2\mu_{new}x + \mu_{new}^2)$$

By matching the coefficients of x^2, we can find our new variance (σ_{new}^2):

$$\frac{1}{\sigma_{new}^2} = \frac{1}{\sigma_1^2} + \frac{1}{\sigma_2^2}$$

Solving for σ_{new}^2:

$$\sigma_{new}^2 = \frac{\sigma_1^2\sigma_2^2}{\sigma_1^2 + \sigma_2^2}$$

Next, by matching the coefficients of x, we can find our new mean (μ_{new}):

$$\frac{\mu_{new}}{\sigma_{new}^2} = \frac{\mu_1}{\sigma_1^2} + \frac{\mu_2}{\sigma_2^2}$$

Substitute our newly found σ_{new}^2 and solve for μ_{new}:

$$\mu_{new} = \left(\frac{\sigma_1^2 \sigma_2^2}{\sigma_1^2 + \sigma_2^2} \right) \left(\frac{\mu_1 \sigma_2^2 + \mu_2 \sigma_1^2}{\sigma_1^2 \sigma_2^2} \right)$$

The cross-terms cancel out, leaving:

$$\mu_{new} = \frac{\mu_1 \sigma_2^2 + \mu_2 \sigma_1^2}{\sigma_1^2 + \sigma_2^2}$$

We have now mathematically proven that multiplying two Gaussians creates a new Gaussian, and we have the exact formulas for its Mean and Variance.

5. The Engineering Translation: The Birth of the Kalman Gain

While the formulas above are mathematically pure, they are not how engineers write tracking code. Let's rewrite our new Mean (μ_{new}) formula by splitting it apart and rearranging it:

$$\mu_{new} = \mu_1 \frac{\sigma_2^2}{\sigma_1^2 + \sigma_2^2} + \mu_2 \frac{\sigma_1^2}{\sigma_1^2 + \sigma_2^2}$$

Through basic algebra, we can factor this into an "update" format—taking our original prediction (μ_1) and adding a correction to it:

$$\mu_{new} = \mu_1 + \left(\frac{\sigma_1^2}{\sigma_1^2 + \sigma_2^2} \right) (\mu_2 - \mu_1)$$

Let's look at this equation through the lens of a tracking engineer:

- μ_1 is our Predicted State ($\hat{x}_{k|k-1}$)
- μ_2 is our Sensor Measurement (z_k)
- $(\mu_2 - \mu_1)$ is our Prediction Error, or Innovation (y_k).

What is that fractional term in the middle? That is the ratio of our prediction uncertainty to the total system uncertainty. It dictates exactly how much we should trust the sensor's innovation.

This is the 1D Kalman Gain (K):

$$K = \frac{\sigma_1^2}{\sigma_1^2 + \sigma_2^2}$$

Substituting K back into our equations yields the exact, universally recognized 1D Kalman Filter update equations:

- State Update: $\hat{x}_{k|k} = \hat{x}_{k|k-1} + K(z_k - \hat{x}_{k|k-1})$
- Covariance Update: $\sigma_{new}^2 = (1 - K)\sigma_1^2$

The Matrix Generalization

When scaling from a 1D example to the multi-dimensional tracking of a drone using an IMM-CKF, the algebra remains identical, but the notation shifts to linear algebra matrices.

- The Prior Variance (σ_1^2) becomes the Predicted Covariance Matrix (P).
- The Likelihood Variance (σ_2^2) becomes the Measurement Noise Matrix (R).
- The addition in the denominator becomes a matrix inversion.

Thus, the multidimensional Kalman Gain emerges directly from the algebra of Gaussian multiplication:

$$K = PH^T(HPH^T + R)^{-1}$$

(Note: The H matrix simply projects the state space into the measurement space so the matrices align properly).

E:The Proof of Linear Observability

In Chapter 5.2, we stated that a discrete linear time-invariant (LTI) system is fully observable if its Observability Matrix (O) has full column rank. Here is the formal mathematical proof.

1. Defining the System

Consider an unforced discrete LTI system (we drop the control matrix B because it is known, and we drop the noise terms as observability is a deterministic property of the system matrices):

$$\mathrm{x}_{k+1} = F\mathrm{x}_k$$

$$\mathrm{z}_k = H\mathrm{x}_k$$

Where $\mathrm{x} \in \mathbb{R}^n$ is the n-dimensional state vector, and $\mathrm{z} \in \mathbb{R}^m$ is the m-dimensional measurement vector.

2. Expanding the Measurements over Time

The goal of observability is to determine the initial state x_0 strictly by looking at a sequence of measurements. Let us expand the first n measurements in terms of x_0:

At $k = 0$:

$$\mathrm{z}_0 = H\mathrm{x}_0$$

At $k = 1$:

$$\mathrm{x}_1 = F\mathrm{x}_0$$

$$\mathrm{z}_1 = H\mathrm{x}_1 = HF\mathrm{x}_0$$

At $k = 2$:

$$\mathrm{x}_2 = F\mathrm{x}_1 = F^2\mathrm{x}_0$$

$$\mathrm{z}_2 = H\mathrm{x}_2 = HF^2\mathrm{x}_0$$

Continuing this pattern up to $k = n - 1$:

$$\mathrm{z}_{n-1} = HF^{n-1}\mathrm{x}_0$$

3. Assembling the Matrix Equation

We can stack these individual measurement vectors into one massive column vector, denoted as Z:

$$Z = \begin{bmatrix} z_0 \\ z_1 \\ z_2 \\ \vdots \\ z_{n-1} \end{bmatrix} = \begin{bmatrix} Hx_0 \\ HFx_0 \\ HF^2x_0 \\ \vdots \\ HF^{n-1}x_0 \end{bmatrix}$$

Because x_0 is a common factor in every term, we can factor it out of the matrix:

$$Z = \begin{bmatrix} H \\ HF \\ HF^2 \\ \vdots \\ HF^{n-1} \end{bmatrix} x_0$$

The stacked matrix block on the right is exactly the Observability Matrix (O):

$$Z = Ox_0$$

(Note: Because we are stacking n individual measurement vectors, each of dimension $m \times 1$, the resulting stacked vector Z has dimensions $(mn) \times 1$, and the Observability Matrix O has dimensions $(mn) \times n$)

4. The Rank Condition Proof

We have now reduced the problem to a standard linear algebra equation: $Ax = b$, where A is our Observability Matrix O, x is our unknown initial state x_0, and b is our stacked measurements Z.

By the fundamental theorem of linear algebra, to find a unique, exact solution for x_0, the matrix O must have an inverse (or a left pseudo-inverse).

> **Deep Drive: What is the Pseudo-Inverse for Rectangular Matrices**
>
> Note: Because the Observability matrix is a tall, rectangular matrix where the number of rows exceeds the number of columns, it cannot be inverted using a standard matrix inverse. Solving the equation $Z = Ox_0$ requires computing the Left Pseudo-Inverse. For a complete mathematical breakdown of how to invert non-square matrices in tracking, see **Appendix F: What is the Pseudo-Inverse for Rectangular Matrices**.

For O to have a left pseudo-inverse, its columns must be linearly independent. In other words, O must have full column rank. Because the state vector x_0 has n dimensions, O must have rank n.

If the rank is strictly less than n, the system is underdetermined. The null space of O is non-empty, meaning there are infinite possible values of x_0 that could produce the exact same sequence of measurements Z. The state is mathematically hidden from the sensor, proving the system is unobservable.

(Why do we stop at $n - 1$? By the Cayley-Hamilton theorem, any power of a matrix F^k for $k \geq n$ can be expressed as a linear combination of its lower powers. Therefore, adding more measurements beyond z_{n-1} provides no fundamentally new information to the rank of the matrix).

F:What is the Pseudo-Inverse for Rectangular Matrices

For an $m \times n$ matrix A to have a standard inverse, the very first and most absolute property it must have is that m must equal n.

In linear algebra, a true, two-sided inverse (where $AA^{-1} = I$ and $A^{-1}A = I$) only exists for square matrices.

If the matrix is square ($n \times n$), it must possess several mathematical properties to be invertible.

However, in engineering, we constantly deal with rectangular matrices ($m \neq n$), which require a Pseudo-Inverse.Here is the exact breakdown of the properties required for both standard inverses and pseudo-inverses.

1. The Standard Inverse (Square Matrices: $m = n$)

If your matrix is square (e.g., a 4×4 State Transition Matrix F), it must be **non-singular** to have an inverse. A matrix is non-singular if it meets all of the following equivalent properties:

- **Non-Zero Determinant**: $\det(A) \neq 0$. This is the fastest mathematical check.
- **Full Rank**: The rank of the matrix must be n. This means all n columns (and all n rows) are linearly independent. None of them can be formed by adding or scaling the other columns.
- **Empty Null Space**: The equation $Ax = 0$ has only one trivial solution: $x = 0$. This means the matrix does not "crush" any vectors down to zero.
- **Non-Zero Eigenvalues**: Every single eigenvalue of A is strictly non-zero. (This links directly to why we regularize the Cholesky decomposition with epsilon if an eigenvalue hits zero!).

2. The Pseudo-Inverse (Rectangular Matrices: $m \neq n$)

If $m \neq n$, the system is either overdetermined (too many equations) or underdetermined (too many unknowns). It cannot have a standard inverse. Instead, we use the Moore-Penrose Pseudo-Inverse (often denoted as A^+).

Whether it has a left or right pseudo-inverse depends entirely on its shape and its rank.

Case A: Tall Matrices ($m > n$) -> Requires a "Left Inverse"

This is an overdetermined system (more rows than columns). The Observability Matrix (O) from our previous discussion is a classic example of a tall matrix.

For a tall matrix to have a unique left inverse, it must have Full Column Rank.

- **Property**: The rank must equal n (the smaller dimension). All columns must be linearly independent.
- **The Math**: The left pseudo-inverse is calculated as $A_{left}^{+} = (A^T A)^{-1} A^T$.

- **The Result**: Multiplying from the left yields the identity matrix: $A^{+}_{left}A = I_n$.
- **Engineering Meaning**: In tracking, this means you have enough unique sensor measurements to uniquely calculate a single, "best-fit" initial state using Least Squares.

Case B: Wide Matrices ($m < n$) -> Requires a "Right Inverse"

This is an underdetermined system (more columns than rows).

For a wide matrix to have a right inverse, it must have Full Row Rank.

- **Property**: The rank must equal m (the smaller dimension). All rows must be linearly independent
- **The Math**: The right pseudo-inverse is calculated as $A^{+}_{right} = A^T(AA^T)^{-1}$.
- **The Result**: Multiplying from the right yields the identity matrix: $AA^{+}_{right} = I_m$.
- **Engineering Meaning**: Because there are more variables than equations, there are infinite solutions. The right pseudo-inverse finds the specific solution that has the absolute smallest magnitude (minimum norm).

G:The Chapman-Kolmogorov Equation

In Chapter 6.1, we introduced the Chapman-Kolmogorov Equation as the mathematical engine of the Kalman Filter's Prediction Step. It calculates the Prior probability distribution ($p(\mathrm{x}_k \mid \mathrm{Z}_{k-1})$) by pushing the previous state forward in time.

But where does this equation come from? It is derived by combining two fundamental concepts from probability theory: **The Law of Total Probability** and the **Markov Property**.

1. The Law of Total Probability (Marginalization)

In statistics, if you want to find the probability of an event A, but that event depends on a series of mutually exclusive underlying events B, you can find the total probability of A by summing (or integrating) the joint probabilities of A and B across all possible states of B.

For continuous random variables, this is called marginalization:

$$p(A) = \int p(A, B) dB$$

By applying the definition of conditional probability ($p(A, B) = p(A \mid B)p(B)$), we can rewrite this as:

$$p(A) = \int p(A \mid B)p(B) dB$$

2. Applying this to Target Tracking

Let us map this to our tracking variables. We want to find the probability of the target's state today (x_k), given all the sensor measurements up to yesterday (Z_{k-1}).

To do this, we marginalize out "yesterday's exact state" (x_{k-1}) by integrating across every possible place the target could have been yesterday.

Substituting our tracking variables into the Law of Total Probability yields:

$$p(\mathrm{x}_k \mid \mathrm{Z}_{k-1}) = \int p(\mathrm{x}_k, \mathrm{x}_{k-1} \mid \mathrm{Z}_{k-1}) d\mathrm{x}_{k-1}$$

Using the conditional probability rule, we split the joint distribution:

$$p(\mathrm{x}_k \mid \mathrm{Z}_{k-1}) = \int p(\mathrm{x}_k \mid \mathrm{x}_{k-1}, \mathrm{Z}_{k-1}) p(\mathrm{x}_{k-1} \mid \mathrm{Z}_{k-1}) d\mathrm{x}_{k-1}$$

3. The Markov Assumption

Look at the first term inside the integral: $p(x_k \mid x_{k-1}, Z_{k-1})$. This asks, "Where is the target today, given exactly where it was yesterday AND all the historical sensor data?"

As defined in Chapter 4.4, kinematic tracking is a **Markov Process**. The target's state today depends *only* on where it was yesterday, and the physics guiding its movement. The historical sequence of camera images (Z_{k-1}) provides no additional predictive power if we already know yesterday's state x_{k-1}.

Therefore, we can drop Z_{k-1} from that specific term:

$$p(x_k \mid x_{k-1}, Z_{k-1}) = p(x_k \mid x_{k-1})$$

4. The Final Equation

Substituting the Markov simplification back into our integral leaves us with the exact, formal definition of the Chapman-Kolmogorov Equation:

$$p(x_k \mid Z_{k-1}) = \int p(x_k \mid x_{k-1}) p(x_{k-1} \mid Z_{k-1}) dx_{k-1}$$

The Engineering Meaning:

Mathematically, this equation is a continuous convolution. It takes the sharp, well-defined probability peak of where we thought the target was yesterday ($p(x_{k-1} \mid Z_{k-1})$) and "smears" it across space using the transition physics and process noise ($p(x_k \mid x_{k-1})$). The result is a wider, flatter curve representing our predicted uncertainty before the camera takes the next picture.

H:The Analytical Impossibility of Non-Linear Integration

In Chapter 6.2, we claimed that the recursive Bayesian integration problem is analytically unsolvable for non-linear systems. To truly appreciate why the Cubature Kalman Filter is a mathematical necessity rather than just an alternative, one must look at the calculus of what happens when Gaussians collide with real-world geometry.

1. The Requirement for Closed-Form Gaussian Integration

An integral has an analytical (closed-form) solution if we can write its answer using a finite number of standard mathematical operations (algebra, exponentials, logarithms, etc.).

In the Bayesian update step, we must calculate the Evidence by integrating the Likelihood multiplied by the Prior:

$$p(\mathrm{z}_k) = \int p(\mathrm{z}_k \mid \mathrm{x}_k) p(\mathrm{x}_k) d\mathrm{x}_k$$

Because we assume our noises are Gaussian, the terms inside this integral take the exponential form $\exp(\dots)$.

For an integral of the form $\int \exp(-f(x))dx$ to be analytically solvable over the range $[-\infty, \infty]$, the function inside the exponent, $f(x)$, **must be a quadratic polynomial** (e.g., $ax^2 + bx + c$). If the exponent takes a quadratic form, we can mathematically "complete the square" and perfectly solve the integral, resulting in the standard, linear Kalman Filter equations.

If $f(x)$ contains *any* non-linear geometry—such as trigonometric functions, roots, or divisions—the exponent is no longer quadratic. The integral becomes a transcendental function with no elementary antiderivative. It is mathematically unsolvable.

2. Concrete Example: The Non-Linear Reality of Bearing Angles

Let us look at a highly standard aerospace tracking scenario. A UAV is flying in a 2D plane (X, Y). An interception system is tracking it using a monocular camera that can only measure the visual bearing angle (θ) to the target.

The State Vector: $\mathrm{x} = [X, Y]^T$ **The Measurement:** $z = \theta$

The non-linear measurement geometry linking the state to the sensor is:

$$h(\mathrm{x}) = \arctan\left(\frac{Y}{X}\right)$$

Now, let us attempt to construct the Bayesian integral to find the Evidence denominator. We will ignore the scaling constants to focus purely on the calculus of the exponents.

The Prior (Predicted State): Assume we predict the target is at (μ_x, μ_y) with some spatial variance σ_x^2 and σ_y^2. The Prior PDF is:

$$p(\mathrm{x}) \propto \exp\left(-\frac{(X-\mu_x)^2}{2\sigma_x^2} - \frac{(Y-\mu_y)^2}{2\sigma_y^2}\right)$$

(*Note: This is a perfect quadratic. If we integrated this alone, it would be easily solvable*).

The Likelihood (Sensor Measurement): The camera measures an angle θ_{meas} with a sensor noise variance of σ_θ^2. The Likelihood PDF is evaluated by plugging our non-linear geometry into the Gaussian formula:

$$p(z \mid \mathrm{x}) \propto \exp\left(-\frac{\left(\theta_{meas} - \arctan\left(\frac{Y}{X}\right)\right)^2}{2\sigma_\theta^2}\right)$$

The Integration Roadblock: To find the Bayesian Evidence, we must multiply these two functions and integrate over X and Y:

$$p(z) \propto \iint \exp\left(-\frac{(X-\mu_x)^2}{2\sigma_x^2} - \frac{(Y-\mu_y)^2}{2\sigma_y^2} - \frac{\left(\theta_{meas} - \arctan\left(\frac{Y}{X}\right)\right)^2}{2\sigma_\theta^2}\right) dXdY$$

Look closely at the final term inside the exponent: $\left(\theta_{meas} - \arctan\left(\frac{Y}{X}\right)\right)^2$.

If you expand this term, you generate $\arctan^2\left(\frac{Y}{X}\right)$.

This is the mathematical dead end. There is no known calculus technique—no integration by parts, no u-substitution, no trigonometric identity—that can find the exact analytical integral of an exponential function containing a squared arctangent of a fraction.

3. The Estimation Solution

Because the exact math fails, tracking engineers must approximate.

- The **Extended Kalman Filter (EKF)** attempts to solve this by calculating the Taylor Series derivative (Jacobian) of $\arctan(Y/X)$ and forcibly pretending the geometry is a straight line. If the UAV's spatial uncertainty is large, this forced flattening drastically distorts the math, and the filter fails.
- The **Cubature Kalman Filter (CKF)** accepts that the integral cannot be solved via calculus. Instead, it utilizes spherical-radial integration theory. It strategically selects $2n$ discrete physical coordinate points, runs those exact numbers through the true, uncorrupted $\arctan(Y/X)$ function, and takes the weighted average of the results. By replacing an impossible continuous integral with a finite set of discrete deterministic evaluations, the CKF achieves near-optimal estimation without ever needing to calculate a derivative.

I:Matrix Derivation of the Kalman Filter

This appendix provides the rigorous matrix calculus to derive the multidimensional Kalman Gain from the Minimum Mean Square Error (MMSE) cost function.

1. **Defining the Error Covariance**

Let our true state be x. After fusing a measurement, our updated estimate is $\hat{x}_{k|k}$. The estimation error is defined as:

$$\tilde{x} = x - \hat{x}_{k|k}$$

The Covariance Matrix of this error ($P_{k|k}$) is the expected value of the outer product of the error:

$$P_{k|k} = E[\tilde{x}\tilde{x}^T]$$

We know from our filter structure that the update equation is:

$$\hat{x}_{k|k} = \hat{x}_{k|k-1} + K(z_k - H\hat{x}_{k|k-1})$$

Substitute the true measurement model ($z_k = Hx + v$) into the update equation, and then substitute that result into the error equation $\tilde{x}$. After expanding the expectation and recognizing that the prediction error and the measurement noise v are entirely uncorrelated ($E[v\tilde{x}^T] = 0$), the updated covariance expands to:

$$P_{k|k} = (I - KH)P_{k|k-1}(I - KH)^T + KRK^T$$

This is the exact **Joseph Form** of the covariance update equation.

2. **Minimizing the Trace (The MMSE Cost Function)**

The trace of a matrix (the sum of its diagonal elements) represents the total variance of the system. To find the MMSE, we must find the specific Kalman Gain matrix (K) that minimizes the trace of $P_{k|k}$.

Cost Function: $J = \mathrm{Tr}(P_{k|k})$

We expand the Joseph form:

$$\mathrm{Tr}(P_{k|k}) = \mathrm{Tr}(P_{k|k-1} - KHP_{k|k-1} - P_{k|k-1}H^TK^T + K(HP_{k|k-1}H^T + R)K^T)$$

3. **Matrix Calculus**

To find the minimum, we take the partial derivative of the trace with respect to the matrix K and set it to zero:

$$\frac{\partial \text{Tr}(P_{k|k})}{\partial K} = 0$$

Using standard matrix derivative identities ($\frac{\partial \text{Tr}(AB^T)}{\partial A} = B$ and $\frac{\partial \text{Tr}(ACA^T)}{\partial A} = 2AC$ if C is symmetric), we differentiate the expanded equation:

$$\frac{\partial \text{Tr}(P_{k|k})}{\partial K} = -2(P_{k|k-1}H^T)^T + 2K(HP_{k|k-1}H^T + R) = 0$$

4. **Solving for K**

Divide by 2 and isolate K:

$$K(HP_{k|k-1}H^T + R) = P_{k|k-1}H^T$$

The term inside the parentheses is the Innovation Covariance, defined as $S = HP_{k|k-1}H^T + R$. We substitute S and multiply both sides by S^{-1}:

$$KSS^{-1} = P_{k|k-1}H^TS^{-1}$$
$$K = P_{k|k-1}H^T(HP_{k|k-1}H^T + R)^{-1}$$

This mathematically proves that this specific formulation of K guarantees the absolute minimum possible mean squared error for a linear system.

J:Adaptive Covariance Matching

When a filter diverges due to unmodeled target maneuvers, the static Process Noise matrix (Q) is no longer sufficient. This appendix details the covariance matching technique (originally established by Myers and Tapley) used to dynamically estimate Q on the fly using a sliding window of recent innovations.

1. The Statistical Basis of the Innovation

Recall the fundamental definition of the Innovation vector y_k:

$$\mathrm{y}_k = \mathrm{z}_k - H\hat{\mathrm{x}}_{k|k-1}$$

The theoretical covariance of the innovation is defined as:

$$E[\mathrm{y}_k \mathrm{y}_k^T] = S_k = HP_{k|k-1}H^T + R$$

By expanding the predicted covariance $P_{k|k-1} = FP_{k-1|k-1}F^T + Q$, we can explicitly expose the Process Noise Q inside the theoretical innovation equation:

$$S_k = H(FP_{k-1|k-1}F^T + Q)H^T + R$$

2. The Sliding Window Approximation

The equation above tells us what the variance should be. To estimate what the variance actually is, we calculate the sample covariance of the real innovations over a sliding historical window of the last N time steps:

$$\hat{C}_{\mathrm{y}_k} = \frac{1}{N} \sum_{j=k-N+1}^{k} \mathrm{y}_j \mathrm{y}_j^T$$

The core philosophy of Covariance Matching is to force the theoretical covariance (S_k) to equal the actual observed sample covariance ($\hat{C}_{\mathrm{y}_k}$):

$$\hat{C}_{\mathrm{y}_k} \approx H(FP_{k-1|k-1}F^T + Q_k)H^T + R$$

3. Solving for Dynamic Process Noise (Q_k)

We must now isolate the dynamic Q_k term. Distribute the H and H^T matrices:

$$\hat{C}_{y_k} = H(FP_{k-1|k-1}F^T)H^T + HQ_kH^T + R$$

Isolate HQ_kH^T:

$$HQ_kH^T = \hat{C}_{y_k} - H(FP_{k-1|k-1}F^T)H^T - R$$

To solve for Q_k, we must strip away the H matrices. Assuming H is a square and invertible matrix, we multiply both sides by H^{-1} on the left and $(H^T)^{-1}$ on the right:

$$\hat{Q}_k = H^{-1}\left(\hat{C}_{y_k} - R\right)(H^T)^{-1} - FP_{k-1|k-1}F^T$$

The Engineering Constraint: In physical tracking systems, H is rarely square (it maps a large state to a smaller measurement vector), meaning H^{-1} does not exist. In these cases, engineers use the Left Pseudo-Inverse (as defined in Appendix F) to approximate the solution: $H^+ = (H^TH)^{-1}H^T$.

$$\hat{Q}_k \approx H^+\left(\hat{C}_{y_k} - R\right)(H^+)^T - FP_{k-1|k-1}F^T$$

Because this raw calculation can sometimes produce negative diagonal values due to statistical noise in the sliding window, a production implementation will immediately zero out any negative eigenvalues, or run a $\max(0, \text{val})$ check on the diagonals of $\hat{Q}_k$ to ensure it remains positive-definite before injecting it back into the Kalman Filter prediction loop.

K: EKF Derivations: Matrices and Jacobians

This appendix provides the rigorous mathematical derivations necessary to implement the Extended Kalman Filter, covering both the discretization of continuous non-linear models and the calculus required for Polar-to-Cartesian radar tracking.

Continuous to Discrete EKF State Transition

If the physical system is defined by a continuous-time non-linear differential equation $\dot{x} = f_c(x, u)$, we cannot directly apply the discrete EKF formulas. We must first linearize the continuous system, and then discretize it.

Step A: Continuous Linearization

We calculate the continuous-time Jacobian matrix $A(t)$ by taking the partial derivatives of the continuous physics model evaluated at the current estimate:

$$A(t) = \left. \frac{\partial f_c(x, u)}{\partial x} \right|_{x=\hat{x}}$$

Step B: Discretization

Once we have the linear $A(t)$ matrix, we can discretize it over the time step Δt using the matrix exponential:

$$F_k = e^{A(t)\Delta t}$$

For embedded systems where calculating a matrix exponential is too CPU-intensive, engineers frequently use the **First-Order Euler Approximation**:

$$F_k \approx I + A(t)\Delta t$$

Where I is the identity matrix. This is the standard transition Jacobian used in most real-time aerospace EKFs.

Derivation of the Radar Measurement Jacobian

In Chapter 9.2, we introduced the 2×4 Measurement Jacobian (H_k) for a 2D radar mapping Cartesian state $[x, y, v_x, v_y]^T$ to Polar measurements $[r, \theta]^T$.

The exact non-linear mapping $h(x)$ is:

1. $r = \sqrt{x^2 + y^2}$

2. $\theta = \arctan(y/x)$

The Jacobian matrix requires calculating the partial derivative of each measurement equation with respect to each state variable:

$$H_k = \begin{bmatrix} \frac{\partial r}{\partial x} & \frac{\partial r}{\partial y} & \frac{\partial r}{\partial v_x} & \frac{\partial r}{\partial v_y} \\ \frac{\partial \theta}{\partial x} & \frac{\partial \theta}{\partial y} & \frac{\partial \theta}{\partial v_x} & \frac{\partial \theta}{\partial v_y} \end{bmatrix}$$

Because the radar only measures position and not velocity, the partial derivatives with respect to v_x and v_y are strictly zero. We only need to calculate the position derivatives.

1. Range Derivatives (Chain Rule):

$$\frac{\partial r}{\partial x} = \frac{\partial}{\partial x}(x^2 + y^2)^{1/2} = \frac{1}{2}(x^2 + y^2)^{-1/2}(2x) = \frac{x}{\sqrt{x^2 + y^2}} = \frac{x}{r}$$

$$\frac{\partial r}{\partial y} = \frac{\partial}{\partial y}(x^2 + y^2)^{1/2} = \frac{y}{\sqrt{x^2 + y^2}} = \frac{y}{r}$$

2. Bearing Derivatives (Quotient Rule):

Recall the standard derivative of arctangent: $\frac{d}{du}\arctan(u) = \frac{1}{1+u^2} \cdot u'$.

$$\frac{\partial \theta}{\partial x} = \frac{1}{1 + (y/x)^2} \cdot \left(-\frac{y}{x^2}\right) = \frac{x^2}{x^2 + y^2} \cdot \left(-\frac{y}{x^2}\right) = \frac{-y}{x^2 + y^2} = \frac{-y}{r^2}$$

$$\frac{\partial \theta}{\partial y} = \frac{1}{1 + (y/x)^2} \cdot \left(\frac{1}{x}\right) = \frac{x^2}{x^2 + y^2} \cdot \left(\frac{1}{x}\right) = \frac{x}{x^2 + y^2} = \frac{x}{r^2}$$

3. The Final Jacobian Matrix:

Substituting these analytical derivatives back into the matrix yields the exact H_k used in the C++ implementation:

$$H_k = \begin{bmatrix} \frac{x}{r} & \frac{y}{r} & 0 & 0 \\ \frac{-y}{r^2} & \frac{x}{r^2} & 0 & 0 \end{bmatrix}$$

L:Taylor Series Verification of the Unscented Transform

In Chapter 10, we stated that the Unscented Transform (UT) captures the true mean and covariance of a non-linear Gaussian distribution better than the Extended Kalman Filter (EKF). This appendix provides the Taylor Series expansion proof demonstrating that the EKF truncates at the 1st order, while the UT successfully reconstructs the true distribution up to the 3rd order.

1. **The True Non-Linear Expectation**

Let x be a random variable with mean $\bar{\mathrm{x}}$ and covariance P_{xx}. Let $\delta \mathrm{x} = \mathrm{x} - \bar{\mathrm{x}}$ be a zero-mean Gaussian perturbation with covariance P_{xx}.

We pass x through a non-linear function: $\mathrm{y} = g(\mathrm{x})$.

To find the true expected mean of y, we expand $g(\mathrm{x})$ using a multidimensional Taylor Series around $\bar{\mathrm{x}}$:

$$g(\mathrm{x}) = g(\bar{\mathrm{x}} + \delta \mathrm{x}) = g(\bar{\mathrm{x}}) + \nabla g \delta \mathrm{x} + \frac{1}{2}\nabla^2 g \delta \mathrm{x}^2 + \frac{1}{6}\nabla^3 g \delta \mathrm{x}^3 + \ldots$$

Taking the expectation $E[\cdot]$ of both sides:

$$\bar{\mathrm{y}}_{true} = E[g(\mathrm{x})] = g(\bar{\mathrm{x}}) + E[\nabla g \delta \mathrm{x}] + \frac{1}{2}E[\nabla^2 g \delta \mathrm{x}^2] + \frac{1}{6}E[\nabla^3 g \delta \mathrm{x}^3] + \ldots$$

Because δx is a zero-mean, symmetric Gaussian:

All odd-order moments are zero: $E[\delta \mathrm{x}] = 0$, $E[\delta \mathrm{x}^3] = 0$.

The second-order moment evaluates exactly to the covariance P_{xx}.

Therefore, the exact, true expected mean is:

$$\bar{\mathrm{y}}_{true} = g(\bar{\mathrm{x}}) + \frac{1}{2}\nabla^2 g P_{xx} + \ldots \text{(higher even orders)}$$

2. **The EKF Approximation**

The EKF approximates the mean by simply passing the prior mean through the function:

$$\bar{\mathrm{y}}_{EKF} = g(\bar{\mathrm{x}})$$

The Truncation Error: Comparing the EKF to the true mean, we see the EKF completely ignores the $\frac{1}{2}\nabla^2 g P_{xx}$ term. If the function is highly curved (large 2nd derivative/Hessian) or the uncertainty is large (large P_{xx}), the EKF mean is violently wrong.

3. **The Unscented Transform Approximation**

The UT approximates the mean by taking a weighted sum of the deterministically generated Sigma Points ($\mathcal{X}_i$):

$$\bar{y}_{UT} = \sum_{i=0}^{2n} W_i g(\mathcal{X}_i)$$

Let us substitute our Sigma Point generation formula ($\mathcal{X}_i = \bar{x} \pm \sqrt{(n+\lambda)P_{xx}}$) into the Taylor series expansion. The perturbation for the i-th point is $\delta x_i = \pm\sqrt{(n+\lambda)P_{xx}}$.

Expanding the weighted sum:

$$\bar{y}_{UT} = W_0 g(\bar{x}) + \sum_{i=1}^{2n} W_i \left(g(\bar{x}) + \nabla g \delta x_i + \frac{1}{2}\nabla^2 g \delta x_i^2 + \dots \right)$$

Because the weights sum to 1 ($\sum W_i = 1$) and the sigma points are perfectly symmetric (for every $+\delta x_i$ there is a $-\delta x_i$), the first-order gradient terms sum perfectly to zero.

$$\bar{y}_{UT} = g(\bar{x}) + \frac{1}{2}\sum_{i=1}^{2n} W_i \nabla^2 g \delta x_i^2 + \dots$$

Substitute the exact values of the UT weights ($W_i = \frac{1}{2(n+\lambda)}$) and the squared perturbations ($\delta x_i^2 = (n+\lambda)P_{xx}$):

$$\bar{y}_{UT} = g(\bar{x}) + \frac{1}{2}\sum_{i=1}^{2n} \frac{1}{2(n+\lambda)} \nabla^2 g \left[(n+\lambda)P_{xx}\right]_i$$

The scaling term $(n+\lambda)$ cancels out perfectly. Summing over the $2n$ symmetrical points perfectly reconstructs the covariance matrix P_{xx}:

$$\bar{y}_{UT} = g(\bar{x}) + \frac{1}{2}\nabla^2 g P_{xx} + \dots$$

Conclusion: By expanding the mathematics, we have proven that the UKF Sigma Point summation perfectly captures the $\frac{1}{2}\nabla^2 g P_{xx}$ term that the EKF ignores. The UT mathematically guarantees accuracy up to the 3rd-order moment of the probability distribution, drastically reducing linearization error during highly dynamic target maneuvers.

M:Proof of CKF Numerical Stability and Positive Definiteness

In Chapter 11.3, we asserted that the Cubature Kalman Filter (CKF) is mathematically immune to the covariance collapse that plagues the Unscented Kalman Filter (UKF). This appendix provides the linear algebra proof demonstrating why the CKF guarantees a positive-definite covariance matrix.

1. **Definition of Positive Definiteness**

A covariance matrix P is positive-definite ($P \succ 0$) if and only if, for any non-zero vector v:

$$\mathrm{v}^T P \mathrm{v} > 0$$

During the Prediction step, the updated covariance matrix $P_{k|k-1}$ is calculated as:

$$P_{k|k-1} = \sum_{i=1}^{2n} W_i (\mathcal{X}_i - \hat{\mathrm{x}})(\mathcal{X}_i - \hat{\mathrm{x}})^T + Q$$

Let $\mathrm{e}_i = \mathcal{X}_i - \hat{\mathrm{x}}$ represent the deviation of the i-th propagated cubature point from the mean. The equation simplifies to:

$$P_{k|k-1} = \sum_{i=1}^{2n} W_i (\mathrm{e}_i \mathrm{e}_i^T) + Q$$

2. **Evaluating the Quadratic Form**

To test for positive definiteness, we pre- and post-multiply the equation by our arbitrary non-zero vector v:

$$\mathrm{v}^T P_{k|k-1} \mathrm{v} = \mathrm{v}^T \left(\sum_{i=1}^{2n} W_i \mathrm{e}_i \mathrm{e}_i^T \right) \mathrm{v} + \mathrm{v}^T Q \mathrm{v}$$

Because scalar matrix multiplication is commutative, we can pull v inside the summation:

$$\mathrm{v}^T P_{k|k-1} \mathrm{v} = \sum_{i=1}^{2n} W_i (\mathrm{v}^T \mathrm{e}_i)(\mathrm{e}_i^T \mathrm{v}) + \mathrm{v}^T Q \mathrm{v}$$

Notice that $(\mathrm{v}^T \mathrm{e}_i)$ is a scalar (the dot product of two vectors). Furthermore, $(\mathrm{e}_i^T \mathrm{v})$ is the exact same scalar. Let $c_i = \mathrm{v}^T \mathrm{e}_i$. The equation becomes:

$$\mathrm{v}^T P_{k|k-1}\mathrm{v} = \sum_{i=1}^{2n} W_i c_i^2 + \mathrm{v}^T Q\mathrm{v}$$

3. **The Weight Constraint**

Because c_i is a real number, its square is always positive or zero ($c_i^2 \geq 0$).

This is the exact point of failure for the UKF. In the UKF, the central weight W_0 can be negative. If $W_0 c_0^2$ yields a large negative number, the entire sum can drop below zero, destroying positive definiteness.

In the CKF, the spherical-radial cubature rule mandates that all weights are identical and strictly positive: $W_i = \frac{1}{2n} > 0$. Therefore, multiplying the strictly positive weight by the positive squared scalar guarantees that every single term in the summation is ≥ 0:

$$\sum_{i=1}^{2n} \left(\frac{1}{2n}\right) c_i^2 \geq 0$$

Finally, we consider the Process Noise matrix, Q. By definition, process noise represents physical environmental uncertainty and is always designed to be strictly positive-definite ($\mathrm{v}^T Q\mathrm{v} > 0$).

Adding a strictly positive number ($\mathrm{v}^T Q\mathrm{v}$) to a positive semi-definite sum yields a strictly positive result:

$$\mathrm{v}^T P_{k|k-1}\mathrm{v} > 0$$

N:Particle Filter Degeneracy and Effective Sample Size

In Chapter 12.1, we introduced the Particle Filter (PF) and stated that without a Resampling step, the filter will inevitably collapse due to "Weight Degeneracy." This appendix provides the mathematical proof of why degeneracy occurs and derives the Effective Sample Size (N_{eff}) equation used by engineers to trigger the resampling algorithm.

1. **The Mathematics of Weight Degeneracy**

In a Particle Filter, the posterior probability density is approximated by a set of N particles $\{x_k^{(i)}\}_{i=1}^N$ and their associated normalized weights $\{w_k^{(i)}\}_{i=1}^N$.

Using Sequential Importance Sampling (SIS), the unnormalized weight of particle i at time step k is updated recursively using Bayes' theorem:

$$w_k^{(i)} \propto w_{k-1}^{(i)} \frac{p(z_k|x_k^{(i)})p(x_k^{(i)}|x_{k-1}^{(i)})}{q(x_k^{(i)}|x_{k-1}^{(i)}, z_k)}$$

Where $p(z_k|x_k^{(i)})$ is the observation likelihood, and $q(\cdot)$ is the proposal distribution (often simply chosen as the transition prior $p(x_k|x_{k-1})$).

The variance of these weights over time is proven to strictly increase. Mathematically, it can be shown that the variance of the true importance weights conditional on the measurement sequence always increases stochastically:

$$Var(w_k) \geq Var(w_{k-1})$$

The Engineering Consequence: Because the variance of the weights strictly increases, after a few recursive iterations, the probability mass will become concentrated on a single particle. For N particles, $N-1$ particles will have a weight of $w^{(i)} \approx 0$, and exactly one particle will have a weight of $w^{(i)} \approx 1$.

At this point, the filter is wasting 99.9% of its CPU power simulating particles that contribute absolutely nothing to the state estimate. The filter has degenerated.

2. **Measuring Degeneracy: Effective Sample Size**

To prevent the CPU from wasting cycles, the filter must monitor the health of its particle swarm. We cannot measure the true variance of the theoretical weights, so we estimate the **Effective Sample Size**, denoted as N_{eff}.

N_{eff} intuitively represents the number of particles that are actually contributing meaningfully to the probability distribution.

The true effective sample size is defined as:

$$N_{eff} = \frac{N}{1 + Var(w_k^*)}$$

(Where w_k are the true, unnormalized weights).*

Because the true weights are unknown, we approximate N_{eff} using the normalized weights we calculate in software ($\sum w_k^{(i)} = 1$). The approximation is derived as:

$$\widehat{N}_{eff} = \frac{1}{\sum_{i=1}^{N} (w_k^{(i)})^2}$$

3. **Analyzing the N_{eff} Equation**

We can mathematically prove why this specific equation perfectly captures the health of the filter by looking at the two extremes:

Case A: Perfect Health (All particles are equally likely) If the filter is perfectly healthy, every particle has the exact same weight: $w^{(i)} = 1/N$.

$$\widehat{N}_{eff} = \frac{1}{\sum_{i=1}^{N} (1/N)^2} = \frac{1}{N \cdot (1/N^2)} = \frac{1}{1/N} = N$$

The effective sample size equals the total number of particles.

Case B: Total Degeneracy (One particle holds all the weight) If the filter has degenerated, one particle has a weight of 1.0, and the remaining $N - 1$ particles have a weight of 0.0.

$$\widehat{N}_{eff} = \frac{1}{1^2 + 0^2 + \cdots + 0^2} = \frac{1}{1} = 1$$

The effective sample size equals 1, regardless of how many thousands of particles are currently sitting in memory.

4. **The Resampling Trigger**

In production $C++$ code, evaluating the Effective Sample Size is a one-line summation loop. Engineers define a strict threshold—typically $N_{eff} < N/2$.

Once $\widehat{N}_{eff}$ drops below 50% of the total particle count, the software halts the tracking loop and triggers the Systematic Resampling algorithm (provided in Section 12.1), mathematically reviving the dead particles and preventing filter collapse.

O: Moment Matching Reduction of Gaussian Mixtures

In Chapter 14.2, we established that real-time IMM execution relies on collapsing a Gaussian sum of r^2 branches down to r branches at the conclusion of every mixing cycle. This appendix provides the formal statistical proof demonstrating that collapsing a Gaussian mixture by matching its first two moments (Mean and Covariance) perfectly preserves the true mean and physical variance of the uncollapsed probability distribution.

1. **Defining the Gaussian Mixture**

Let $p(\mathrm{x})$ be a continuous probability density function represented exactly by a mixture of N distinct Gaussian components:

$$p(\mathrm{x}) = \sum_{i=1}^{N} w_i \mathcal{N}(\mathrm{x}; \mu_i, P_i)$$

Where the scalar weighting coefficients sum to unity: $\sum_{i=1}^{N} w_i = 1$.

We seek to approximate this complex multi-modal distribution $p(\mathrm{x})$ with a single, uncollapsed global Gaussian distribution $q(\mathrm{x}) = \mathcal{N}(\mathrm{x}; \mu_m, P_m)$.

2. **Matching the First Moment (The Mean)**

By the definition of the expected value, the true global mean (μ_m) of the composite mixture is:

$$\mu_m = E[\mathrm{x}] = \int_{-\infty}^{\infty} \mathrm{x} p(\mathrm{x}) d\mathrm{x}$$

Substitute the Gaussian mixture definition into the integral:

$$\mu_m = \int_{-\infty}^{\infty} \mathrm{x} \left(\sum_{i=1}^{N} w_i \mathcal{N}(\mathrm{x}; \mu_i, P_i) \right) d\mathrm{x}$$

Because the integral of a finite sum is the sum of the integrals, we distribute the integration operator:

$$\mu_m = \sum_{i=1}^{N} w_i \left(\int_{-\infty}^{\infty} \mathrm{x} \mathcal{N}(\mathrm{x}; \mu_i, P_i) d\mathrm{x} \right)$$

Look at the inner integral: $\int \mathrm{x}\mathcal{N}(\mathrm{x};\mu_i,P_i)d\mathrm{x}$. This is the exact fundamental definition of the mean of the individual i-th Gaussian component, which evaluates identically to μ_i.

Therefore, the first moment collapses to the exact weighted sum of the individual component means:

$$\mu_m = \sum_{i=1}^{N} w_i \mu_i$$

3. **Matching the Second Central Moment (The Covariance)**

By the definition of the covariance matrix, the true global covariance (P_m) of the composite mixture is:

$$P_m = E\left[(\mathrm{x}-\mu_m)(\mathrm{x}-\mu_m)^T\right] = \int_{-\infty}^{\infty}(\mathrm{x}-\mu_m)(\mathrm{x}-\mu_m)^T p(\mathrm{x})d\mathrm{x}$$

Substitute the Gaussian mixture definition into the integral:

$$P_m = \sum_{i=1}^{N} w_i \int_{-\infty}^{\infty}(\mathrm{x}-\mu_m)(\mathrm{x}-\mu_m)^T \mathcal{N}(\mathrm{x};\mu_i,P_i)d\mathrm{x}$$

To evaluate the inner integral, we must re-center the quadratic term around the individual component mean μ_i rather than the global mean μ_m. We execute the algebraic addition of zero $(-\mu_i+\mu_i)$:

$$\mathrm{x}-\mu_m = (\mathrm{x}-\mu_i)+(\mu_i-\mu_m)$$

Let the mean deviation vector be defined as $\mathrm{d}_i = \mu_i - \mu_m$. Substitute this expansion back into the outer product:

$$(\mathrm{x}-\mu_m)(\mathrm{x}-\mu_m)^T = \left[(\mathrm{x}-\mu_i)+\mathrm{d}_i\right]\left[(\mathrm{x}-\mu_i)+\mathrm{d}_i\right]^T$$

$$= (\mathrm{x}-\mu_i)(\mathrm{x}-\mu_i)^T + (\mathrm{x}-\mu_i)\mathrm{d}_i^T + \mathrm{d}_i(\mathrm{x}-\mu_i)^T + \mathrm{d}_i\mathrm{d}_i^T$$

We now integrate these four distinct terms individually against the component Gaussian density $\mathcal{N}(\mathrm{x};\mu_i,P_i)$:

1. **The First Term:** $\int(\mathrm{x}-\mu_i)(\mathrm{x}-\mu_i)^T\mathcal{N}_i d\mathrm{x}$ is the exact theoretical definition of component covariance, evaluating to P_i.
2. **The Second Term:** $\left[\int(\mathrm{x}-\mu_i)\mathcal{N}_i d\mathrm{x}\right]\mathrm{d}_i^T$. Because x is centered at μ_i, the expected value of its own deviation is strictly zero. The term vanishes $(=0)$.g
3. **The Third Term:** Vanishes to zero by the exact same symmetry property $(=0)$.
4. **The Fourth Term:** $\mathrm{d}_i\mathrm{d}_i^T\int\mathcal{N}_i d\mathrm{x}$. Because a probability density integrates to unity $(\int\mathcal{N}_i = 1)$, this evaluates strictly to the outer product $\mathrm{d}_i\mathrm{d}_i^T$.

Summing the unvanished terms across all N weighted components yields the exact, definitive IMM covariance combination formula:

$$P_m = \sum_{i=1}^{N} w_i\left[P_i + (\mu_i-\mu_m)(\mu_i-\mu_m)^T\right]$$

Conclusion: We have mathematically proven that approximating a complex Gaussian mixture by collapsing it to a single mean and covariance does not discard physical uncertainty. The inclusion of the spread-of-the-means outer product $(\mu_i - \mu_m)(\dots)^T$ perfectly captures the between-model variance, guaranteeing that the IMM estimator remains mathematically bounded and structurally stable across highly volatile target maneuvers.

P: Proof of Acceleration Surrogate via Process Noise Inflation

In Chapter 15, we established that deploying an 8-state Constant Velocity (CV) model with a massively inflated Process Noise Covariance matrix (Q) acts as a computationally efficient surrogate for an 11-state Constant Acceleration (CA) model. This appendix provides the mathematical proof demonstrating that inflating Q drives the Kalman Gain to effectively bypass the linear kinematic memory, snapping the state directly to the raw measurement.

1. **The Process Noise Formulation**

Let the discrete-time state transition equation for the 8-state target be defined as:

$$x_k = Fx_{k-1} + \Gamma a_k$$

where F is the linear Constant Velocity transition matrix, a_k is the unknown true target acceleration vector (which we are not explicitly tracking), and Γ is the input matrix mapping acceleration into the position and velocity states over time step Δt. For a single spatial axis, the input mapping is:

$$\Gamma_x = \begin{bmatrix} \frac{1}{2}\Delta t^2 \\ \Delta t \end{bmatrix}$$

Because our 8-state model explicitly assumes zero deterministic acceleration, the term Γa_k is treated entirely as zero-mean Gaussian process noise $w_k \sim \mathcal{N}(0, Q)$.

The theoretical Process Noise Covariance matrix is computed as the expected value of this noise formulation:

$$Q = E[w_k w_k^T] = \Gamma E[a_k a_k^T] \Gamma^T$$

Assuming the unknown high-agility maneuver is an uncorrelated zero-mean white noise acceleration process with a defined variance of σ_a^2, the block diagonal sub-matrix of Q for a given spatial dimension evaluates to:

$$Q_{spatial} = \begin{bmatrix} \frac{1}{4}\Delta t^4 & \frac{1}{2}\Delta t^3 \\ \frac{1}{2}\Delta t^3 & \Delta t^2 \end{bmatrix} \sigma_a^2$$

2. **The Kalman Gain Limit Proof**

In the standard Airborne CV model, σ_a^2 is tuned to a minimal value representing minor atmospheric drift. The filter relies heavily on its internal velocity memory.

In the "High-Agility Maneuvering" IMM branch, we artificially inflate σ_a^2 by several orders of magnitude (e.g., modeling potential 10G accelerations). This massive scalar inflation directly impacts the a priori State Covariance Prediction ($P_{k|k-1}$):

$$P_{k|k-1} = FP_{k-1|k-1}F^T + Q$$

The Kalman Gain (K_k) mathematically arbitrates how much the filter trusts the incoming visual measurement (z_k) versus its own internal kinematic prediction ($\hat{x}_{k|k-1}$). For explanatory purposes, assuming a linearized measurement matrix H and visual measurement noise R, the gain is:

$$K_k = P_{k|k-1}H^T(HP_{k|k-1}H^T + R)^{-1}$$

As the target executes a violent maneuver, the IMM shifts weight to the Maneuvering model, introducing the massively inflated Q matrix. This causes $P_{k|k-1}$ to grow exponentially large relative to the sensor noise R. We take the mathematical limit of the Kalman Gain as the predicted uncertainty P approaches infinity:

$$\lim_{P\to\infty} PH^T(HPH^T + R)^{-1} = H^{-1}$$

Deep Dive: The Woodbury Matrix Identity Limit

To rigorously prove this algebraic limit without violating matrix inversion rules, we apply the Woodbury Matrix Identity to the Kalman Gain formulation:

$$K_k = (P_{k|k-1}^{-1} + H^TR^{-1}H)^{-1}H^TR^{-1}$$

As the process noise inflates toward infinity ($P \to \infty$), the inverse covariance approaches the zero matrix ($P^{-1} \to 0$). Substituting this yields:

$$K_k \approx (0 + H^TR^{-1}H)^{-1}H^TR^{-1} = (H^TR^{-1}H)^{-1}H^TR^{-1}$$

Expanding the inverse yields $H^{-1}RH^{-T}H^TR^{-1}$, which perfectly cancels out the measurement noise R, leaving exactly H^{-1}.

3. **Evaluating the Update Equation**

When $K_k \approx H^{-1}$, we evaluate the final state update equation:

$$x_k = \hat{x}_{k|k-1} + K_k(z_k - H\hat{x}_{k|k-1})$$

Substituting the limit of the Kalman Gain:

$$x_k \approx \hat{x}_{k|k-1} + H^{-1}(z_k - H\hat{x}_{k|k-1})$$

$$x_k \approx \hat{x}_{k|k-1} + H^{-1}z_k - \hat{x}_{k|k-1}$$

$$x_k \approx H^{-1}z_k$$

Conclusion: The mathematical limit proves that inflating the process noise variance σ_a^2 effectively disables the filter's Constant Velocity memory. By driving the Kalman Gain toward H^{-1}, the Maneuvering model forces the final state vector (x_k) to snap directly to the raw, un-smoothed visual measurement (z_k). This allows the tracker to instantly follow a target through a high-G turn without executing the 40% computational overhead required to explicitly track a full 11-state spatial acceleration vector.

Q:Derivation of the Thermal Blooming Coefficient (β_{th})

In Chapter 15, we stated that when a tracking payload toggles from a Visible CMOS sensor to an Infrared Uncooled Vanadium Oxide (VOx) microbolometer, the Measurement Covariance (R_k) must be dynamically inflated by a scalar (α). This scalar incorporates the resolution disparity and the Thermal Blooming Coefficient ((β_{th})). This appendix mathematically models the physical heat smear of the sensor to derive β_{th}.

1. **Parameter Definitions**

- v_t: Target transverse velocity relative to the observer (m/s)
- d: Slant range to the target (m)
- f: IR lens physical focal length (m)
- p: Sensor pixel pitch (m/pixel, e.g., $12\mu m$)
- τ: Microbolometer thermal time constant (s, typically 8-12ms)
- W_{true}: True optical width of the target on the sensor plane (pixels)

2. **Kinematic to Optical Mapping (Pixel Velocity)**

We first determine the angular velocity of the target relative to the camera, $\omega = v_t/d$. This is multiplied by the sensor's pixel density mapping (f/p) to calculate the target's transit speed across the focal plane array in pixels per second, v_{px}:

$$v_{px} = \left(\frac{v_t}{d}\right)\left(\frac{f}{p}\right)$$

3. **Thermal Smear Calculation**

Unlike CMOS sensors, microbolometers absorb physical heat. When a heat signature moves away from a pixel, that pixel requires time to cool back to ambient temperature, governed by τ. The number of additional pixels illuminated by the trailing heat signature before the sensor completes one cooling cycle is defined as the thermal smear (S_{mear}):

$$S_{mear} = v_{px} \cdot \tau$$

Deep Dive: Microbolometer Thermal Physics

The thermal time constant τ represents the time required for a microbolometer pixel to dissipate 63.2% of its absorbed heat energy after the thermal stimulus (the target) moves away. It is derived from Newton's Law of Cooling, where $\tau = C/G$ (Heat Capacity C divided by Thermal Conductance G). For defense-grade Vanadium Oxide (VOx) sensors, balancing a low τ (to prevent fast-moving smear) against a high sensitivity (NETD) is the primary hardware trade-off that dictates the tracking software's R_k inflation threshold.

4. **Bounding Box Inflation and Variance Scaling**

The neural network (e.g., YOLO) perceives the target and the trailing smear as a single contiguous hot mass, generating a bloomed bounding box width (W_{bloom}):

$$W_{bloom} = W_{true} + S_{mear}$$

Because the measurement noise covariance matrix (R_k) in the Cubature Kalman Filter represents spatial variance (the square of standard deviation), the measurement uncertainty induced by thermal inertia scales with the square of the bounding box distortion ratio. Thus, the Thermal Blooming Coefficient is derived as:

$$\beta_{th} \approx \left(\frac{W_{bloom}}{W_{true}} \right)^2$$

5. **Empirical Application**

Consider an interception scenario: A drone traveling transversely at 50 m/s at a distance of 1000 m. The system utilizes an IR payload with a 50 mm (0.05 m) focal length, a $12\mu m$ (12×10^{-6} m) pixel pitch, and a standard VOx thermal time constant of 10 ms (0.01 s). The target's true optical width is 10 pixels.

Evaluating the pixel velocity:

$$v_{px} = \left(\frac{50}{1000} \right) \left(\frac{0.05}{12 \times 10^{-6}} \right) \approx 208.33 \text{ pixels/s}$$

Evaluating the thermal smear:

$$S_{mear} = 208.33 \cdot 0.01 \approx 2.08 \text{ pixels}$$

Evaluating the blooming coefficient:

$$\beta_{th} \approx \left(\frac{10 + 2.08}{10} \right)^2 \approx 1.46$$

If the system simultaneously downgrades from a 1080p visible sensor to a 480p IR sensor (a spatial quantization penalty ratio of roughly 6.0), the total dynamic covariance scalar evaluates to $\alpha = 6.0 \cdot 1.46 = 8.76$. This mathematical derivation perfectly aligns with the empirically calibrated operational range, proving the requirement for discrete R_k matrix inflation during multispectral transitions.

Bibliography

Articles

[1] Arasaratnam, I., & Haykin, S. (2009). *Cubature Kalman Filters*. IEEE Transactions on Automatic Control.

[2] **For Adaptive Covariance Matching:** Myers, K.A., Tapley, B.D.: *Adaptive sequential estimation with unknown noise statistics*. IEEE Transactions on Automatic Control **21**(4), 520-523 (1976). (This is the original paper proving the sliding window Q estimation).

[3] Jeffrey K. Uhlmann Simon J. Julier. "New extension of the Kalman filter to nonlinear systems". In: Proc. SPIE 3068, Signal Processing, Sensor Fusion, and Target Recognition VI (July 1997). doi: https://doi.org/10.1117/12.280797 (cited on pages 283, 284, 337).

[4] Gordon, N. J., Salmond, D. J., & Smith, A. F. M. (1993). Novel approach to nonlinear/non-Gaussian Bayesian state estimation. IEE Proceedings F (Radar and Signal Processing), 140(2), 107-113. (Note: This is the seminal paper that introduced the Sampling Importance Resampling (SIR) algorithm, often considered the birth of the modern Particle Filter).

[5] Arulampalam, M. S., Maskell, S., Gordon, N., & Clapp, T. (2002). A tutorial on particle filters for online nonlinear/non-Gaussian Bayesian tracking. IEEE Transactions on Signal Processing, 50(2), 174-188. (Note: Highly recommended for the bibliography; it provides an excellent breakdown of weight degeneracy and the Effective Sample Size equation we detailed in Appendix N)

[6] Evensen, G. (1994). Sequential data assimilation with a nonlinear quasi-geostrophic model using Monte Carlo methods to forecast error statistics. Journal of Geophysical Research: Oceans, 99(C5), 10143-10162. (Note: The seminal paper where Geir Evensen originally invented and introduced the Ensemble Kalman Filter for high-dimensional oceanographic tracking).

[7] For the Original IMM Proof: Blom, H. A., & Bar-Shalom, Y. (1988). The interacting multiple model algorithm for systems with Markovian switching coefficients. IEEE Transactions on Automatic Control, 33(8), 780-783

[8] Cui, N., et al. (2016). An Improved Interacting Multiple Model Filtering Algorithm Based on the Cubature Kalman Filter for Maneuvering Target Tracking. Sensors, 16(5), 805.

[9] Zhang, Y., et al. (2023). An Interacting Multiple Model Algorithm Based On Cubature Kalman Filter. IEEE Xplore, Document ID: 10380078. Available: https://ieeexplore.ieee.org/document/10380078

[10] "State estimation with the Interacting Multiple Model (IMM) method," arXiv preprint, arXiv:2207.04875, 2022. Available: https://arxiv.org/pdf/2207.04875

[11] Rhudy, M., Gu, Y., & Gross, J. N. (2010). Interacting Multiple Model Adaptive Unscented Kalman Filters for Navigation Sensor Fusion. Proceedings of the International Council of the Aeronautical Sciences

(ICAS). Available: https://www.icas.org/icas_archive/ICAS2010/PAPERS/280.PDF

Books

[12] Simon, D. (2006). *Optimal State Estimation: Kalman, H∞, and Nonlinear Approaches*. John Wiley & Sons.

[13] **For NIS and Gating:** Bar-Shalom, Y., Li, X.R., Kirubarajan, T.: *Estimation with Applications to Tracking and Navigation*. (Chapter 5 directly covers Innovation Analysis, Validation Regions, and the Chi-Square statistics).

[14] **For Numerical Stability (Joseph Form):** Simon, D.: *Optimal State Estimation: Kalman, H∞, and Nonlinear Approaches*. (Chapter 6 covers roundoff errors, Joseph Form, and U-D factorization).

[15] Uhlmann Jeffrey. Dynamic map building and localization: new theoretical foundations, Thesis (Ph.D.). University of Oxford, 1995 (cited on pages 283, 320).

[16] Doucet, A., de Freitas, N., & Gordon, N. (2001). Sequential Monte Carlo Methods in Practice. Springer. (Note: The definitive textbook on particle filtering and Monte Carlo estimation).

[17] Evensen, G. (2009). Data Assimilation: The Ensemble Kalman Filter (2nd ed.). Springer. (Note: The primary textbook detailing the EnKF theory, specifically tailored for massively high-dimensional state spaces).

[18] For Practical Aerospace Implementations: Bar-Shalom, Y., Li, X. R., & Kirubarajan, T. (2001). Estimation with Applications to Tracking and Navigation: Theory Algorithms and Software. Wiley-Interscience. (Specifically Chapter 11, which covers GPB algorithms, IMM derivation, and the Coordinated Turn models).

[19] Becker, A. Introduction to Kalman Filter from the Ground Up. Available: https://www.kalmanfilter.net

Index